SPEAKER SCRIPT

Shor's Algorithm as Quantum Circuits

How factoring and the elliptic-curve discrete logarithm are built, run and costed: what to say on each of the 211 slides, and a focused one-hour talk.

2 October 2026

SAY what to say, in your own words

POINT TO where to send the eye on the slide

THEN the bridge to the next slide

CORE **OPTIONAL** keep or skip; times are speaking times

Contents

Your 60-minute talk: what to learn, what to say 3

1. Shor's algorithm is phase estimation of a group operation 3
2. Cost is counted in T gates, and a Toffoli costs 7 of them 4
3. Reversible arithmetic must clean up after itself 6
4. Measuring can clean up for free, if you measure the right way 7
5. Windowing: fewer additions, by looking the answer up 7
6. Elliptic-curve keys: the same circuit, a two-dimensional hidden period 7
7. From logical counts to real machines 8

What the optimisations in the repo bought 8

Part 1 — How Shor works (about 10.5 min) 10

Part 2 — What it costs (about 13.5 min) 12

Part 3 — How the cost was cut (about 10.5 min) 16

Part 4 — Elliptic-curve keys (about 11.5 min) 18

Part 5 — Where things stand in 2026 (about 9 min) 21

Overview: routes, timing and questions 25

Script 1 — Opening to Act VI (slides 1–39) 28

Opening 28

Act I — One unitary and one measurement 28

Act II — Which gates, and what they cost 30

Act III — Reversible arithmetic must clean up 32

Act IV — Building the factoring circuit 34

Act V — Simulation, and reading the answer 36

Act VI — Counting qubits, Toffolis and T gates 37

Script 2 — Act VII (slides 40–68) 39

Act VII — the optimisations, one idea each 39

Script 3 — Acts VIII and IX (slides 69–109) 47

Act VIII — The elliptic-curve discrete logarithm 47

Act IX — Why ECDSA falls before RSA 55

Script 4 — Acts X and XI (slides 110–163) 58

Act X — Where gates come from: Hamiltonians and hardware 58

Act XI — The 2026 frontier and the machines 64

Script 5 — Act XII and Closing (slides 164–211) 73

Act XII — The 2026 optimisations, built and measured 73

Closing 84

Backup slides (B1–B18) 85

Your 60-minute talk: what to learn, what to say

Start here. This part replaces the one-hour route in the overview. It takes 51 slides through the deck, of which 14 are marked "cut first": about 55 minutes of talking without them, 71 with every slide. Every slide on the route explains one of seven ideas, or shows what follows from one. First the seven ideas: what each one says, how to explain it, and where people get stuck (marked ⚠). Then the route, slide by slide. The one-hour deck, shor_talk60.pdf in shor/presentation, has exactly these slides in this order, with a divider for each part, then an appendix, “Optimisations in detail”, with every other optimisation slide of the full deck: diagrams, Qiskit code and measurements. Slide numbers are the full deck's, so they match this script and Scripts 2, 3 and 5; N1–N6 are new background slides and S0–S5 new summary slides.

How to spend the three hours

If the talk is part of your three hours, you have two hours to prepare:

Time	What to do
0:00–0:25	Read "The seven ideas" below. After each one, look away and say it aloud in two or three sentences.
0:25–1:05	Open the slides next to this script and go through the route slide by slide. Say each "Land" sentence aloud.
1:05–1:25	Practise the ⚠ explanations aloud until you can give each without notes. Ideas 1, 3, 4 and 6 matter most.
1:25–1:50	One timed run against the checkpoints below. Wherever you run over, drop the slides marked "cut first".
1:50–2:00	Learn your first and last minute word for word, then skim "Likely questions".

If you have three full hours before the talk, add a second timed run after 1:50 and stop by 2:30.

Checkpoints (without the cut-first slides): slide 39 at about 24 min · S2 at 34.5 min · slide 109 at 46 min · slide 211 at 55 min. If you are more than two minutes behind, skip the next "cut first" slide.

The seven ideas

They match the seven items on the closing slide (211), so the talk ends by naming what you explained.

1. Shor's algorithm is phase estimation of a group operation

The idea: factoring reduces to finding the period r of a^x mod N. A quantum computer finds periods by interference: it evaluates a^x for all x at once, and an inverse Fourier transform turns the repetition into a sharp peak.

How to explain it: the circuit makes the counting register "spin" at a fixed frequency s/r: branch x carries the phase e^(2πi·x·s/r), a phase ramp. The inverse QFT is a frequency meter: a pure tone of frequency φ goes in, the integer Q·φ comes out ($Q = 2^t$). So each run reads a random fraction s/r, as y/Q, and continued fractions turn y/Q back into s/r, and so into r.

⚠ **Where people get stuck:** "The work register starts in |1⟩, so where do the eigenvectors come from?" You never prepare one. |1⟩ is an equal mix of all r eigenvectors of "multiply by a", one for each s. Every run lands on one of them at random, and each gives its own peak at Q·s/r. Second one: "Why is this hard classically?" The period is a property of the whole sequence, not of any one value. Classically you have to compute many values of a^x and wait for a repeat, which is hopeless for a 2048-bit N.

⚠ **Phase kickback, the step people find hardest:** if U|ψ⟩ = e^(2πiφ)|ψ⟩, controlled-U sends (α|0⟩ + β|1⟩) ⊗ |ψ⟩ to α|0⟩⊗|ψ⟩ + β|1⟩ ⊗ e^(2πiφ)|ψ⟩ = (α|0⟩ + βe^(2πiφ)|1⟩) ⊗ |ψ⟩. Two rules do it. Linearity: U acts on each branch separately, and only the |1⟩ branch gets the phase. Bilinearity of ⊗: a number multiplying a product belongs to the product, not to either factor, (λa)⊗b = λ(a⊗b) = a⊗(λb), so the phase can be written on the control just as well as on the target.

What it means physically: on the target alone the phase is global, so it is invisible: the eigenstate is physically untouched. But it multiplies only one branch of the control, and a phase on one branch is a relative phase, which interference (the final QFT) can read. Nothing travels from target to control: the phase belongs to the pair, and the control is simply the factor that differs between the branches. Picture a Mach–Zehnder interferometer with a sample in one arm that the photon crosses undisturbed: the sample is unchanged, that arm picks up a phase, and the detectors see it. In hardware a controlled phase is an Ising ZZ coupling (slide 111), a joint energy shift of |11⟩ with no control or target at all. If |ψ⟩ were not an eigenstate, the target would keep a record of which branch happened, and the control's interference would shrink by the overlap ⟨ψ|U|ψ⟩: the same overlap rule as idea 3.

Two-qubit check: CNOT on |+⟩|−⟩ gives |−⟩|−⟩. The "target" is unchanged and the "control" flipped, because X|−⟩ = −|−⟩.

In Shor: |1⟩ is an equal mix of the eigenvectors |u_s⟩ of "multiply by a", with phases s/r. Rung j kicks e^(2πi·2^j·s/r) onto counting qubit j, and together the rungs build the ramp Σ_x e^(2πi·x·s/r)|x⟩ that the inverse QFT reads.

Carried by slides N2, 3, 4, N3, 5, 6, 33.

---

2. Cost is counted in T gates, and a Toffoli costs 7 of them

The idea: on an error-corrected computer the Clifford gates (H, S, CNOT) are cheap and the T gate is expensive, so every cost in the deck is a count of T gates or Toffolis.

How to explain it: error correction spreads one logical qubit over many physical ones. The safe gates act the same way on every physical qubit ("transversal"), so a single fault cannot spread. But no code makes a universal set of gates transversal (Eastin–Knill), and on the surface code the missing gate is T. It is done by consuming a "magic state" made in a factory, and the factories take a sizeable share of the machine; their output rate often sets how fast it runs. A Toffoli (a reversible AND: c ⊕= a·b) costs 7 T, or 4 when its target starts at 0. A small rotation has to be approximated from H and T, at about 60–100 T, which is the price of ten Toffolis.

⚠ **Where people get stuck:** "Why not count every gate?" Because next to a magic state a Clifford is almost free: the T count sets how many factories you need and how long the run takes.

Idea 2, unpacked term by term.

Physical qubit. An actual piece of hardware that holds one qubit: a superconducting circuit (a transmon), a trapped ion or a neutral atom. Every operation on it fails with a small probability, about 10^{-3} per gate on today's best machines.

Why error correction is needed. Shor at RSA or ECC size runs 10^8–10^{10} Toffolis. With errors of 10^{-3} per gate, the computation would be ruined after a few thousand gates, so errors have to be caught and fixed while it runs.

Logical qubit, error correction. The fix is to store one protected "logical" qubit in the joint state of many physical ones. The classical idea is the repetition code: store 0 as 000 and 1 as 111, and one flipped bit is outvoted. A quantum state cannot be copied (no-cloning), but $\alpha|0\rangle + \beta|1\rangle$ can be encoded as $\alpha|000\rangle + \beta|111\rangle$. Then you measure parities: "are qubits 1 and 2 equal?", "are 2 and 3 equal?". These checks show where an error happened without revealing α and β, so they don't disturb the computation. A classical decoder reads the check results (the *syndrome*) and works out the correction. A real code must also catch phase flips, not only bit flips.

Distance, threshold. A code's distance d is the smallest number of physical errors that can change the logical state without being caught; the code corrects any $(d - 1)/2$ errors. If the physical error rate is below a threshold (about 1% for the surface code), each step up in d divides the logical error rate by a roughly constant factor, so the logical error falls exponentially with d. That is why a long Shor run at $p = 10^{-3}$ needs $d \approx 27$.

Surface code. The leading code. It is a $d \times d$ square patch of data qubits on the chip, with measurement qubits in between. Each measurement qubit checks the parity of its four neighbours, alternating between bit-flip (Z) checks and phase-flip (X) checks, so both kinds of error are caught. It needs only nearest-neighbour gates and costs about $2d^2$ physical qubits per logical qubit: around 1,500 at $d \approx 27$.

Fault, fault tolerance. A fault is one component going wrong: one gate, one measurement or one idle qubit. A circuit is fault tolerant if any single fault leaves at most a correctable error in each code block. The danger is spreading: a CNOT copies a bit flip from its control to its target, so one faulty gate between two qubits of the same block can turn one error into two.

Transversal gate. A logical gate done by applying the same physical gate to each qubit separately, or pairwise between the i-th qubits of two blocks, but never between two qubits of the same block. One fault then touches at most one qubit per block, and the code corrects it. Transversal gates are the safe, cheap gates.

Clifford gates. The gates generated by H, S (a quarter turn, $\sqrt{Z}$) and CNOT. They are the easy gates of error correction: on many codes they are transversal or nearly so, and on the surface code they are done cheaply by lattice surgery or tracked in software. But a circuit of Clifford gates alone can be simulated efficiently on a classical computer (the Gottesman–Knill theorem), so it cannot run Shor. One more gate is needed.

T gate. $T = \mathrm{diag}(1, e^{i\pi/4})$: an eighth of a turn about z, the square root of S. Clifford + T is universal: any quantum operation can be approximated with H, S, CNOT and T.

Eastin–Knill theorem (2009). No error-detecting code has a universal set of transversal gates. The intuition: a universal set reaches gates arbitrarily close to the identity, and a tiny rotation looks to the code just like a tiny error, so a code that let such rotations through untouched could not be detecting errors. At least one gate of every universal set must be done another, more expensive way. On the surface code that gate is T.

Magic state, gate teleportation. The other way is to prepare a special state $|T\rangle = T|+\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ on a separate qubit. Then a CNOT, a measurement and, half the time, an S correction (all Clifford) apply T to the data qubit. The hard part moves from the gate to a resource state, which can be made in advance, tested, and thrown away if it is bad.

Factory, distillation, cultivation. Magic states come out noisy, so they are purified. Distillation takes 15 noisy copies with error p and returns one with error about $35p^3$ (the 15-to-1 protocol of Bravyi and Kitaev), repeated in rounds if needed. A factory is a region of the chip that does this non-stop.

Cultivation (2024) grows a good magic state inside a single patch, for about ten times less. Many designs make a CCZ state directly, one per Toffoli. Factories take a sizeable share of the machine (7% of IonQ's ECDLP blueprint, much more in older designs). Their output rate often sets the speed, because every T gate waits for a magic state.

Toffoli, reversible AND, c ⊕= a·b. The Toffoli flips its third qubit c exactly when a and b are both 1: |a, b, c⟩ → |a, b, c ⊕ ab⟩. Here ⊕ is XOR (addition mod 2), and "c ⊕= a·b" is shorthand for "c becomes c XOR (a AND b)", like `c ^= a & b` in code. It is reversible, since applying it twice undoes it. Together with NOT it can build any classical function reversibly, so it is the building block of all the arithmetic in Shor.

7 T, or 4. Written in Clifford + T, a Toffoli needs 7 T gates (the minimum without extra qubits), or one CCZ magic state. When the target starts in |0⟩, computing a·b into it (a *temporary* AND) needs only 4 T. Undoing it later costs no T at all: an X-basis measurement and a classically controlled CZ (idea 4).

Small rotation, synthesis. Some circuits need rotations by arbitrary angles: the QFT's controlled phases by $\pi/2^k$, and Draper's Fourier adders. Clifford + T makes exactly only multiples of $\pi/4$, so any other angle must be approximated by a sequence of H, S and T, to an accuracy ε. The best methods (Ross–Selinger) need about $3\cdot\log_2(1/\varepsilon)$ T gates, which is 60–100 T for ε between about 10^{-6} and 10^{-10}. The more rotations a circuit has, the smaller each one's share of the error budget, so the more T each one needs.

The price of ten Toffolis. At 7 T per Toffoli, 60–100 T is the cost of 9–14 Toffolis. So one small rotation costs about as much as ten Toffolis. A circuit with many rotations, such as the textbook Fourier arithmetic, pays most of its bill in rotations: 98.5% of the T count at N = 15. That is why the deck moves to Toffoli arithmetic.

Carried by slides N4, N5, N6, 11, 12, 13, 17, 39.

---

3. Reversible arithmetic must clean up after itself

The idea: the box inside the circuit is classical arithmetic made reversible. Any scratch value left behind that depends on x destroys the interference, so every ancilla must be back at |0⟩ before the QFT.

How to explain it: interference needs the branches x and x′ to be identical everywhere outside the counting register. If a scratch register still holds something that depends on x, the branches can be told apart, and the effect is the same as measuring x: the peaks vanish and the output is flat. Bennett's fix is compute, copy the answer out, then run the computation backwards. That doubles the cost, and the doubling runs through the whole deck: multiply, swap, un-multiply (slide 28); the second division in a point addition (slide 77).

⚠ **Where people get stuck:** "We never measure the garbage, so why does it matter?" The problem is not measuring; it is that the branches *could* be told apart. A register that records x is a which-path record, like a detector at one slit of the double-slit experiment.

Why, in one formula: after the inverse QFT, $P(y) = (1/Q^2)\ \Sigma$ over x, x′ of $\omega^{-(x-x')y} \cdot \langle \text{rest}(x') | \text{rest}(x) \rangle$. Every interference term ($x \neq x'$) is multiplied by the overlap of everything outside the counting register. Clean scratch gives overlap 1 and the peaks survive. Scratch that holds x gives overlap 0, every cross term dies, and P(y) is flat at 1/Q, which is exactly what measuring x would give. Nothing in the formula asks whether anyone reads the register.

Whiteboard example (two qubits): $H|0\rangle$ gives $(|0\rangle+|1\rangle)/\sqrt{2}$, and a second H gives |0⟩ every time: the two paths interfere. Copy x into a scratch qubit first: the CNOT gives $(|00\rangle+|11\rangle)/\sqrt{2}$, and H on the first qubit now gives $\tfrac{1}{2}(|00\rangle+|10\rangle+|01\rangle-|11\rangle)$, so it reads 0 or 1 at 50/50, although the scratch was never measured. A second CNOT (the uncompute) restores $(|0\rangle+|1\rangle)|0\rangle$, and H gives |0⟩ again.

Why the output register f(x) is harmless: it records only x mod r. Branches one period apart still overlap, and those are exactly the pairs whose interference makes the peaks at multiples of Q/r.

Carried by slides 18, 19, 21, 28.

---

4. Measuring can clean up for free, if you measure the right way

The idea: instead of running a computation backwards, measure the scratch register in the X basis and remove the small phase that is left. Most of the 2019–2026 savings rest on this trick.

How to explain it: measuring a qubit in the X basis gives a random bit m that says nothing about the value the qubit held. All it leaves is a sign, (−1)^(m·value), on each branch. You know m, so you can remove that sign with cheap gates: a classically controlled CZ for an AND, a "phase lookup" for a table. So the temporary AND costs 4 T to compute and nothing to uncompute (instead of 7 + 7), and undoing a table lookup costs about √L instead of L.

⚠ **Where people get stuck, and the most likely question of the talk:** "You just said garbage destroys interference, and now you measure it?" The difference is the basis. A measurement in the computational (Z) basis reads the value, so it reveals x and kills the peaks. An X-basis measurement reads a coin flip that is independent of x, so nothing learns which branch you are in. The only trace is a phase you know, and you undo it. Practise this one aloud.

Carried by slides 51, 59.

---

5. Windowing: fewer additions, by looking the answer up

The idea: instead of one controlled addition per bit, take w bits at a time, look up the right multiple in a precomputed table, and add once per window.

How to explain it: "add k if bit q is 1" is the same as "add T[q]" from the two-entry table [0, k]. Take w bits as the address and the table has 2^w entries, which a classical computer fills in before the run, so n additions become n/w. A lookup costs about 2^w Toffolis, so w is chosen to balance the lookups against the additions they save. The 2026 secp256k1 estimates use 16-bit windows and about 28–33 windowed point additions in total.

⚠ **Where people get stuck:** "A table of 2^{16} entries is exponential." It is a fixed 65,536 Toffolis per lookup, against about a million for the point addition it saves; that balance is exactly how w is chosen.

Carried by slides 53, 57, 59, 94.

---

6. Elliptic-curve keys: the same circuit, a two-dimensional hidden period

The idea: an ECDSA key k, with public point Q = [k]P, hides as a period in two dimensions. The same phase-estimation circuit finds it. Only the box changes: add a point instead of multiplying by a number, and adding points needs a modular division.

How to explain it: let P have prime order r and Q = [k]P. Define f(u,v) = [u]P + [v]Q = [u + kv]P. Then f is constant exactly on the cosets of the hidden subgroup $L = \{(a,b) : a + kb \equiv 0 \bmod r\} = \langle(-k, 1)\rangle$. In other words f(u − k, v + 1) = f(u, v): one more Q is k more copies of P. Fourier sampling returns (s,t) in $L^\perp$, the pairs with $t \equiv ks \bmod r$, so $\mathbf{k = t \cdot s^{-1} \bmod r}$ for any $s \neq 0$, then check [k]P = Q. Because r is public and prime, this is one modular division, with no continued fractions. The circuit has two counting registers of about n bits each, and the oracle is about 2n controlled additions of the known points 2^jP and 2^jQ into an accumulator point; with w-bit windows, about 2n/w additions of looked-up points.

Toy check (slide 71): r = 5 and k = 4, so $L = \langle(1,1)\rangle$. A shot (s,t) = (2,3) gives $k = 3 \cdot 2^{-1} = 3 \cdot 3 = 9 \equiv 4 \bmod 5$.

Why it is cheaper than RSA: the cost grows as the cube of the key length, and at 128-bit security the key is 12 times shorter (256 bits against 3072). That is $12^3 \approx 1{,}700$ times fewer operations, divided by a point addition about 7 times heavier: about 150–270 times fewer Toffolis.

⚠ **Where people get stuck:** (a) "Is f periodic?" Yes, but it has a whole lattice of periods instead of one number. Steps of r along each axis are periods too, but they carry no information because r is public; the step that hides the key is (−k, 1). And "exactly" matters: different cosets give different points, which is why the measurement lands only on $L^{\perp}$. (b) "Why is a point addition so expensive?" The slope $\lambda = (y_2 - y_1)/(x_2 - x_1)$ is a modular division. Done reversibly it is a long extended-Euclid loop of conditional subtractions and halvings, and it runs twice, once to make λ and once to clear it: 80% of the cost. (c) "Real registers have 2^m states, not r." Then the peaks sit near $s \cdot 2^m / r$: round, divide, and let several shots vote (slide 76).

Carried by slides 69–73, 76, 77, 79, 94, 107, 109.

---

7. From logical counts to real machines

The idea: circuits are counted in logical qubits and Toffolis. A machine needs many physical qubits for each logical one, and the runtime is the Toffoli count times the time per Toffoli.

How to explain it: a surface-code logical qubit at distance d uses about $2d^2$ physical qubits. With physical errors of 10^{-3} a long Shor run needs $d \approx 27$, so about 1,500 physical qubits per logical one, plus the magic-state factories. Where things stand in 2026: ECDLP-256 needs about 1,200–1,500 logical qubits and $4–9 \times 10^7$ Toffolis. Google's design uses under 500,000 superconducting qubits for 18–23 minutes; designs for ions or neutral atoms use about 20,000–26,000 qubits but run for days to weeks, because their clocks are about 1,000 times slower. RSA-2048 needs just under a million qubits for about five days (Gidney 2025). The deck's own contribution (Act XII): one point addition built at 1,457 qubits and 1.31 million Toffolis, below IonQ's published 1.39 million.

⚠ **Where people get stuck:** "So when does Bitcoin break?" Don't give a date. The estimates are for machines that don't exist yet: the processors in Act X have about 100 good qubits (Willow 105, Helios 98). The cheapest designs also depend on what is least tested: long-range connections, real-time decoding and parallel lattice surgery. What is certain: elliptic curves fall before RSA, and traffic recorded today can be decrypted later.

Carried by slides N4, 157, 164, 207, 208, 206, S0, 211.

What the optimisations in the repo bought

This is the text behind summary slides S0–S5; use it when someone asks what the repo adds. Every number is a count the repo measured on the circuits it built.

Factoring. The repo starts from Beauregard's textbook circuit and adds twelve optimisations. Each one is a switch, and each is measured against the circuit without it.

- **Ripple-carry adders** give the biggest gain. They replace Draper's Fourier adders with Toffoli adders that need no small rotations, which cuts the T count by 95% at n = 4 and by 99% at n = 12.
- **The semiclassical QFT** replaces the 2n-qubit counting register with one reused qubit, so the width falls from 4n + 2 to 2n + 3. That is −41% at N = 21 (22 → 13 qubits) and −50% at RSA-2048 size.
- **Combined**, these two take the N = 15 circuit from 18 qubits and 978,000 T to 17 qubits and 42,600 T (−96% T).
- **Placing controls** only on the blocks that don't cancel cuts CNOTs per modular adder by 92% (2,634 → 206). Controlling only the gates that write the target cuts controlled gates by 67% (6n → 2n).

- **The temporary AND** brings a compute–uncompute pair from 14 T to 4 (−71%).
- **Windowing** replaces n controlled additions with ⌈n/w⌉ table lookups and additions. At N = 255 and w = 4 that is 75% fewer additions, 56% fewer CNOTs and 70% less depth, for 12 extra qubits (+63%).
- **Unary iteration** then cuts the lookup's ancillas from 59 to 31 (−47%). **Measured uncomputation** cuts the cost of undoing a lookup from 4,094 to 157 Toffolis at w = 11 (−96%).
- **The smaller ones:** nested windows halve the multiplications (8 → 4 at N = 15), and the coset representation removes the modular reduction for 67–74% fewer Toffolis per addition. The approximate QFT drops 34% of the rotations at n = 8.

Elliptic curves, toy sizes. The gate-level tricks add up:

- temporary ANDs inside the modular inversion: −60% T;
- Gidney's adder with copy-then-add: a controlled addition goes from 128 to 63 Toffolis (−51%);
- pseudo-Mersenne doubling: 25 → 4 Toffolis (−84%);
- the Euclidean dialog with replay: −54% Toffolis and −43% qubits;
- 16-bit windows with a zig-zag schedule: 514 → 34 point additions (−93%).

Elliptic curves, full size (n = 256, secp256k1). The repo builds the two 2026 eprints as printed, at 6.28M Toffolis per windowed point addition. Three changes bring that down to 1.72M (−73%):

- measured uncomputation of the lookups: 1.31M → 197k (−85%);
- IonQ's conditionally inverted GCD: each in-place multiplication goes from 1.96M to 0.72M (−63%);
- a dedicated squarer with a pseudo-Mersenne fold: 1.05M → 79k (−92%).

Further steps:

- IonQ's "carry is the flag" modular adder halves the cost of each GCD replay step (712 → 352 Toffolis, −51%), about −16% per addition.
- Signed windows halve every table and take off another 7%.
- Freeing scratch at whichever step sets the peak brings the width from 2,233 to 1,457 qubits (−35%) without adding a single Toffoli.

The result is 1,310,693 executed Toffolis per addition on 1,457 qubits, 5.9% below IonQ's published count. The whole ECDLP-256 comes to 37.1M Toffolis, against 213M as printed (−83%). There are also trade-offs along other axes:

- lean cells reach 1,246 qubits (−45%) at the cost of 65% more Toffolis;
- three field registers reach 851 qubits, at 85.2M Toffolis per addition;
- carry-lookahead adders cut Toffoli depth by 74% (1.50M → 0.39M), for 4.6× the Toffolis.

The route, slide by slide

For each slide: what to explain, the sentence to land, and, where the slide uses a word the talk has not yet introduced, a *Define* line to say. Part times leave out the slides marked "cut first": about 55 minutes in all, 71 with every slide. Slide numbers are the full deck's; N1–N6 are the new background slides and S0–S5 the summary slides.

Part 1 — How Shor works (about 10.5 min)

N1 The road map for this hour — 1 min

EXPLAIN five parts, each answering one question with only what came before: how Shor works (period finding), what it costs (error correction, T gates, Toffolis, clean-up), how the cost was cut, elliptic-curve keys and why they fall first, and where things stand in 2026. Two background slides fill the gaps: qubits and phases now, error correction at the start of Part 2.

LAND **"Almost all of Shor's cost is reversible arithmetic, so that is what we build, run and count."**

N2 Background: qubits, gates and phases — 1.5 min

EXPLAIN the minimum notation. A qubit is $\alpha|0\rangle + \beta|1\rangle$, and measuring gives 0 or 1 with probabilities $|\alpha|^2$ and $|\beta|^2$; n qubits hold an amplitude for every n-bit integer. The gates on the right are all we use: H makes and undoes superpositions, CNOT XORs one qubit into another, controlled-U applies U only where the control is 1. The QFT writes an integer as a phase ramp, and its inverse reads a ramp back. Stress the phase line: global phases are invisible, relative phases are what interference reads.

LAND **"Gates act on every branch at once; the answer comes from interference, phases arranged so wrong answers cancel."**

3 Factoring reduces to finding a period — 2 min

EXPLAIN the powers of 7 mod 15 are 1, 7, 4, 13, 1, so r = 4. If r is even, $a^r - 1 = (a^{(r/2)} - 1)(a^{(r/2)} + 1)$ is a multiple of N, so each factor shares a divisor with N (unless $a^{(r/2)} \equiv -1$). Here $7^2 = 49$, its neighbours 48 and 50 give gcd 3 and gcd 5.

LAND **"The period belongs to the whole sequence, not to any single value, and finding it is the only step that needs a quantum computer."**

DEFINE The slides still carry the old "Act ..." and "Difficulty N" labels (e.g. "Act VII" on 17 and 39, "Act VIII" on 164) — translate aloud each time: Act VII = Part 3, Acts VIII–IX = Part 4, Act XII = Part 5.

4 Phase estimation: the circuit

1.5 min

EXPLAIN a counting register of 2n qubits, each put in superposition by a Hadamard, and a work register holding 1. Counting qubit i controls "multiply by a^(2^i) mod N", with the powers computed classically. Then an inverse QFT and one measurement, y.

LAND **"One fixed circuit pattern; only the box inside it changes." (Idea 1)**

DEFINE U_a is never defined on the slide — "U_a multiplies the work register by a modulo N, U_a|y⟩ = |ay mod N⟩; rung i applies it 2^i times, which is one multiplication by a^(2^i) mod N, computed classically."

DEFINE QFT/QFT† first appears here and is not in N2 — "the quantum Fourier transform sends |x⟩ to the equal superposition of all |y⟩ with phases e^(2πi·xy/Q), a phase ramp of slope x/Q; QFT† turns such a ramp back into the integer."

DEFINE Why t = 2n is never justified on the route (slide 36 skipped) — "2n counting qubits, so $2^t \geq N^2$, give enough precision for y/2^t to pin down s/r exactly."

N3 Phase kickback: how the control learns the phase

1.5 min ⚠ idea 1

EXPLAIN the step people find hardest (idea 1). The target is an eigenvector, so U only multiplies it by a phase, and only in the branch where the control is 1. A number multiplying a product belongs to neither factor, so write it on the control: there it is a relative phase, which interference can read. Show the two-qubit check: CNOT on |+⟩|−⟩ flips the "control". Then the takeaway: each rung adds one bit of the ramp.

LAND **"The data register stays put; the counting register turns."**

5 Why the measurement reveals s/r

2 min ⚠ idea 1

EXPLAIN "multiply by a" has eigenvectors with phases s/r, and |1⟩ is an equal mix of them. Kickback: each controlled multiplication leaves the eigenvector alone and writes its phase onto the control qubit, so the counting register becomes a phase ramp. The inverse QFT turns a ramp of slope φ into a peak at Q·φ. Use the frequency-meter picture.

LAND **"Each run reads a random fraction s/r; continued fractions turn it into r."**

DEFINE |u_s⟩ is used without a definition — "|u_s⟩ is the equal superposition of the r values a^k mod N with phases e^(−2πi·sk/r); multiplying by a shifts k by one, so it picks up e^(2πi·s/r), and averaging over s gives back |1⟩."

DEFINE "They do not interfere" is too compressed — "the |u_s⟩ are orthogonal on the work register, so each s makes its own peak and the probabilities simply add; s comes out uniformly at random."

6 QFT† in action: a ramp becomes an integer

1 min

EXPLAIN Make slide 5 concrete with the smallest case: three counting qubits, so Q = 8, and s/r = 1/4. After kickback, branch x carries the phase e^(2πi·x/4) = i^x, a quarter turn per step: 1, i, −1, −i, 1, … That ramp is exactly QFT|2⟩, the quantum Fourier transform of the basis state 2, whose phases turn by 2/8 of a circle per step. So QFT† undoes it and returns |2⟩ with certainty. Divide by Q: y = 2 gives s/r = 2/8 = 1/4, the ramp's slope read as an integer.

LAND **"The inverse QFT turns a phase ramp into one integer: every shot gives y = 2, and 2/8 = 1/4 is the ramp's slope."**

33 N = 15: four sharp peaks

1.5 min CUT FIRST

EXPLAIN 4,096 shots with t = 8: peaks at 0, 64, 128 and 192, the multiples of 256/4. A shot of 64 gives 64/256 = 1/4, so r = 4. The samples match the exact distribution to within shot noise.

LAND **"The peaks sit at multiples of Q/r, here 256/4 = 64."**

DEFINE First mention of Aer, and the circuit it runs is only built in Part 2 — "Aer is Qiskit's simulator; this is the full N = 15 circuit we build in Part 2, shown now so you see the payoff first."

DEFINE The route never shows how r is read from a peak (slides 34–36 skipped) — "peaks sit at y ≈ 2^t·s/r: with t = 8, y = 64 gives 64/256 = 1/4 and y = 192 gives 3/4, so r = 4, then gcd(48, 15) = 3 and gcd(50, 15) = 5 as on slide 3; in general, continued fractions do this reduction."

Part 2 — What it costs (about 13.5 min)

N4 Background: error correction in one slide

2 min ⚠ idea 2

EXPLAIN why the cost model has T gates in it. Physical qubits fail about once per thousand gates, so a logical qubit is stored in many physical ones, and parity checks find errors without reading the data. Below a threshold of about 1%, the logical error falls exponentially with the code distance d; at 10^{-3}, d ≈ 27, about 1,500 physical qubits per logical one in the surface code (the picture). Transversal gates act qubit by qubit, so a fault stays one error: that makes H, S and CNOT cheap. T is the gate left over.

LAND **"Error correction makes Clifford gates cheap and leaves one gate expensive: T."**

N5 The surface code: parity checks, errors and the decoder

1.5 min ⚠ idea 2

EXPLAIN the picture is a d = 5 logical qubit: dots are data qubits, each square is a check. Every microsecond each square measures the parity of its four corners: blue Z-squares catch bit flips, red X-squares phase flips, and no check reveals the stored state. The bit flip in the centre lights the two blue squares beside it; a chain of errors lights only its two ends, and a classical decoder pairs the lit squares to find the correction. The stored qubit is a string across the whole patch (purple), so corrupting it takes d errors in a line. Below the 1% threshold each step d → d + 2 divides the logical error by about 10 at $p = 10^{-3}$.

LAND **"To corrupt the data an error chain has to cross the whole board; making the board bigger makes that exponentially unlikely."**

N6 From one patch to a computer

1 min CUT FIRST

EXPLAIN how big and how it computes. A Shor run has about 10^3 logical qubits for 10^9–10^{10} cycles, so it needs a logical error near 10^{-15} per cycle: with a factor 10 per step that is d ≈ 27, about 1,500 physical qubits per logical one. Google's Willow (2024) measured the logical error halving at each step d = 3 → 5 → 7. Gates: X and Z are tracked in software; a CNOT is lattice surgery, two patches merged along an edge for about d cycles; T needs a magic state from a factory.

LAND **"A machine is a floor of patches, memory, routing and factories, and the Toffoli count sets the runtime."**

11 Why the T gate is the expensive one

1.5 min ⚠ idea 2

EXPLAIN transversal gates, Eastin–Knill, T by magic states (idea 2).

LAND **"Error correction has to make one gate expensive, and here that gate is T."**

DEFINE The T gate is first named here (slide 10 skipped) — "T = P(π/4) = diag(1, e^(iπ/4)); Clifford gates plus T are universal, and T is the one that cannot be done transversally."

DEFINE The takeaway cites Solovay–Kitaev (slide 16 skipped) — "any one-qubit gate can be approximated to error ε by a Clifford+T sequence whose length grows only like a power of log(1/ε)."

12 Magic states: T by teleportation

1 min CUT FIRST

EXPLAIN Slide 11 said T cannot be done transversally; this is how it is done instead. Prepare a magic state $|T\rangle = T|+\rangle$, where $|+\rangle = H|0\rangle$. CNOT from the data onto it, then measure it in Z: outcome 0 leaves $T|\psi\rangle$, outcome 1 leaves $T^\dagger|\psi\rangle$, and an S gate fixes that, since $S \cdot T^\dagger = T$. CNOT, measurement and S are all Clifford, so each T costs one fresh $|T\rangle$. Factories, regions of the chip, make them: distillation (2005) turns 15 noisy copies into 1 with error $\approx 35\varepsilon^3$, ε being each copy's error; cultivation (2024) is ≈10× cheaper.

LAND **"Every T gate costs one magic state; everything around it is Clifford and nearly free."**

13 The Toffoli: a reversible AND

1 min

EXPLAIN Introduce the gate all the arithmetic is made of. The Toffoli is a doubly controlled NOT: it flips c when a and b are both 1, so it writes $c \oplus ab$, an AND that loses nothing. On basis states it only swaps $|110\rangle$ and $|111\rangle$ and fixes the other six; a permutation of basis states is unitary, and applying it twice gives $c \oplus ab \oplus ab = c$, so it is its own inverse. In Clifford+T it costs exactly 7 T gates, so a classical function built from Toffolis and CNOTs has an exact T price.

LAND **"Reversible arithmetic is Toffolis plus CNOTs, and its T cost is exactly seven times the number of Toffolis."**

17 The price list

1.5 min

EXPLAIN Clifford 0, T 1, Toffoli 7 (4 as a temporary AND), small rotation 60–100 T. Slide 13 is skipped, so say here that a Toffoli is a reversible AND.

LAND **"One small rotation costs about as much as ten Toffolis, so we build the arithmetic from Toffolis."**

DEFINE "Small rotation" and its $\approx 3\log_2(1/\varepsilon)$ price come from skipped slides 15–16 — "a small rotation is a phase gate $P(\theta)$ with θ not a multiple of $\pi/4$; Clifford+T cannot make it exactly, so it is approximated to error ε with about $3\log_2(1/\varepsilon)$ T gates, and with R of them in a circuit each gets ε/R of the error budget."

DEFINE "4 as a temporary AND, Act VII" — say "four, with the temporary AND we meet in Part 3".

18 Act III — arithmetic leaves scratch values behind

1 min

EXPLAIN A quantum circuit cannot erase a bit, because erasing is irreversible. So when we compute f(x) = a^x mod N from Toffolis, every intermediate result (carries, flags, partial products) is written into an ancilla, a scratch qubit that started in $|0\rangle$, and stays there. Call everything left behind the garbage, g(x). In Shor, x is the counting register in superposition, so the state is a sum over all $Q = 2^t$ values of x of $|x\rangle|f(x)\rangle|g(x)\rangle$: the garbage is entangled with x. Then ask: if we never measure the garbage, does it matter?

LAND **"Reversible arithmetic always leaves scratch values behind, and in Shor they are entangled with the counting register."**

19 Garbage destroys the interference: the formula

2 min ⚠ idea 3

EXPLAIN P(y) depends on the overlaps of the other registers. Garbage that depends only on f(x) keeps the peaks; garbage that records x makes P(y) flat at 1/Q. Point at the two underbraces, then the grey box against the peach box.

LAND **"Garbage that records x is a which-path measurement: the peaks disappear."**

21 Bennett's fix: compute, copy, uncompute

1.5 min

EXPLAIN run C, copy the answer into a clean register, run C backwards. About twice the gates, and the pattern comes back all through the talk.

LAND **"Cleaning up doubles the cost, and that factor of two runs through every count in the talk."**

23 Five levels, each a product of the one below

1 min CUT FIRST

EXPLAIN This is the build plan for the box U_a, bottom to top. L0 is the QFT; L1 adds a classical constant X through it; L2 makes that addition modulo N in seven blocks: add X, subtract N, copy the sign into a flag, add N back if set, then subtract X, clear the flag, add X; L3 multiplies by a as n controlled modular additions of $a \cdot 2^i$; L4 makes it in place; L5 is the whole of Shor. Each level is checked on every input, every ancilla back to $|0\rangle$, before the next uses it.

LAND **"Each level is built from the one below and verified on every input before anything is built on top of it."**

26 L1 — Draper's adder: addition is a phase

1 min CUT FIRST

EXPLAIN Show that in the Fourier basis, addition needs no carries. After a QFT on an m-qubit register holding y, qubit i carries the phase $e^{\wedge}(2\pi i \cdot y \cdot 2^i / 2^m)$. Adding a constant X multiplies it by $e^{\wedge}(2\pi i \cdot X \cdot 2^i / 2^m)$: one phase gate P per qubit, nothing passed between qubits. QFT† returns y + X as an ordinary number. The figure adds X = 3 on 3 qubits, checked equal to $y \mapsto y + 3 \bmod 8$. To control the adder, control only the phases; with the control off, the QFT and QFT† cancel. These phases are the small rotations of the price list.

LAND **"In the Fourier basis adding a constant is one phase per qubit with no carries, but each phase is a small rotation, and that is what we pay for."**

28 L3–4 — multiply, swap, un-multiply

1.5 min CUT FIRST

EXPLAIN phase estimation needs $|x\rangle \to |ax\rangle$ in place. Shift-and-add writes ax into a fresh register; swap the two; then add $-a^{-1}$ times the new value to wipe out the old x. Mention the levels underneath: QFT, adder, modular adder, multiplier (slide 23, skipped).

LAND **"The second multiplication exists only to clean up."**

DEFINE a^{-1} and the controlled SWAP are new — "a^{-1} mod N is the number with $a \cdot a^{-1} \equiv 1 \pmod N$, computed classically, and it exists because gcd(a, N) = 1; the controlled SWAP (a Fredkin gate) exchanges the two registers only when the rung's control is 1."

39 At RSA-2048 size

1.5 min

EXPLAIN the textbook circuit at RSA-2048: 8,194 qubits and 6.9×10^{10} Toffolis, with 10^{16}–10^{17} T, mostly from rotations. The state of the art: Gidney–Ekerå 2021 (6,189 qubits, 2.7×10^9 Toffolis), Gidney 2025 (1,399 qubits, 6.5×10^9), which is under a million physical qubits for about five days.

LAND **"Polynomial but enormous, and mostly rotations, so the first thing to remove is the rotations."**

DEFINE The formulas behind 8,194 qubits and 6.9×10^{10} Toffolis (slide 37) are skipped — "$8{,}194 = 4n + 2$ at $n = 2048$: $2n$ counting qubits, n target, n scratch, a sign and a flag; the Toffolis grow as $8n^3$."

DEFINE "Approximate QFTs" and "approximate residue arithmetic" are undefined — "an approximate QFT drops the controlled rotations too small to matter; Gidney's residue arithmetic works modulo many small primes instead of the 2048-bit N, keeping only an approximate result."

Part 3 — How the cost was cut (about 10.5 min)

47 Toffoli arithmetic: the ripple-carry adder — 1.5 min

EXPLAIN the rotations in the arithmetic come from the Fourier adder, so replace it. Cuccaro's ripple-carry adder runs MAJ blocks that carry from the low bits up, then UMA blocks that come back down and write the sum. Each block is one Toffoli and two CNOTs, so an m-bit adder is 2m Toffolis, one carry qubit and no rotations at all. The modular adder's seven blocks stay the same; only the adder inside changes. Measured: T −95% at n = 4, −99% at n = 12.

LAND **"Toffoli arithmetic has a small, exact T count; Fourier arithmetic pays for synthesis on every rotation."**

DEFINE "The seven blocks of the modular adder" were never shown (slide 27 skipped) — point back to slide 23's L2 line: "the same add / subtract N / flag / add back / clean-up blocks; only the adder inside each block changes."

DEFINE MAJ and UMA are only names on the slide — "MAJ computes the carry (the majority of three bits) in place on the way up; UMA ('UnMajority and Add') undoes it on the way down while writing the sum bit."

43 One counting qubit: the semiclassical QFT — 1.5 min

EXPLAIN every rotation in the inverse QFT is controlled by a qubit that is about to be measured. So measure it first and let the classical bit decide whether to rotate: the output does not change. The counting register becomes one qubit, reused 2n times; qubits drop from 4n + 2 to 2n + 3.

LAND **"Measure early and control classically: 2n counting qubits become one."**

DEFINE QFT†'s gate structure (slide 24) was skipped, so "every rotation of QFT† is controlled by a qubit about to be measured" lands cold — "QFT† is Hadamards plus controlled phase rotations, each controlled by a counting qubit that is measured right after; measure it first and apply the rotation only if the bit came out 1."

DEFINE "4n + 2 → 2n + 3" is never derived on the route — "4n + 2 is 2n counting + n target + n scratch + sign + flag; one recycled counting qubit replaces all 2n, leaving 2n + 3."

51 The temporary AND: uncomputation for free — 2 min ⚠ idea 4

EXPLAIN compute an AND into a fresh $|0\rangle$ for 4 T. Uncompute it by measuring in the X basis and applying a CZ only if the outcome is 1: no T at all. A compute–uncompute pair falls from 14 T to 4.

LAND **"An X-basis measurement reveals nothing about the inputs, so it can clean up for free."**

DEFINE It prices "a table lookup" before lookups exist (53 and 57 come after) — "say 'windowing, next, turns additions into table lookups, and lookups are made of these compute–uncompute AND pairs'"; an X-basis measurement is a Hadamard followed by an ordinary measurement.

53 Windowing in three steps

2 min ⚠ idea 5

EXPLAIN a controlled addition is a lookup; address w bits at once and add once per window; the outer control is one more address bit, so the adder itself needs no control.

LAND **"Look the answer up and add once per window: n additions become n/w."**

DEFINE T here is a table, not the T gate, and "the outer control" is new wording — "say 'the table T' aloud: a classical list of numbers; the outer control is the rung's control qubit c."

57 Making the lookup cheap: unary iteration

1 min CUT FIRST

EXPLAIN Windowing needs a table lookup, or QROM (quantum read-only memory): from an address register in superposition, write the classical entry T[address] into an output register. The one-hot version holds all $L = 2^m$ address states on 2^m ancillas. Unary iteration walks the address tree, keeping only the AND of the current prefix: a node is its parent AND one address bit, its sibling costs one CNOT, each leaf loads T[j] with CNOTs. Measured: exactly $2(L - 1)$ Toffolis on m ancillas, $L - 1$ with temporary ANDs; at $N = 255$, $w = 4$, 59 qubits → 31.

LAND **"Unary iteration loads a table of L entries with L − 1 Toffolis on just m ancillas instead of 2^m."**

59 Uncomputing a lookup for free

1.5 min

EXPLAIN measure the lookup's output in the X basis. That leaves a phase $(-1)^{(m \cdot T[a])}$ on the address, which a phase lookup repairs in about $\sqrt{L}$ steps. At $w = 11$ the clean-up falls from 4,094 Toffolis to 157.

LAND **"The same move as the temporary AND, applied to a whole table."**

DEFINE "A phase lookup costs $O(\sqrt{L})$" is too compressed — "the repair needs only one sign per address, not a whole word, so splitting the address into two halves makes it about $2\sqrt{L}$ ANDs instead of L."

S1 Factoring: what each optimisation bought (1/2)

1 min

EXPLAIN one row per optimisation: what it removes and its measured gain. Say the top two (ripple-carry adders −95 to −99% T; semiclassical QFT −41% qubits at $N = 21$, −50% at size) and the windowing row. The footnote: precomputed powers and controlled MAJ/UMA come free.

LAND **"Each optimisation removes one thing, and each gain is measured on the circuit with and without it."**

S2 Factoring: what each optimisation bought (2/2)

1 min

EXPLAIN the lookup-side gains (unary iteration −47% qubits, measured uncomputation −96% of the clean-up), nested windows halving the multiplications, the coset representation at −67 to −74% Toffolis per addition, and the approximate QFT at −34% rotations. End on the takeaway.

LAND **"All together at N = 15: 978,000 T down to 42,600, a 96% cut."**

Part 4 — Elliptic-curve keys (about 11.5 min)

69 ECDSA keys are discrete logarithms

1.5 min

EXPLAIN a secret number and a public point Q = dG. The points form a group; adding two points takes the three formulas, and λ is a quotient. The slide calls the secret d; say you will call it k from the next slide on.

LAND **"Point addition needs a modular division, and that is the whole cost difference from factoring."**

DEFINE ECDSA/ECDH are not expanded, and the notation drifts (secret d and base point G here, k and P on 71–73, G again on 70 and 76) — "ECDSA signs, ECDH agrees keys, both publish Q = d·G; d and k are the same secret, G and P the same base point; O is the group's zero, P + (−P) = O, where −P is P with y negated."

70 A toy curve

1 min CUT FIRST

EXPLAIN Over F_7, the integers mod 7, $y^2 = x^3 + 5x + 4$ has nine points, plus the point at infinity O, the group's zero. kP means adding P to itself k times. G = (0,5) has order r = 5: G, 2G, 3G, 4G, and 5G is back at O, the same idea as the order of 7 mod 15 in Part 1. The public key Q = (0,2) is 4G, so the secret is k = 4 (slide 69 called it d). Here you find k by trying every multiple; at 256 bits you cannot.

LAND **"A public key is a known point added to itself a secret number of times, and that number is what Shor recovers."**

71 Why the key is a hidden period

2.5 min ⚠ idea 6

EXPLAIN the slide now uses the hidden-subgroup wording of idea 6. Left panel: f(u,v) = [u + kv]P is constant exactly on the cosets of $L = \langle(-k, 1)\rangle$. Right panel: Fourier sampling lands in $L^\perp$, so $t \equiv ks$ and $k = t \cdot s^{-1}$. Finish on the toy line at the bottom: (2,3) gives k = 4. Take your time on this slide; it is the heart of Part 4.

LAND **"f is periodic in two dimensions: the key is the slope of the hidden subgroup, and one division mod r reads it out."**

DEFINE Cosets, hidden subgroup and L^⊥ have no plain reading — "f(u, v) = (u + kv)·P does not change when you step by (−k, 1), so the inverse QFTs return a pair (s, t) 'perpendicular' to that step, $t \equiv k \cdot s \bmod r$, and $k = t \cdot s^{-1}$."

72 Shor for ECDLP: what changes from factoring

1.5 min

EXPLAIN go down the rows: the secret is k and r is public; a period in two variables, so two counting registers; the box adds a point instead of multiplying, and costs a division; a second division clears λ; read-out by one division instead of continued fractions; 256 bits instead of 3072.

LAND **"Same skeleton, a different box: a point addition costs a division, but the key is 12 times shorter."**

DEFINE "128-bit security" is first used here (slides 104–105 skipped), and the takeaway then switches to RSA-2048 — "128-bit security means the best classical attack takes about 2^{128} operations: RSA needs n = 3072 for it, a curve n = 256; the 3.7×10^{7} vs 7.0×10^{9} line compares with RSA-2048, not 3072."

73 Two registers, one accumulator

1 min CUT FIRST

EXPLAIN This is the ECDLP circuit, laid out like Part 1's phase estimation. Two blue counting registers, u and v (three qubits each here), start with Hadamards. One red accumulator holds a curve point in 2n qubits. Each counting qubit controls the addition of a fixed, classically precomputed point (P, 2P, 4P, then Q, 2Q, 4Q), like factoring's powers of a. The accumulator ends at S + (u + kv)P, so it depends only on u + kv mod r; two QFT†s turn that into (j_1, j_2). S is a fixed start point, because slide 69's formulas cannot represent O.

LAND **"Same skeleton as factoring: counting qubits control additions of precomputed points, and the accumulator depends on u and v only through u + kv."**

76 Post-processing: the key from (j_1, j_2)

1 min CUT FIRST

EXPLAIN real registers have 2^m states, so scale each outcome by $r/2^m$, round, divide, and let every shot vote. On the toy curve k = 4 takes 42% of the vote, against 13–16% for each wrong key.

LAND **"Every shot votes, and the right key stands out."**

DEFINE (j_1, j_2) and m are new names for slide 71's (s, t) — "m = n + 1 qubits per counting register; rounding $j_i\cdot r/2^m$ recovers the pair, so $k = a_2\cdot a_1^{-1}$ is slide 71's $t\cdot s^{-1}$."

77 One point addition is eleven reversible steps

1.5 min

EXPLAIN in place, with no garbage register. Step 3 divides to get λ; step 8 is the same division, and by then it clears λ. The two divisions are 80% of the Toffolis.

LAND **"Two divisions per addition, one to make λ and one to clean it up, and they are most of the cost."**

79 Making ECDLP fast: one optimisation per level

1.5 min CUT FIRST

EXPLAIN The map for making the point-addition oracle fast, level by level as for factoring, from L0 (single gates) up to L5 (the whole oracle). Most ideas come from two 2026 preprints: [106] Kim et al. and [1128] Schrottenloher. Two to define: the Euclidean dialog runs slide 77's binary Euclid loop once, records only its decision bits, and replays them on another register to multiply: −54% Toffolis, −43% qubits. Jacobian coordinates store a point as (X:Y:Z), so an addition needs no division: −45% Toffolis. Windows take 514 additions to 34. Approximate variants carry a failure rate, like 3.1% at L2.

LAND **"Every level of the point addition has its own optimisation, each measured on a fixed task and re-checked against the classical model before it counts."**

94 L5: window by window — look up, add, unlook up

1.5 min

EXPLAIN windowing applied to points. Each table holds the 2^w multiples of a known point, computed classically; look up the point the window selects, add it, unlook it up. At n = 256 and w = 16: 16 windows per scalar, 32 additions in all (28 in the 2026 designs).

LAND **"The biggest savings come from doing fewer point additions, not cheaper ones."**

DEFINE "Mixed point addition" is undefined, and the accumulator is called R here but S on 73 — "the accumulator is kept in Jacobian coordinates (slide 79) while the table entry is a plain (x, y) point; adding one of each is a 'mixed' addition."

S3 Elliptic curves at toy size: what each level bought

1 min CUT FIRST

EXPLAIN the same idea, level by level: L0 temporary AND −60% T; L1 Gidney's adder −51% per controlled addition; L2 pseudo-Mersenne doubling −84%; three ways to do the division at L3, where the Euclidean dialog wins (−54% Toffolis, −43% qubits); L4 Jacobian coordinates −45%, but with garbage; L5 windows, −93% point additions.

LAND **"Halving a point addition is worth half as much as running fifteen times fewer of them."**

107 At 128-bit security

1.5 min

EXPLAIN 12^3 = 1,728, divided by the 6.7 times heavier group operation, is about 260. The deck's own circuits give about 270; the literature gives 148–180.

LAND **"Two orders of magnitude fewer Toffolis for a 256-bit curve than for RSA-3072, however you count."**

DEFINE "1,728/6.7" uses slide 106's measurement, which is skipped; "P-256" is first named here — "one point addition costs 6.1–6.7 controlled modular multiplications of the same width, so the 12^3 = 1,728 advantage is divided by about 6.7; P-256 is NIST's 256-bit curve."

109 What it means

1.5 min

EXPLAIN the 2026 estimates: 4–9×10^7 Toffolis for a 256-bit curve against 6.5×10^9 for RSA-2048, so about 20 minutes against 5 days on the same hardware. Key exchange: harvest now, decrypt later. Signatures: once a machine exists, anyone can forge. Migrate to ML-KEM and ML-DSA.

LAND **"The short key that makes elliptic curves efficient also makes them the easier quantum target."**

DEFINE "≈7× heavier group operation" is 107's 6.7, and "20 minutes against 5 days on the same hardware" rests on the hardware acts (X–XI) that are skipped — "tie '5 days' to slide 39's <10^6-qubit RSA-2048 estimate; ML-KEM and ML-DSA are NIST's post-quantum key-exchange and signature standards."

Part 5 — Where things stand in 2026 (about 9 min)

157 The logical frontier, September 2026

1.5 min

EXPLAIN Toffolis against logical qubits. The purple staircase runs from Luo et al. (835 qubits, 2.0×10^9 Toffolis) to IonQ (1,457 qubits, 3.9×10^7); the red corner is RSA-2048 (1,399 qubits, 6.5×10^9).

LAND **"In 2026 a 256-bit curve came down to about a thousand logical qubits and 4–9×10^7 Toffolis."**

DEFINE The purple staircase is never explained, and "the 2026 progress on RSA was in the machines" points at skipped acts — "each step is the fewest Toffolis published at that many logical qubits or fewer; Schrottenloher is [1128] from slide 79; drop the 'machines' clause or say 'in hardware estimates'."

164 Act XII — from the papers as printed to below IonQ's count

1.5 min

EXPLAIN Pose Part 5's question. Part 4's circuit followed the two 2026 eprints as printed (the box says 2.6M Toffolis per point addition), while IonQ reports 1.39M on 1,457 qubits: where is the factor 2? The table splits one windowed point addition (n = 256, w = 16) into its parts. Three changes take it from 6,276,990 to 1,722,769 Toffolis, 3.6×: measured uncomputation of the lookups (slide 59's trick), a cheaper GCD inside the two in-place multiplications, and a squarer for λ^2. With 34 → 28 additions, the whole ECDLP-256 falls from 213M to 48M.

LAND **"Three changes (measured uncomputation, a cheaper GCD and a squarer) make one point addition 3.6× cheaper and take the whole ECDLP-256 from 213M to 48M Toffolis."**

DEFINE The box's "2.6M as printed" and the table's "before" 6,276,990 look contradictory side by side — check what the "before" build is and say it in one sentence, or lead with the table alone.

DEFINE The title promises "below IonQ's count" but the table ends at 1,722,769, above IonQ's 1.39M — "this table is the first three changes; the IonQ-adder, signed-window and SELECT-SWAP rows on 207–208 bring it to 1.31M executed on 1,457 qubits (slide 206)."

DEFINE "Measured uncomputation" is presented as new, but the audience saw it on slide 59 — call it "slide 59's trick, applied to the curve's lookups".

DEFINE The box says 2.6M, the table 6.28M — 2.6M is the eprint's published count with its approximate arithmetic; 6.28M is this repo's build of the same design with exact arithmetic and recomputed lookups, the table's "before".

207 Act XII in plain words (1/3)

1 min

EXPLAIN Scan the table, then stop on two rows. Each line is one optimisation, what it does, and in vermillion what it bought, per windowed point addition at n = 256, w = 16. Row 1 is slide 59's trick at full size: measure the loaded table away instead of recomputing it, 1.31M → 197,364 Toffolis. Row 4 is IonQ's adder: the carry-out of the addition also decides whether to subtract p, and is then cleared by a measurement and a phase fix, as in the temporary AND; the replay step falls 712 → 352, the addition 1.71 → 1.44M.

LAND **"The biggest single saving is measuring the loaded table away instead of recomputing it: lookups fall from 1.31M to 197,364 Toffolis per addition."**

DEFINE Rows 2 and 5 are opaque — "conditionally inverted GCD: re-encode one value so both stay odd, and every Euclid step becomes one plain adder (1.96 → 0.72M is per multiplication; 164 shows two). 37 careful steps: x + y = p is the one sum the carry flag cannot see, and it can only happen early, so the first 37 steps keep the exact cell."

208 Act XII in plain words (2/3)

1 min

EXPLAIN Same drill: scan, then stop on three rows. Signed windows read each window as ±d, look up |d|, and negate y for the sign (−P is P with y negated): tables halve to 2^{15} entries, never hold O, and the count lands 4.6% below IonQ. Peak chasing: the qubit count is set by the busiest moment, so free the scratch of whichever step sets that peak: 2,233 → 1,457 qubits, no extra Toffoli. Lean cells show the opposite trade: 1,246 qubits, but 2.82M Toffolis. CDKM is slide 47's ripple-carry adder; a Fredkin is a controlled SWAP.

LAND **"Signed windows take the count below IonQ's, and peak chasing gets down to IonQ's 1,457 qubits without a single extra Toffoli."**

DEFINE "Peak" now means the moment of highest qubit use, not a histogram peak as in Part 1, and "executed" is new — "executed Toffolis are the average number actually run, because some phase repairs run only when a measurement calls for them."

S4 secp256k1 at full size: what each change bought

1.5 min CUT FIRST

EXPLAIN this is the repo's own work at n = 256. Three changes take one point addition from 6.28M to 1.72M Toffolis (−73%): measured lookup clean-up (−85%), IonQ's inverted-step GCD (−63%) and the squarer with its fold (−92%). Then IonQ's carry-is-flag adder halves the replay step, signed windows take another 7%, and peak chasing cuts the qubits by 35% at no Toffoli cost. If asked about other trade-offs: 1,246 qubits for 65% more Toffolis; 851 qubits at 85.2M per addition; Toffoli depth −74% for 4.6× the Toffolis.

LAND **"1.31M Toffolis per addition on 1,457 qubits, 5.9% below IonQ; the whole ECDLP-256 down 83%."**

S5 Other trade-offs: fewer qubits, or less depth

1 min CUT FIRST

EXPLAIN for the question "is 1,457 qubits the minimum?". Lean cells reach 1,246 qubits for 65% more Toffolis, Luo et al.'s register-shared Euclid 1,107 for 30× the Toffolis, and three field registers 851 at 85.2M Toffolis per addition. In the other direction, carry-lookahead adders cut Toffoli depth by 74% for 4.6× the Toffolis.

LAND **"Going narrower or shallower is possible, and it costs Toffolis."**

206 The built frontier

1.5 min

EXPLAIN this is the work behind the deck. Every grey dot is a circuit built and counted at n = 256. The frontier ends at 1,457 qubits and 1.31 million Toffolis per point addition, below IonQ's own point. The factor of two between the eprint as printed and IonQ comes down to three ideas: measured uncomputation of lookups, a cheaper GCD, and a dedicated squarer.

LAND **"Every number on this plot is a circuit that exists and was counted, not an estimate."**

DEFINE secp256k1 replaces Part 4's P-256, and Qualtran is new — "secp256k1 is the 256-bit curve Bitcoin uses (P-256 is NIST's); Qualtran is Google's open-source resource-counting library, here an independent check on the counts; the plot labels (three registers 851, lean dialog 1,246, IonQ cells + SELECT-SWAP) are rows of 207–208."

S0 What the optimisations in the repo bought

1 min

EXPLAIN the repo's whole contribution on one slide, read row by row. Factoring: ripple-carry adders −95 to −99% T, one counting qubit −41 to −50% qubits, measured clean-up −96%, so T −96% at N = 15. Elliptic curves at toy size: the dialog −54% Toffolis, windows −93% additions. secp256k1 at full size: −73% per addition, 1,457 qubits, 5.9% below IonQ, the whole ECDLP-256 −83%.

LAND **"Every number is a count the repo measured on circuits it built, each optimisation switched on and off."**

211 Seven things to keep

1.5 min

EXPLAIN read the seven items; they are the seven ideas above. End on item 7 and leave the slide up for questions.

LAND **"Shor's cost is reversible arithmetic, and in 2026 a 256-bit curve key came down to about a thousand logical qubits."**

DEFINE Item 6 (Hamiltonians, Lie brackets, "the CNOT point") comes from Act X, which is not on the route, and item 7's physical counts (5×10^5 superconducting qubits for minutes, 2×10^4 ions or atoms for weeks) were never built up — skip item 6 or say "hardware, which we skipped today"; for item 7 say "each logical qubit costs many physical ones (Part 2's background), and these are the published totals."

Likely questions

"How many qubits to break a Bitcoin or ECDSA key?" About 1,200–1,500 logical qubits and $4–9\times10^7$ Toffolis. Physically, under 500,000 superconducting qubits for about 20 minutes, or 20,000–26,000 ions or atoms for days to weeks. No such machine exists yet.

"When?" Don't give a date. Say what it would take (idea 7) and that the cheapest designs rest on the least-tested assumptions.

"Why does ECC fall first if a point addition is harder?" The cost is cubic in the key length and the key is 12 times shorter; the 7 times heavier operation only takes back a small part of 12^3.

"Is f(u,v) periodic?" Idea 6 (a): yes, with a lattice of periods; (−k, 1) is the one that hides the key.

"Doesn't measuring destroy the interference?" Idea 4: only a measurement that reveals x does; an X-basis measurement reveals nothing and leaves a known phase.

"Toffolis or T gates?" One Toffoli is 7 T (4 as a temporary AND). The deck counts Toffolis because the arithmetic is built from them.

"Are these numbers yours?" Act XII's are circuits built and counted in the repo, checked on every input at toy sizes and pinned by regression tests. Act XI's are published estimates, quoted as their authors report them.

"What should we do now?" Move key exchange to ML-KEM first, because recorded traffic can be decrypted later, then signatures to ML-DSA.

For anything else, the overview has twelve more questions with answers, and the backup slides (last part) cover verification, post-processing and references.

Overview: routes, timing and questions

The full deck runs about 3 h 40 min over 211 slides; the core path about 2 h 35 min. Slide numbers here are the ones printed bottom-right of each slide (B1–B18 for backups).

Before you start

Each slide in the script gets three lines: **Say** (spoken text to put in your own words), **Point to** (where to look on the slide) and **Then** (the bridge to the next slide). Slides marked *optional* can be skipped without breaking the story.

- **Colours carry meaning.** Blue is the counting register, vermillion the target register and the arithmetic, purple the period or the key. Say it once on slide 1; after that, point.
- **Each act is one question.** The lavender Difficulty box asks it; the Resolved box answers it. Read the question aloud at the start of each act, and the answer at its end.
- **Diagrams: point first, then talk.** Give the audience two seconds to find the thing before explaining it.
- **Code slides: one line each.** Say which line matters and what it does; never read code aloud.
- **One number per slide.** Each slide has a headline number, usually in bold or in the takeaway box. Say that one; let the table hold the rest.
- **Recap tables** (66–67, 100–101, 207–209): scan the rows in ten seconds, then stop on two or three.
- **The 1,151-qubit walkthrough** (139–156) quotes the challenge's reported numbers for a circuit with a classical addend. Say so once on 139 and again on 156.

Choose a route

Pick the route by the time you have, then follow the script for those slides only; each skipped run of slides is bridged by the **Then** line of the slide before it.

Route	Length	Slides
Full deck	about 3 h 40 min	1–211, backups for questions
Core only	about 2 h 35 min	every slide marked *core*
Session 1 of 3	76 min	1–68: opening, Acts I–VII
Session 2 of 3	66 min	69–133: Acts VIII–X
Session 3 of 3	80 min	134–211: Acts XI–XII, closing
One hour	about 50 min + questions	the 30-slide route in Part 1 (the deck shor_talk60.pdf), “Your 60-minute talk”
Half an hour	30 min	2–4, 17, 21, 39–40, 47, 53, 66–67, 72–73, 77, 79, 102, 107, 109, 157, 164, 206, 211

Act X (110–133) is self-contained hardware physics: drop it for a cryptography audience, keep it for physicists. The short routes lean on the recap tables (66–67, 100–101) to cover optimisations whose own slides they skip.

Timing by act

Acts VII, VIII, XI and XII hold two thirds of the time; their optional slides are mostly code and second detail slides.

Act	Slides	Full (min)	Core (min)
Opening	1–2	1.75	1.75
I — one unitary and one measurement	3–9	8.75	7
II — which gates, and what they cost	10–17	8.5	7.5
III — reversible arithmetic must clean up	18–21	4.5	4.5
IV — building the factoring circuit	22–31	10.25	7.75
V — simulation and reading the answer	32–36	6.25	5.5
VI — counting qubits, Toffolis and T	37–39	4.5	4.5
VII — the optimisations, one idea each	40–68	31.25	22.5
VIII — the elliptic-curve discrete logarithm	69–103	37.25	25
IX — why ECDSA falls before RSA	104–109	7	6
X — Hamiltonians and hardware	110–133	22	13.75
XI — the 2026 frontier and the machines	134–163	32	23.25
XII — the 2026 optimisations, built here	164–209	45	23.5
Closing	210–211	2.5	2.5

Questions to expect

Short answers from the deck's own numbers, and the slide to put up.

- **Is this a threat today?** Not yet. ECDLP-256 needs about 10^3 logical qubits and $4–9 \times 10^7$ Toffolis; on the deck's surface-code model that is 2.14M physical qubits for 15 minutes. The machines in Act X run around a hundred physical qubits. Show 205, then 163.
- **Why does ECC fall before RSA?** Same cubic cost, a 12× shorter key. Built here at about 1,460 qubits: 189× fewer Toffolis than RSA-2048, 537× fewer than RSA-3072. Show 72, 107, 205; B13 for other key sizes.
- **Does measuring the counting qubits early change the answer?** No: each rotation it controls is applied classically instead, and the distribution is the same. Show 43, then 46 (N = 21 on 13 qubits instead of 22).
- **How do you know the circuits are right?** Every basis input at toy sizes, every ancilla back to $|0\rangle$; steps that measure are checked for phase with a sparse state-vector simulator; a regression test pins every count. Show 165, B12.
- **Why count Toffolis rather than T gates?** A Toffoli is exactly 7 T, or 4 as a temporary AND, and Toffolis map onto magic-state factories. Show 13 and 17.
- **Act XII says 2.6M Toffolis per addition as printed, but its table starts at 6.28M. Which is it?** 2,588,962 is Schrottenloher's published count, with his approximate arithmetic. The 6,276,990 in the table is this repo's build with exact arithmetic and recomputed lookups; the "after" column uses the papers' approximate settings. Show 164.

- **Is the 1,151-qubit leaderboard entry better than your 1,457?** Not the same object: it adds a classical point and leaves out the lookups, 197,364 Toffolis per addition here, and validates its truncations on the benchmark's inputs. Show 156.
- **What are "executed" Toffolis?** Repairs after a measurement run only on one outcome, about half the time; IonQ counts what executes, and the deck gives both. Show 175 and 180.
- **Do the approximations break the algorithm?** Failures compound, so papers use 40–50 comparison bits at n = 256, and the replay keeps 37 exact steps. Failure rates are measured at toy sizes, not at n = 256. Show 84 and 177.
- **How long would a run take?** Toffolis × time per Toffoli, set by reaction time and factories: 15 minutes for ECDLP-256 and 2.1 days for RSA-2048 on one model. Be ready: Gidney 2025 shows 4.96 days on 158 and 163 (his paper) but 4.63 days on 159 (Toffolis × time per Toffoli). Show 159, 205.
- **Did you actually run the elliptic-curve attack?** On a toy curve: the 12-qubit table oracle runs on Aer and the key comes out; the 69-qubit arithmetic oracle is checked on every basis state but too big to simulate. Show 75–76, B11.
- **Can I reproduce the numbers?** Yes, the commands are on B14.

Script 1 — Opening to Act VI (slides 1–39)

About 44.5 min in full, 38.5 min core.

Opening

TIME 1.75 min.

1 How to read these slides

CORE 45 s

SAY Two conventions run through the whole talk. Colours carry meaning: blue is the counting register, the one we measure; vermillion is the target register, where the arithmetic happens; purple is what we are after, the period r or the key k. Each act opens with a "Difficulty" box that states its question and closes with a "Resolved" box that answers it, and costs live in the peach boxes. Every number on these slides is computed by the code in shor_qiskit, the same code the walkthrough notebook runs.

POINT TO the three colour lines on the left, then the three boxes on the right.

2 The road map, in plain words

CORE 1 min

SAY This is the whole journey on one page; follow the arrows. First the algorithm itself, then the price of gates, then why arithmetic must be reversible and clean. Then we build the circuit, run it and count it, and spend Act VII on twelve optimisations, each one measured. After that come elliptic curves, why ECC falls before RSA, the hardware, and the 2026 frontier. The line at the bottom is the thesis: almost all of Shor's cost is reversible arithmetic, so that is what we build, run and count.

POINT TO the arrows snaking from Act I down to Act XII, then the italic sentence at the bottom.

THEN Act I: what Shor's algorithm actually is.

Act I — One unitary and one measurement

QUESTION How does a quantum computer find the order r of a modulo N, and why does that factor N?

TIME 8.75 min (core 7 min).

LAND THIS Shor is phase estimation of a group operation: one fixed circuit pattern, and only the box inside it changes.

3 Act I — factoring reduces to finding a period

CORE 1.5 min

SAY Factoring comes down to one question: what is the order of a modulo N? Take 7 modulo 15: the powers run 1, 7, 4, 13 and back to 1, so the period is 4. With an even period, a^r − 1 splits into two factors and N divides their product; unless a^(r/2) is −1, each factor shares a divisor with N. Here the two neighbours of 7 squared are 48 and 50, and Euclid gives 3 and 5. The period belongs to the whole sequence, not to any single value, and finding it is the only step that needs a quantum computer.

POINT TO the table row returning to 1 after four steps, then the two gcds.

THEN So we need a machine that finds periods: phase estimation.

4 Phase estimation: the circuit

CORE 1.5 min

SAY This is the entire algorithm. In blue, a counting register of t = 2n qubits, each put into superposition by a Hadamard; in vermillion, an n-qubit work register holding the integer 1. Counting qubit i controls one rung, which multiplies the work register by a to the 2^i; those powers are computed classically, so every rung is just another modular multiplication. Then an inverse QFT on the counting register, and we measure one integer, y. One unitary, one fixed input, one measurement.

POINT TO the Hadamards on the blue wires, the rungs from left to right, the purple QFT† box, then the meters giving y.

THEN Why should y say anything about r?

5 Why the measurement reveals s/r

CORE 2 min

SAY Three steps. First, U_a has eigenvectors whose phases are s/r, and the input 1 is an equal mix of all of them. Second, kickback: each rung leaves the eigenvector alone and turns its control qubit instead, so the counting register ends up as a phase ramp, where branch x carries phase φ times x. Third, the inverse QFT turns a ramp of slope φ into a sharp peak at y ≈ Qφ. Each eigenvector makes its own peak and they do not interfere, so we read y ≈ Q·s/r with s at random.

POINT TO the words "phase ramp" in item 2, then the two cases at the end of item 3 (≈1 near Qφ, ≈0 elsewhere).

THEN Let me show you a ramp becoming an integer.

6 QFT† in action: a ramp becomes an integer

CORE 1 min

SAY The smallest case: three counting qubits and s/r = 1/4. The arrows on the left turn 90 degrees from one x to the next, and that ramp is exactly the QFT of the basis state 2. So the inverse QFT returns 2 with certainty, and all 1000 shots on Aer read y = 2. Two over eight is one quarter: the slope, read off as an integer.

POINT TO the ring of purple arrows, then the single bar at y = 2.

THEN That was the lucky case, where Q·s/r is a whole number.

7 When Q s/r is not an integer: a peak at the nearest one

OPTIONAL 1 min

SAY Now s/r = 1/6: the arrows turn 60 degrees per step, and eight sixths is 1.33, not a whole number. The readout is a sum of Q unit arrows; they line up only near Q·s/r and largely cancel elsewhere. So the peak lands on the nearest integer, y = 1, with probability 0.688; Aer measured 0.691. That is above $4/\pi^2$, the floor phase estimation guarantees for the nearest integer.

POINT TO the tall bar at y = 1, the dotted line at 1.33, and the small leftover bars either side.

8 The readout in Qiskit

OPTIONAL 45 s

SAY The same experiment in code. Look at the loop in the middle: instead of real rungs, it puts a phase on each counting qubit, exactly the ramp the rungs would leave. Then the inverse QFT and a measurement. It prints 010 on all 1000 shots: every shot is y = 2.

POINT TO the loop with `qc.p`, then the `qft(t).inverse()` line.

9 The same pattern breaks elliptic-curve keys

CORE 1 min

SAY Why this matters beyond RSA: the same pattern breaks elliptic-curve keys. For factoring, the group is multiplication mod N, the box multiplies by a, and the unknown is the order r. For ECDSA and ECDH, the group is the points of a curve, the box adds a fixed point, the unknown is the key k, and there are two counting registers instead of one. Both are phase estimation of a group operation; only the box changes, and that box is what the rest of the talk builds, runs and pays for.

POINT TO the two "box" lines side by side, then the Resolved box.

THEN To pay for the box, we need a price list.

Act II — Which gates, and what they cost

QUESTION What does each gate cost on an error-corrected computer?

TIME 8.5 min (core 7.5 min).

LAND THIS Cost is counted in T gates: a Toffoli is an exact 7 T, while one small rotation costs about as much as ten Toffolis.

10 Act II — a fault-tolerant computer has a small gate set

CORE 1 min

SAY An error-corrected computer does not give you arbitrary unitaries. It gives you Clifford gates, H, S and CNOT, which are cheap on the surface code but on their own are classically simulable, by Gottesman–Knill. Add the T gate and the set becomes universal. But T cannot be applied directly; it is bought with a prepared magic state. So the cost model for the rest of the talk is simple: count T gates, and treat Cliffords as almost free.

POINT TO the cheap column on the left against the expensive column on the right.

THEN Why is T the odd one out?

11 Why the T gate is the expensive one

CORE 1 min

SAY A logical qubit is spread over many physical qubits. The safest logical gate is transversal, the same gate on every physical qubit, so one fault cannot spread; on the surface code CNOT is transversal and H and S nearly so. Eastin and Knill proved in 2009 that no code has a universal set of transversal gates: a code that ignores small local errors cannot let small local operations move its logical state. So some gate must be done another way, and here that gate is T.

POINT TO the takeaway, which previews the next slides: Eastin–Knill, Solovay–Kitaev, magic states.

THEN How is T done? By teleportation.

12 Magic states: T by teleportation

CORE 1.5 min

SAY Prepare a special state, T applied to plus, CNOT it with the data, and measure it. Outcome 0 leaves T on the data; outcome 1 leaves T-dagger, and one S gate fixes it, because S times T-dagger is T. Everything in this circuit is Clifford; the only expensive ingredient is one fresh magic state. Those come from factories on the chip: distillation turns 15 noisy copies into one with error about $35\varepsilon^3$, and cultivation, from 2024, grows a copy inside a small code and is about ten times cheaper.

POINT TO the purple $|T\rangle$ wire, the meter, and the double classical wire into the S box.

THEN Now the gate the arithmetic is made of.

13 The Toffoli: a reversible AND

CORE 1 min

SAY The Toffoli is a reversible AND: it XORs a·b into the third bit. It swaps just two basis states, 110 and 111, and fixes the other six, so it is a permutation and its own inverse. In Clifford+T it costs exactly 7 T gates. That gives any classical function an exact price: count its Toffolis, multiply by seven.

POINT TO the equation at the top, then "7 T gates" on the right.

14 Any function becomes reversible: keep the input

CORE 1 min

SAY The trick for making anything reversible is to keep the input and XOR the output into a fresh register. Done twice it cancels, so it is reversible even when f is not: AND throws information away, yet its reversible version is just the Toffoli. A whole classical circuit converts gate by gate: NOT and XOR are already reversible, and each AND becomes a Toffoli onto a fresh zero. The catch is in the last line: those fresh bits keep every intermediate value, g(x), and that garbage is Act III's problem.

POINT TO g(x) in the right-hand formula.

THEN First, the other half of the price list: rotations.

15 Phase shifts are exact only for multiples of π/4

CORE 1 min

SAY The QFT and the Fourier adders are built from phase gates, and only three of them are exact in Clifford+T: P(π) is Z, P(π/2) is S, P(π/4) is T. Anything finer, π/8, π/16 and on, has entries outside the ring that Clifford+T can produce, so it can only be approximated. A controlled phase is two CNOTs and three phases of half the angle, so it inherits the same problem one level down. And the circuits we build in Act IV are mostly these small rotations.

POINT TO the grey "Exact" box against the peach "Not exact" box.

THEN So how much does an approximation cost?

16 Solovay–Kitaev: approximation is cheap, but not free

OPTIONAL 1 min

SAY Solovay–Kitaev says any universal gate set approximates any one-qubit gate with polylogarithmically many gates, but the exponent is about 3.97. For z-rotations, Ross and Selinger's gridsynth does much better: about three $\log_2(1/\varepsilon)$ T gates, near-optimal. On P(π/16) at similar error, that is 20 T against 472.

POINT TO the red Solovay–Kitaev curve shooting up against the gentle blue Ross–Selinger line.

17 The price list

CORE 1 min

SAY This is the table to keep in your head for the rest of the talk. Cliffords cost nothing, a T gate costs one magic state, and a Toffoli costs seven, or four as a temporary AND, which we meet in Act VII. A small rotation costs about three $\log_2(1/\varepsilon)$, in practice 60 to 100 T, and with R rotations each gets only ε/R of the error budget. So one rotation costs about as much as ten Toffolis: Toffoli circuits have an exact, low T count, and rotation circuits pay for synthesis on every gate.

POINT TO the last row of the table, then the Resolved box.

THEN Toffolis, then. But reversible arithmetic has a hidden catch.

Act III — Reversible arithmetic must clean up

QUESTION Reversible arithmetic leaves scratch values in ancillas: does this garbage matter if we never measure it?

TIME 4.5 min.

LAND THIS Garbage that records x is a which-path measurement that wipes out the peaks, so every scratch register must be back in $|0\rangle$ before the QFT.

18 Act III — arithmetic leaves scratch values behind

CORE 1 min

SAY A quantum circuit cannot erase. Computing f(x) reversibly leaves carries, flags and partial products in ancillas, scratch qubits that started at zero. In Shor, x is the counting register in superposition, so the state holds x, f(x) and this garbage g(x), all entangled. The question for this act: if we never measure the garbage, does it matter?

POINT TO g(x) in the $|\Psi\rangle$ formula.

THEN Ask the formula.

19 Garbage destroys the interference: the formula

CORE 1.5 min

SAY Apply the inverse QFT to the counting register and measure y. The other registers enter only through their overlaps, one for the target and one for the garbage. With no garbage, or garbage fixed by f(x), the overlaps just say x and x′ agree mod r, so each period interferes and we get peaks near multiples of Q/r. If the garbage records x itself, every cross term dies and P(y) is flat at 1/Q. Garbage is a which-path record: if it tells x apart, it has measured x.

POINT TO the two underbraces, target and garbage, then the grey box against the peach box.

THEN We can check that on the simulator.

20 The formula on Aer: N = 15, a = 7, t = 4 counting qubits

CORE 1 min

SAY N = 15, a = 7, four counting qubits; each panel adds one line after the multiplications. Left, the clean circuit: four peaks. Middle, a copy of f(x) in a spare register: nothing changes. Right, a copy of x: the peaks vanish and every outcome has probability exactly 1/16. Dots are exact probabilities, bars are 2048 shots, and they agree.

POINT TO the three panels, left to right.

THEN So garbage must go. Bennett showed how.

21 Bennett's fix: compute, copy, uncompute

CORE 1 min

SAY Compute, copy, uncompute. Run C, which leaves the answer and the garbage in the work register; CNOT the answer into a clean output; then run C backwards, and the work register returns to zero. The price is about twice the gates plus a register for the copy, a factor of 2 that runs through every count that follows. And this pattern returns all through the talk: the modular adder's flag, multiply–swap–unmultiply, the point addition's second division, the temporary AND.

POINT TO the three stages and the states written under each, then the Resolved box.

THEN Now we know the rules, we can build.

Act IV — Building the factoring circuit

QUESTION How does "multiply by a modulo N" become gates?

TIME 10.25 min (core 7.75 min).

LAND THIS The box is modular multiplication made from Fourier adders, built in levels that are each checked exactly, and uncomputed twice.

22 Act IV — Qiskit in a few idioms

OPTIONAL 1 min

SAY A short Qiskit primer, because these idioms appear on every code slide. A builder is just a function that appends gates; here, one that adds 1, using multi-controlled X gates for the carries. Look at the line with `.inverse()`: running a sub-circuit backwards is uncomputation in one call. The example starts at 3, adds 1, undoes it, and prints 011 every time; note that qubit 0 is the rightmost bit.

POINT TO `add_one`, the `to_gate().inverse()` line, and the idioms list on the right.

23 Five levels, each a product of the one below

CORE 1 min

SAY Here is the plan for the box, bottom to top. Level 0 is the QFT; level 1 adds a constant through it; level 2 makes that addition modular; level 3 multiplies with a stack of modular additions; level 4 makes the multiplication in place; level 5 is the whole of Shor. On the right are the matching function names in the code. The rule throughout: each level is checked exhaustively, every input and every ancilla back to zero, before anything is built on it.

POINT TO L0 at the bottom up to L5, then the function names on the right.

THEN Start at the bottom.

24 L0 — the QFT is Hadamards and controlled phases

CORE 1 min

SAY The QFT of a basis state is a product state: each qubit just gets its own phase. That makes the circuit simple: each qubit gets one Hadamard, then one controlled phase per lower bit, with the angle halving each time. The swaps at the end only undo a bit reversal. Notice the angles, $\pi/2$ and $\pi/4$ here and smaller as n grows: these are the rotations we just priced.

POINT TO the three-qubit diagram: the Hadamards, the $P(\pi/2)$ and $P(\pi/4)$ gates, and the final swap.

25 L0 in Qiskit

OPTIONAL 45 s

SAY The same circuit in a few lines: a Hadamard, a loop of controlled phases, then the swaps. It was checked against the DFT matrix and Qiskit's own QFTGate for n from 1 to 6. On cost: the controlled $\pi/2$ is exact at 3 T, and the rest are small rotations, $3\cdot(n-1$ choose 2) per QFT. The `approx` argument drops the smallest angles; we come back to that in Act VII.

POINT TO the `cp` line and the `if approx` line above it.

26 L1 — Draper's adder: addition is a phase

CORE 1 min

SAY Draper's idea: in the Fourier basis, addition is a phase. Each qubit of QFT|y⟩ carries a phase set by y; adding X multiplies it by one more phase, one gate per qubit and no carries at all. The figure adds 3 on three qubits, checked equal to adding 3 mod 8. And to control the adder, control only the phases: when they do not fire, the QFT and its inverse cancel.

POINT TO the three purple P gates between QFT and QFT†.

THEN That is addition, but not yet modular.

27 L2 — the modular adder in seven blocks

CORE 1.5 min

SAY A modular addition is seven plain additions. Blocks 1 to 4 add X, subtract N, copy the sign into a flag (did it go negative?), and add N back if so. Now y is correct mod N, but the flag still remembers whether we reduced, and that is garbage. Blocks 5 to 7 erase it: subtract X so the sign reveals the flag, clear the flag, add X again; Bennett inside one adder. Only blocks 1 and 6 need the outer controls; the others cancel in pairs when the controls are off.

POINT TO the numbered blocks (1) to (7), the flag wire, and the controls on blocks 1 and 6.

THEN Stack these and you get a multiplication.

28 L3–4 — multiply, swap, un-multiply

CORE 1.5 min

SAY Shift-and-add: a times x is the sum of $a \cdot 2^i$ over the bits of x. So CMULT(a) is n modular additions, each controlled by the rung and by one bit of x; but it writes the product into a new register, and phase estimation needs U_a in place. So: multiply into a fresh register, swap the two, then multiply by minus a-inverse to wipe out the old x. One rung costs 2n modular additions and n controlled swaps, and the whole second multiplication exists only to uncompute.

POINT TO the states along the bottom: |x⟩|0⟩, |x⟩|ax⟩, |ax⟩|x⟩, |ax⟩|0⟩.

29 L3–4 in Qiskit

OPTIONAL 45 s

SAY In code, two short functions. The line to look at is `pow(A, -1, N)`: the inverse exists because a is coprime to N, and it is computed classically. The rung was checked on every x, every a coprime to N, and both control values.

POINT TO the `cswap` loop and the `pow(A, -1, N)` line.

30 L5 — the whole circuit, N = 15, a = 7

CORE 1 min

SAY The complete circuit for 15 with a = 7. Eight counting qubits in blue; 4 target, 4 scratch, a sign and a flag qubit: 4n + 2 = 18 in all. Rung i multiplies by 7 to the 2^i mod 15, which gives 7, then 4, then 1 from there on. We keep the ×1 rungs on purpose: skipping them would use r to build the circuit that is meant to find r.

POINT TO the multipliers ×7, ×4, ×1 … along the vermillion wire.

31 L5 in Qiskit

CORE 45 s

SAY And the whole circuit as code: Hadamards, the target set to 1, a loop of t rungs, the inverse QFT, a measurement. Five short functions, each built on the one before, give the complete order-finding circuit. So the box is modular multiplication made of Fourier adders, uncomputed twice: once inside each adder, once inside each rung.

POINT TO the `for i in range(t)` loop, where `pow(A, 2**i, N)` is computed classically, then the Resolved box.

THEN Next, we run it.

Act V — Simulation, and reading the answer

QUESTION The measurement returns one integer y: how is r read from it, and how do we know the circuit is right?

TIME 6.25 min (core 5.5 min).

LAND THIS Continued fractions turn a peak into r, a bad shot costs a rerun but never a wrong answer, and the samples match the exact distribution to within shot noise.

32 Act V — running it on Aer

CORE 1 min

SAY We run it on Aer, Qiskit's simulator. Build the circuit for a = 7, N = 15, transpile it into the simulator's gate set, take 4096 shots, which takes about 2 seconds, and post-process. It prints 18 and 4: eighteen qubits, order 4. The question for this act is how one integer y becomes r, and how we know the circuit is right.

POINT TO the printed output, "18 4", below the code.

33 N = 15: four sharp peaks

CORE 1.5 min

SAY Here are the 4096 shots with t = 8: four peaks of about equal height, at 0, 64, 128 and 192. On the right is the exact distribution in closed form, and Aer's exact probabilities equal it. The samples match too: total variation distance 0.006, where a perfect sampler would exceed 0.032 only 0.01% of the time at this shot count. That is the shot-noise bound, and it is how we know the circuit is right.

POINT TO the purple dots sitting on top of the bars.

THEN Now turn a peak into r.

34 From y to r to the factors

CORE 1.5 min

SAY Each peak sits at 2^t times s/r, so divide by 2^8 and let continued fractions reduce the fraction; its denominator is a candidate r, checked by computing a^r. 64 and 192 give a quarter and three quarters, so r = 4, and it checks; 128 gives a half, and 2 fails because 7 squared is not 1 mod 15; 0 tells us nothing. Then Euclid: 7 squared is 4 mod 15, not −1, so $7^2 - 1$ and $7^2 + 1$ give the factors 3 and 5. One shot succeeds with probability one half here, measured 0.501, so about two runs; and a failed shot costs a run, never a wrong answer.

POINT TO the two ticked rows of the table, then the Euclid box.

35 Post-processing in code

OPTIONAL 45 s

SAY Post-processing is a few lines. The key call is `limit_denominator(N - 1)`: the best fraction with denominator below N, which is the continued-fraction convergent we want. Every failure is caught: y = 0, or s sharing a factor with r, means take another shot; r odd, or a^(r/2) ≡ −1, means pick another a, which happens with probability at most a half.

POINT TO the `limit_denominator` line and the `pow(A, r, N) == 1` check.

36 When r does not divide 2^t: N = 21, a = 2, r = 6

CORE 1.5 min

SAY For 15 the period divides 2^t, so the peaks are perfect; here is the general case, N = 21, a = 2, period 6. Six peaks, one per s, smeared because 1024 is not a multiple of 6. Take y = 171: the convergents of 171/1024 are 1/5, then 1/6, and 6 checks because 2^6 is 1 mod 21. That is why t = 2n: Legendre says that if $y/2^t$ is within $1/(2r^2)$ of s/r, then s/r is a convergent, and $2^t \geq N^2$ guarantees that. The cost of the smearing is a success probability of 0.322 per shot.

POINT TO the six panels, s = 0 to 5, peaking at 0, 171, 341, 512, 683, 853, with the small side bins around each.

THEN The circuit works. Now, what does it cost?

Act VI — Counting qubits, Toffolis and T gates

QUESTION How many qubits and gates does the circuit need, and which of them dominate the fault-tolerant cost?

TIME 4.5 min.

LAND THIS The textbook circuit's T count is almost all small rotations from the Fourier arithmetic, not Toffolis, so the rotations are what to remove first.

37 Act VI — the counts follow from the construction

CORE 1.5 min

SAY Because we built the circuit in levels, the counts follow level by level. A controlled modular adder: ten QFTs and 2m + 3 Toffolis. A rung: 2n of those plus n controlled swaps. The full circuit: 4n + 2 qubits, $8n^3 + 22n^2$ Toffolis, and about $20n^4$ controlled phases, one power of n worse. Every count the code measures equals its formula exactly for n from 4 to 8; the check is in the backup.

POINT TO the bold bottom row of the table.

THEN What does that mean in T gates, for 15?

38 N = 15: where the T gates go

CORE 1.5 min

SAY For N = 15 the exact part is small: 864 Toffolis cost 6048 T, and the π/4 phases another 8261 T. Then come 12,855 small rotations, each synthesised to its share of a 10^{-3} error budget, costing about 964,000 T. Total: about 978,000 T, and 98.5% of it is rotations. The Toffolis are not the problem; the rotations of the Fourier arithmetic are.

POINT TO the grey "Exact in Clifford+T" box against the peach "To be synthesised" box, then the bold total.

THEN Does that change at real key sizes?

39 At RSA-2048 size

CORE 1.5 min

SAY Scaled to RSA-2048 with the verified formulas, this circuit needs 8194 qubits and 6.9×10^{10} Toffolis, and about 2×10^{17} T with exact QFTs, or 10^{16} with approximate ones: still dominated by rotations. The state of the art: Gidney and Ekerå in 2021, 6189 qubits and 2.7×10^9 Toffolis with ripple-carry adders and windowing; Gidney in 2025, 1399 qubits and 6.5×10^9 Toffolis with approximate residue arithmetic. Physically, the right-hand box quotes fewer than a million qubits for five days, or about 94,000 for a month. Polynomial, but enormous, and mostly rotations, so Act VII removes those first.

POINT TO the peach box (this circuit) against the grey box (state of the art), then the Resolved box.

THEN Act VII: the optimisations, one idea each.

Script 2 — Act VII (slides 40–68)

About 31.25 min in full, 22.5 min core.

Act VII — the optimisations, one idea each

QUESTION The circuit we counted is polynomial but enormous, and mostly rotations: what can we take out of it, and what does each removal buy?

TIME 31.25 min (core 22.5 min).

LAND THIS Remove the rotations first, then the qubits, then the additions; each idea removes one thing, and every saving is measured.

40 Act VII — what each optimisation removes

CORE 1.5 min

SAY The last act left us with a bill that is polynomial but enormous, and mostly rotations. So this act asks one question: what can we take out, and what does each removal buy? Here are twelve ideas, one per row. The middle column is what each one removes; the right column is what we measured once it was gone. All twelve are built and checked in the code, and I'll take them roughly in this order.

POINT TO The middle column, "what it removes". Then the three rows the act will end on: ripple-carry adder ("every rotation in the arithmetic"), semiclassical QFT ("the counting register"), windowing ("most modular additions").

THEN The first two cost nothing at all.

41 Free, before anything else: the powers collapse

CORE 1 min

SAY Rung j needs U_a to the power 2^j, and done naively that is 2^j applications of U_a. But the power can be taken classically. I compute a^(2^j) mod N by repeated squaring on a laptop, and the rung becomes one multiplication by that number. So t rungs instead of $2^t - 1$ applications: 4 instead of 15 at t = 4, and exponentially many saved at size. For N = 15 and a = 7 the constants are 7, 4, 1, 1, so from the third rung on we multiply by 1, and a circuit compiled for 15 simply deletes them; that is what the 7-qubit NMR experiment of 2001 did.

POINT TO The four vermillion boxes ×7, ×4, ×1, ×1, one under each blue control. Then the code line: the `pow(A, 2**i, N)` argument is where the classical squaring happens.

42 Free, before anything else: control only what does not cancel

CORE 1.5 min

SAY Every rung is a controlled U, and the lazy way puts the control on every gate. But with the control off, the uncontrolled gates still run, and that is harmless as long as what runs cancels. Conjugation is the first case: in A B A-dagger only B needs the control, so Draper's adder controls its phases, not its QFTs. Compensating residue is the second: in the seven-block modular adder, with the control off, blocks 2 to 4 and 6 cancel each other because y is below N. Every control we drop is a controlled gate we never build.

POINT TO Top row: Beauregard controls blocks 1, 5 and 7 (shaded). Bottom row: this code controls only block 1, and block 6 as a Toffoli. Then the peach box: one modular adder goes from 2,634 to 206 CNOTs, 12.8×; the whole controlled U_a from 20,892 to 1,748, 12.0×. Depth is the second number of each pair, and it falls just as sharply.

THEN Those two were free. The next one changes the register count.

43 One counting qubit: the semiclassical QFT

CORE 1.5 min

SAY Look at the inverse QFT at the end of the circuit. Every controlled rotation in it is controlled by a qubit that is about to be measured. So I measure that qubit first and let the classical bit decide whether to rotate: the output distribution does not change. The counting register then collapses to one qubit used over and over: Hadamard, rung, a phase correction computed from the bits already read, Hadamard, measure, reset, heaviest rung first. Qubits go from 4n + 2 to 2n + 3.

POINT TO The single blue wire; the vermillion target with its rung; the grey double line carrying m_0 … m_(i−1) into the purple P(−φ_i) box; then the meter and the X^(m_i) reset at the right.

THEN Here is what that looks like in Qiskit.

44 In Qiskit: a dynamic circuit

OPTIONAL 45 s

SAY This is the loop as Qiskit draws it, on a toy where phase gates stand in for the rungs, with y = 5 and t = 3. Each box marked "If" is a gate applied only when an earlier measurement gave 1. That is Qiskit's `if_test`: mid-circuit measurement and feed-forward. Aer reads back 101 on every shot, which is 5 in binary.

POINT TO Follow the top wire left to right. The first meter writes m_0; downstream, the "If" boxes hang off m_0 and m_1. The X boxes are the resets; the P(−π/2) and P(−π/4) boxes are the inverse-QFT phases, now classically controlled.

45 The loop in code

OPTIONAL 45 s

SAY This is the whole loop. Look at three lines: `rungs[t - 1 - i]` takes the heaviest rung first, the inner loop applies the inverse-QFT phases under `if_test`, and the last `if_test` resets the qubit for the next rung. The rungs are callbacks, so the same function drives factoring here and the elliptic-curve problem in the next act. It is checked exactly: enumerating every measurement branch, its distribution equals the full circuit's to 10^{-13}.

POINT TO The three commented lines: "heaviest rung first", "the inverse QFT, classically", "reset for the next rung".

46 N = 21 on 13 qubits instead of 22

CORE 1 min

SAY Now a real case: N = 21. The full circuit needs 22 qubits; with one counting qubit it needs 13. The bars are Aer with 1,024 shots, the dots are the exact distribution, and they agree within shot noise: total variation distance 0.075 against a bound of 0.108. It also still succeeds: one shot gives r = 6 with probability 0.322 exactly and 0.313 measured, and 21 = 7 × 3.

POINT TO The six panels, one per peak s = 0 to 5 at $s \cdot 2^{10}/6$; the dots sitting on top of the bars.

THEN That removed qubits. Now the biggest item on the bill: the rotations.

47 Toffoli arithmetic: the ripple-carry adder

CORE 1.5 min

SAY The rotations in the arithmetic come from the Fourier adder, so I replace it. Cuccaro's ripple-carry adder runs a chain of MAJ blocks that carry from the low bits up, then UMA blocks that come back down and write the sum. Each block is one Toffoli and two CNOTs, so an m-bit adder is 2m Toffolis, one carry qubit, and no rotations at all. The seven blocks of the modular adder stay exactly as they were; only the adder inside them changes.

POINT TO The staircase: MAJ blocks stepping down the wires under the "carry" arrow, UMA blocks stepping back up under "sum". The c wire at the top is the one carry qubit.

48 Controlling the ripple-carry adder: two gates of six

OPTIONAL 1 min

SAY This is control placement again, now inside a single bit. A MAJ and its UMA are six gates, but only the two CNOTs into b need the outer control. With the control off, MAJ leaves two gates and UMA leaves the same two in reverse, so the chain cancels from the inside out. That gives 2n controlled gates instead of 6n.

POINT TO The two vermillion control dots on the ctrl wire. In the code, the two `qc.mcx([*ctrls, ...], b)` lines: with no controls they are plain CNOTs, and that is the only change.

49 Ripple-carry against Fourier, measured

OPTIONAL 1 min

SAY One modular adder, built both ways. Ripple-carry pays in qubits, 13 against 7 at N = 15, but it has zero rotations; rotations grow as n^2 and Toffolis only as n, and from n = 6 ripple-carry is shallower too. So to simulate small N, I use Fourier, because qubits are the whole cost of a statevector. For fault tolerance I use ripple-carry, because there the T count is the bill. And the takeaway: the smallest Fourier rotation at n = 2048 is 10^{-616} radians, so at that size Fourier arithmetic cannot run at all until it is approximated.

POINT TO The bold zeros in the "rot." column; then the bold 844 against 948 for depth at N = 63; then the grey "Which to use" box.

50 Toffolis are cheaper than rotations

CORE 1 min

SAY Now I price one rung in T gates, with every rotation synthesised. Ripple-carry actually uses seven to eight times more Toffolis, but it has no rotations, and that wins by a wide margin. The T count falls by a factor of 19 at n = 4 and 99 at n = 12. The gap grows like n, because rotations scale as n^4 and Toffolis as n^3.

POINT TO The log scale. Top purple line: Fourier adders. Bottom vermillion line: ripple-carry. The ÷ labels, ÷19 rising to ÷99, are the ratio between those two. The blue line in between is windowing, which comes in a few slides.

THEN Once the bill is Toffolis, the next step is to make each Toffoli cheaper.

51 The temporary AND: uncomputation for free

CORE 1.5 min

SAY In a table lookup, Toffolis come in pairs: compute an AND into a fresh qubit, use it, uncompute it. Because the target starts in $|0\rangle$, the compute costs 4 T, not 7. To uncompute, I measure the target in the X basis, which reveals nothing about a and b, and fix the phase with a classically controlled CZ, a Clifford with no T at all. A compute–uncompute pair drops from 14 T to 4, and a table lookup needs 3.5 times fewer T.

POINT TO Left of the grey barrier, the four T and T† gates on q_2. Right of it: H, the meter into m, and the "If" box holding the CZ and the X that returns the ancilla.

52 The temporary AND, in code

OPTIONAL 45 s

SAY The gadget is two short functions. The table is the check: one unary-iteration lookup transpiled to Clifford+T, from 2 to 5 address bits, and the saving is exactly 3.5 in every row. That is 7/2: half the Toffolis were uncomputes and are now free, and the other half fell from 7 T to 4. One warning from the slide: the last gate is S-dagger, three plausible variants differ only there, and all three are wrong.

POINT TO `qc.sdg(t)` at the end of `and_compute`; in `and_uncompute`, the measure and the `if_test` holding `cz` and `x`; the "saving" column.

53 Windowing in three steps

CORE 1.5 min

SAY

Now I cut the number of additions. Step one: a controlled addition is a lookup, because adding k when q is 1 is adding T[q] from the table [0, k]. Step two: address w bits at once, tabulate all 2^w contributions of a window classically, and add once per window, so n additions become n over w, rounded up. Step three: the outer control is one more address bit whose c = 0 entries are 0, so the modular adder needs no control at all. Choosing w near $\log_2 n$ balances lookups against additions, and the gain grows with n.

POINT TO

The vermillion x register cut into windows j_0, j_1, j_2, feeding the 2^w-entry table and then a single "+ mod N".

54 Windowing: look the answer up, then add once

OPTIONAL 45 s

SAY

As a circuit, one window is three steps: look up T[j] into a temporary register, add it into y mod N, then look up again to clear the temporary. What is new is that the addend is now quantum: it sits in a register, not in a classical constant. Draper's phase ramp copes, because it takes its rotations from the bits of tmp, and the seven blocks are unchanged.

POINT TO

LOOKUP, "+ mod N", LOOKUP† across the three wires. In the code, the comments (1) to (7): the same seven blocks we built earlier, with `add_quantum` where the constant addition used to be.

55 Windowing, in code

OPTIONAL 45 s

SAY

The whole windowed multiplier is one loop. For each window, Python builds the table classically, and the circuit does lookup, add, lookup. The grey box gives the trick: when the control is 0 the lookup writes 0, the seven blocks with t = 0 undo themselves, and the adder needs no controlled gate. Note the second `lookup_ui`: it exists only to uncompute the first, and the next slides remove that cost.

POINT TO

The `table = [...]` line, then the two `lookup_ui` calls, the second one marked "# uncompute".

56 Windowing, measured

CORE 1 min

SAY

The first table is y += kx at N = 255. With windows of 4 bits, 8 additions become 2, and depth falls from 13,020 to 3,866. Depth falls faster than the number of additions because lookups are shallow and the additions they replace were not: at n = 8, CNOTs divide by 2.3 and depth by 3.4, for 12 more qubits. On the full N = 15 circuit, windows of 2 roughly halve the depth. And it runs end to end on Aer: N = 15 on 24 qubits puts all 2,048 shots on the four peaks, and N = 21 on 29 qubits finds r = 6 in 68 minutes.

POINT TO

The bold w = 4 row in the top table; the bold 17,028 against 34,537 in the bottom one.

57 Making the lookup cheap: unary iteration

CORE 1.5 min

SAY

The lookup has its own cost. The obvious version, a one-hot QROM, holds all 2^m address states at once, on 2^m ancillas. Unary iteration walks the address tree instead and holds only the AND of the current prefix: one AND per node, one CNOT to move to its sibling, and CNOTs at each leaf to load T[j]. Measured, that is exactly 2(L − 1) Toffolis on m ancillas, and half that with temporary ANDs. At N = 255 with w = 4 it takes the qubit count from 59 to 31, and the CNOT saving over shift-and-add grows from 1.67 to 2.26 times.

POINT TO

The tree for m = 3, L = 8: ctrl at the root, then $\wedge\bar{a}_2$ and $\wedge a_2$, then the $\wedge a_1$ level, then leaves T_0 to T_7. Then "59 qubits → 31" in the peach box.

58 Unary iteration, in code: m ancillas, the control at the root

OPTIONAL 30 s

SAY

The walk is recursive. It splits on the most significant address bit, computes ctrl AND a into one ancilla for the upper half, flips it with one CNOT for the lower half, recurses, then clears it. At a leaf the data load is only CNOTs, and the assert states the budget: one ancilla per level.

POINT TO

The `qc.ccx(ctrl, a, node)` and `qc.cx(ctrl, node)` lines with their comments; the `assert` in `lookup_ui`.

59 Uncomputing a lookup for free

CORE 1.5 min

SAY

Every windowed addition still pays for two lookups: one to compute and one to clean up. Here is how to drop the second, with the same move as the temporary AND. Measure the output register in the X basis: you get a random m, the output is gone, and all that is left is a phase, minus one to the power m·T[a], on the address. Repairing that phase is a phase lookup, which costs only O(√L): one-hot the low half of the address and unary-iterate the high half applying CZ. At w = 11 the cleanup falls from 4,094 Toffolis to 157, so compute plus uncompute costs 1.04 lookups instead of 2.

POINT TO

LOOKUP, the X-basis meter, and the classical wire into the (−1)^F[a] box. Then the bold 26.1× in the table.

60 Measured uncomputation, in code: the controller sits in the middle

OPTIONAL 45 s

SAY

The repair is a one-hot gate on the low half, a unary walk applying CZ on the high half, and the one-hot gate undone. But F depends on the measurement outcome, so this is not one static circuit: it runs as circuit, measure, classical computation, circuit, as it would on hardware. It is checked to be exactly the diagonal (−1)^F[a] for every split, and in the windowed accumulate, 180 mid-circuit measurements leave every register exact.

POINT TO

`_walk_cz(...)` in the middle, sitting between `qc.append(g, ...)` and `g.inverse()`.

61 Nested windows: window the exponent too

CORE 1.5 min

SAY So far we windowed x, the vermillion target. Now I window the blue exponent as well, so one table, addressed by both windows, holds $k^{(v \cdot 2^i) \cdot u \cdot 2^s} \bmod N$. Three things follow: $\lceil t/w_e \rceil$ multiplications instead of t; no control, because exponent window 0 selects $k^0 = 1$; and the swap after each multiplication becomes a relabelling, with no Fredkins. At N = 15 with w_e = 2, 8 multiplications become 4, with the lowest CNOT count and depth in the table. But windows have an optimum: at w_e = 4 the 2^6-entry table costs more than the two multiplications it saves.

POINT TO The blue e-window v and the vermillion x-window u feeding one box. Then the bold w_e = 2 row, and the worse w_e = 4 row under it.

62 Nested windows, in code

OPTIONAL 45 s

SAY Two loops: the outer one over exponent windows builds the powers of k classically, and the inner one over x-windows builds the table that both address. Look at the last line, `A, B = B, A`: the swap is a Python relabelling and costs nothing. It is checked on every e below 16, and it factors 15 on 23 qubits with 2 multiplications. One limit: it does not combine with one counting qubit, because w_e exponent bits must exist together as an address; the two ideas are opposite ends of one spectrum.

POINT TO The `kes = [pow(k, (2**i) * v, N) ...]` line; the last line, `A, B = B, A`.

63 Coset representation: delete the modular reduction

CORE 1.5 min

SAY The seven blocks are what make an addition modular; the coset representation lets a plain adder do that job instead. I store k as a comb, an equal superposition of jN + k over c padding qubits, after Zalka and Gidney–Ekerå. Every branch is congruent to k mod N, so a plain addition of a leaves every branch congruent to k + a. When k + a reaches N, the comb re-indexes, and it differs from the ideal state only in two edge teeth out of 2^c: that is the entire approximation. One addition then needs 3.0 to 3.9 times fewer Toffolis with ripple-carry, and 1.7 to 2.8 times fewer CNOTs with Fourier.

POINT TO "ideal" against "plain add": the dashed vermillion tooth marked "missing" on the left and the solid one marked "extra" on the right; here $2^c = 4$. In the code, `add` is just `add_const`; the only "mod N" left is on the classical constant.

64 Coset representation, measured

OPTIONAL 1 min

SAY This idea is approximate by design, so the tests measure an error rate, not equality. At N = 15, after 8 additions, the error halves with each padding qubit and stays under the bound $8/2^c$. Beware a friendly modulus: at N = 21 the error was 0 for every c, because $2^n - N = 11$ leaves so much headroom that no branch wraps, which is why the tests use 15 and 31, just below 2^n. Multiplication works too, with a worst error of 0.0135 at N = 7 and c = 4, and the coset order-finding circuit is checked at N = 15.

POINT TO The error column against the bound column, from 0.749 at c = 2 down to 0.000 at c = 6. Then the peach box.

65 Approximate QFT: fewer rotations, but errors add up

CORE 1 min

SAY

The last idea drops every controlled phase finer than $\pi/2^d$, in every adder's QFT. At N = 15, d = 4 drops only a handful of rotations and changes nothing visible: success stays at 0.501. At d = 3 the distance from exact jumps to 0.138, at d = 2 to 0.594, and the success rate roughly halves. The reason is that 320 Fourier additions each bring their own error and those errors add up; one error budget is shared by every approximation, so d must grow like $\log_2$ of the number of additions over ε. At n = 8, d = 4 already removes a third of the rotations.

POINT TO

The bold 0.138 and 0.594, against the shot-noise bound of 0.032 under the table.

66 Act VII in plain words (1/2)

CORE 45 s

SAY

Here is the first half again, one line per idea, with what it bought in red. I won't read it; three rows matter most. Precomputed powers: t rungs instead of $2^t - 1$, so t multiplications instead of exponentially many. Semiclassical QFT: 4n + 2 down to 2n + 3 qubits, which put N = 21 on 13. And ripple-carry: no small rotations, and the T count divided by 19 to 99.

POINT TO

Run a hand once down the red column, then stop on three rows only: Precomputed powers, Semiclassical QFT, Ripple-carry adder. Pass over control placement, controlled MAJ/UMA and the temporary AND in one breath; each already had its slide.

67 Act VII in plain words (2/2)

CORE 45 s

SAY

The second half, same layout, and again three rows to keep. Windowing: n additions become n over w, and one rung falls from 4.0 to 1.2 million T. Measured uncomputation: the cleanup lookup drops from 4,094 Toffolis to 157 at w = 11. And the approximate QFT, the one that comes with a warning: 34 percent fewer rotations, but the errors add up.

POINT TO

Stop on Windowing, Measured uncomputation and Approximate QFT. Pass over unary iteration, nested windows and coset representation in one breath.

68 All together: N = 15

CORE 1 min

SAY

Here is everything on one example, with rotations synthesised at $\varepsilon = 10^{-3}$. The textbook circuit is 18 qubits and 978k T, and almost all of that bar is pale purple: synthesised rotations. Windowing and nested windows bring it to 593k and 280k, but it is still rotations, and one counting qubit cuts qubits to 11 while leaving the T count where it was. Ripple-carry is what removes the purple, and with one counting qubit it gives 17 qubits and 42,600 T against 18 and 978,000: twenty times better. So the order is: remove rotations first, then qubits, then additions.

POINT TO

Left panel: qubits. Right panel: T, split into vermillion (from Toffolis) and pale purple (from rotations). Compare the top bar, Beauregard, with the bottom bar, one counting qubit + ripple-carry. Finish on the Resolved box.

THEN

Next: does the same circuit break elliptic-curve keys?

Script 3 — Acts VIII and IX (slides 69–109)

About 44.25 min in full, 31 min core.

Act VIII — The elliptic-curve discrete logarithm

QUESTION Does the circuit that finds a period also recover an elliptic-curve private key d from its public point Q = dG?

TIME 37.25 min (core 25 min).

LAND THIS The curve key is a hidden period found by the same circuit; its oracle costs $O(n^3)$ Toffolis, dominated by modular division, and the biggest savings come from doing fewer point additions, not cheaper ones.

69 Act VIII — ECDSA keys are discrete logarithms

CORE 1.5 min

SAY Difficulty 8 moves from RSA to the keys behind ECDSA and ECDH: a secret number d and a public point Q = d times G on a curve. Getting d back from Q is the elliptic-curve discrete logarithm, and the question is whether the circuit we just built solves it. The points of the curve, plus a point at infinity O, form a group, and adding two points takes the three formulas on the slide. Look at λ: it is a quotient. That modular division is the whole cost difference from factoring, and it will dominate this act.

POINT TO the purple d in the Difficulty box; the fraction that defines λ; the takeaway box.

THEN Let me make this concrete on a curve small enough to draw.

70 A toy curve

CORE 1 min

SAY This is $y^2 = x^3 + 5x + 4$ over the field with 7 elements: nine points, plus O. Multiplying a point by k just means adding it to itself k times. G = (0,5) has order 5: G, 2G, 3G, 4G, and 5G is back at O. The public key Q = (0,2) is 4G, so the secret here is k = 4, and you can find it by trying all four multiples.

POINT TO the red labelled points [1]G, [2]G, [3]G, and Q = [4]G in purple at (0,2); the grey points are on the curve but are not multiples of G.

71 Why the key is a hidden period

CORE 1.5 min

SAY The trick is to turn the logarithm into a period, in two dimensions. Take two integers u and v and compute u copies of P plus v copies of Q; since Q is k copies of P, that is u + kv copies of P. So take k away from u and add one to v, and nothing changes: (−k, 1) is the hidden period. From there it is Act I again: superpose, evaluate, inverse QFTs, measure (s, t), and one modular division gives $k = t\,s^{-1} \bmod r$. All the work is in the oracle, the double-scalar multiplication [u]P + [v]Q.

POINT TO panel 1, the identity f(u − k, v + 1) = f(u, v); panel 2, the purple $k \equiv t\,s^{-1} \pmod r$.

THEN Before the circuit, what actually changes from factoring.

72 Shor for ECDLP: what changes from factoring

CORE 1.5 min

SAY Here is the whole comparison on one slide. The skeleton does not change: phase estimation of a group operation, then a little classical work. What changes is the secret and the box. In factoring the unknown is the order r itself; here r is public and the unknown is k, hidden as a period in two variables, so there are two counting registers of n + 1 qubits instead of one of 2n. The controlled box stops being a multiplication by a constant and becomes the addition of a known point, and that costs a modular division, which is also how it stays clean: a second division clears λ. The read-out gets simpler, no continued fractions, just $k = t\, s^{-1} \bmod r$ and a check. And at the same classical security the key is 256 bits instead of 3072, which is why, built here at about 1,460 qubits, ECDLP-256 costs 3.7×10^7 Toffolis against 7.0×10^9 for RSA-2048.

POINT TO the "controlled box" and "one box costs" rows, red multiply against red add and the bold division; then the last row, 3072 against 256.

THEN Here is that oracle as a circuit.

73 Two registers, one accumulator

CORE 1.5 min

SAY Blue: two counting registers, u and v, three qubits each in this drawing. Red: one accumulator that holds a point, 2n qubits for its x and y. Each counting qubit controls the addition of a fixed point, computed classically — P, 2P, 4P, then Q, 2Q, 4Q — the same way factoring multiplies by fixed powers of a. The accumulator ends at $S + (u + kv)P$, so it depends only on $u + kv \bmod r$, and the two inverse QFTs turn that into (j_1, j_2). We start from a fixed point S, not O, because the affine formulas cannot represent O; we hit it only with probability of order 1/p.

POINT TO the six controlled boxes, +P to +4Q, on the red wire; the purple k in the equation underneath.

74 ECDLP in Qiskit

OPTIONAL 45 s

SAY The builder is short. Two oracles plug into it: a table oracle that simply permutes the curve's points, 12 qubits and small enough for Aer, and the real arithmetic oracle at 69 qubits, which we check on every basis state.

POINT TO the loop with two `permutation` calls per bit (controlled $+[2^i]P$ from u, $+[2^i]Q$ from v), then the two inverse QFTs.

75 On Aer: the key comes out

CORE 1 min

SAY Left, the exact distribution of (j_1, j_2); right, Aer with 4096 shots, and it is the same picture. The distance from exact is 0.031, inside its bound of 0.054. One shot yields k with probability 0.611, and we measure 0.610. The one-qubit trick from Act VII carries over: with the semiclassical QFT both registers share one qubit, and on a 13-point curve mod 11 we recover k = 9 on 9 qubits instead of 16.

POINT TO the same few dark cells in both heat maps, including the corner (0, 0).

76 Post-processing: the key from (j_1, j_2)

CORE 1 min

SAY A peak satisfies the congruence at the top: scale each outcome by r over 2^m, round, and divide, $k = a_2\, a_1^{-1} \bmod r$. Every shot casts a vote, and we check the winner classically with $kG = Q$. When 2^m is not a multiple of r the rounding can miss, so the code also tries ±1 around it, following Ekerå. On the toy curve k = 4 takes 42 percent of the vote, against 13 to 16 percent for each wrong key: a signal, not luck.

POINT TO the tall purple bar at 4 against the four grey bars.

THEN So it works. What does the oracle cost? (The voting code is backup B10.)

77 One point addition is eleven reversible steps

CORE 1.5 min

SAY Here is one point addition written in place, with no garbage register. Step 3 divides to get λ; step 4 clears y, because after the subtractions y equals x times λ. Step 8 is the same division again, and since y over x is λ once more, this time it clears λ: that is the uncomputation. Each division is Kaliski's binary extended-Euclid inversion, 2n rounds of shifts and subtractions. The two red bars tell the story: the divisions are 80 percent of the 2,744 Toffolis of one addition at p = 7.

POINT TO the two long red bars at steps 3 and 8, against the short grey ones.

78 ECDLP costs: $O(n^2)$ per addition, $O(n^3)$ overall

CORE 1 min

SAY A division is 2n rounds of O(n) Toffolis, so one addition is $O(n^2)$; the measured slope is 1.9. Shor adds one point per bit of u and v, 2n additions, so the whole attack is $O(n^3)$ Toffolis on O(n) qubits. And there are no rotations: the arithmetic is Toffolis and CNOTs only, and the inverse QFTs become classically controlled phases.

POINT TO the red line (Toffolis per addition, slope 1.9) and the blue line (qubits, about 17n + 16) on the log scale.

THEN That is the plain circuit. Now the same game as Act VII: one optimisation per level.

79 Making ECDLP fast: one optimisation per level

CORE 1.5 min

SAY This table is the map for the rest of the act, level by level as for factoring: L0 gates, L1 adders, L2 reduction, L3 multiplication and inversion, L4 the point addition, L5 the whole oracle. Most ideas come from two 2026 preprints, Kim et al. and Schrottenloher. The right-hand column is what each one bought on a fixed task; the two to watch are the dialog at L3, 54 percent fewer Toffolis, and windows at L5, 514 additions down to 34. Every variant is re-checked against the classical model before it is counted, and the approximate ones carry a failure rate.

POINT TO the level column, top to bottom; then the "Euclidean dialog" row and the L5 row.

THEN First, L0. (Short talk: skip from here to 88, the dialog.)

80 L0: temporary ANDs — and which Toffolis are counted

OPTIONAL 1 min

SAY L0 is the temporary AND from Act VII, applied inside one modular inversion: the T count falls by a flat 2.50 times at every size. In principle it could be 3.5 times, 2 T on average against 7, but Fredkins and dirty-target Toffolis cannot use the trick. The grey box matters for every number that follows: the papers count CCX, CCZ and AND gates, and treat a measured AND-dagger as free. Count every three-qubit gate instead and Gidney-style circuits nearly double, so our cost tool reports both; quoting the wrong one halves every optimisation here.

POINT TO the n = 12 row, 79,254 T against the bold 31,740; the line (4+0)/2 = 2 T against 7; then the grey box.

81 L1: which adder, and how to control it

OPTIONAL 1 min

SAY Uncontrolled, Gidney's adder needs n − 1 ANDs where CDKM needs 2n Toffolis, at the price of more qubits: 3n − 1 against 2n + 1. The control matters as much as the adder. "Promote" turns every CNOT inside the adder into a Toffoli; copy-then-add copies the addend under the control and then adds unconditionally. At n = 32 that takes a controlled add from 128 Toffolis to 63: 2.0 times fewer, of which 1.33 times comes from the control style alone.

POINT TO the n = 32 row: 128 under "promote" against the bold 63 under "copy, Gidney".

82 Copy-then-add, in circuit and in code

OPTIONAL 45 s

SAY Only the copy ever sees the control: n ANDs write c·x into a clean register, the adder in the middle is the cheap uncontrolled one, and n AND-daggers clear the copy for free. The same pattern controls every adder in the ECDLP circuit.

POINT TO the AND, ADD and AND† boxes on the left; in the code, the loop of `and_`, the single `add`, and the loop of `and_dg`.

83 L2: approximate and pseudo-Mersenne reduction

OPTIONAL 1 min

SAY L2 is the "at least p? then subtract p" step after every addition and doubling. First idea: compare only the top few bits, which goes wrong only when those bits tie, with probability about 2 to the minus the number of bits. Second idea: use the shape of the prime. For secp256k1, $p = 2^u - f$ with $f = 2^{32} + 977$, so "at least p?" is almost "is bit u set?", and subtracting p is clearing that bit and adding f to the low bits. On p = 4093 a doubling drops from 25 Toffolis to 4, for a 3.1 percent failure rate.

POINT TO the bit strip (bit x_u cleared, the purple +f on the low bits); the bold 4 in the doubling column; the footnote, where 8 bits give 0.17 percent.

84 Pseudo-Mersenne doubling, in code — and why failures compound

OPTIONAL 45 s

SAY Nothing in this code compares anything: clearing the overflow bit is the subtraction of 2^u. Failures compound, though: a 2-bit comparison failing 12 percent per step makes a whole multiplier fail 81 percent at p = 31, hence 40 to 50 bits at n = 256. And one case is not rare: Algorithm 10 cannot reduce x + y = p, every multiplication meets it once, and Algorithm 11 catches it with its flag test gated on the control.

POINT TO the line commented "clears the bit == subtracts 2^u", and the controlled constant add of f just above it.

85 L3: Montgomery multiplication over a carry-save tree

OPTIONAL 1 min

SAY Kim et al. rewrite Montgomery's reduction step with a compile-time constant d, so that it has the shape of a multiplication step and one carry-save tree serves both. The tree is made of 3:2 compressors: three numbers in, two out, one AND per bit and no carry chain, so the depth grows like log w. Be clear what this buys: depth, halved at n = 16, for four times the qubits. It does not beat the QROM multiplier on gates.

POINT TO the compressor picture (A, B, C becoming S and the shifted 2M); in the table, depth 13,043 against the bold 6,669, and qubits 86 against 324.

86 The 3:2 compressor, in code

OPTIONAL 30 s

SAY Two loops: the sum S is three CNOTs per bit, and M is the majority, written one place up with one AND per bit. The note underneath is the detail that bites: the rewritten reduction over-adds, so the accumulator needs n + 2w bits, not Montgomery's n + 2, and the tests measure that width rather than trust it.

POINT TO the single `and_` line in the second loop.

87 L3: Kaliski with every round run, and the reductions postponed

OPTIONAL 1 min

SAY Two changes to the inversion itself. Run every round: an inert round only doubles r, so every input ends at the same $-x^{-1}2^{2n}$, and the counter and the corrective halvings become one negation. Reduce once, at the end: with r and s on 2n bits, a modular doubling is a shift, just a renaming of wires, and a modular addition is a plain one. At n = 12 that is 29 percent fewer Toffolis and 41 percent less depth, for 11 percent more qubits.

POINT TO the bold n = 12 entries, 4,653 and 16,059; in the code, the comments "plain, not mod p" and "free: no reduction".

88 L3 rebuilt: the Euclidean dialog

CORE 1.5 min

SAY Schrottenloher, after Khattar et al., starts from one observation: inversion and multiplication are one job. Run the binary GCD on (p, x) but drop the Bézout coefficients, and keep only two decision bits per round: is v odd, and is u bigger than v. That transcript is the dialog. It takes about 1.413n rounds, because each round shrinks u·v to about 3/8; measured, 1.383n at n = 32 and 1.409n at n = 256, plus a margin of order √n.

POINT TO the grey pseudo-code, especially "record b_0 and b_0b_1"; then the packing note: (0,1) never occurs, so three pairs fit in 5 bits and the record shrinks from 2.83n to 2.355n.

THEN Why keep a record at all? Because it can be replayed.

89 One round of the dialog, in code

OPTIONAL 30 s

SAY One round, line for line with the pseudo-code. The comparison u > v is computed, ANDed into the record and uncomputed; then come a controlled swap and a controlled subtraction, and the halving is just a row of swaps, because v is even by then.

POINT TO the comments down the right-hand side: b0 = v mod 2, b0b1 = b0 AND b1, swap, v −= u, v /= 2.

90 The replay: the same decisions, applied to any pair

CORE 1 min

SAY This is why the record is worth keeping. The updates the full extended Euclid makes to its pair (r, s) are linear in the recorded bits (double, maybe add, maybe swap), so they can be replayed afterwards onto any pair. Read the record backwards from (y, 0) and you end at (0, xy mod p): a multiplication by x, using only doublings. Forwards it would divide, with halvings, so to divide you run the circuit in reverse.

POINT TO the three-line update rule; the peach "Direction matters" box; `reversed(recs)` in the code.

91 In-place multiplication: dialog, replay, dialog backwards

OPTIONAL 1 min

SAY Put together, an in-place multiplication is three moves: run the dialog on x, replay the record onto (y, 0), then run the dialog backwards, which rebuilds x and consumes the record, so nothing is left behind. Against invert-then-multiply, at p = 61 that is 54 percent fewer Toffolis and 43 percent fewer qubits, and the margin widens with n. One circuit gives both multiplicative steps of a point addition.

POINT TO the comments # 1, # 2, # 3 in the code; the bold p = 61 row, 1,875 Toffolis on 66 qubits against 4,068 on 115.

92 L4 rebuilt: Jacobian coordinates, no inversion at all

CORE 1 min

SAY L4 answers the division the other way: do not divide. In Jacobian coordinates a point is a triple (X : Y : Z), and adding an affine point costs eleven multiplications and no division, the fewest of any quantum projective addition according to Kim et al. At p = 11 that is 45 percent fewer Toffolis. The catch is in the peach box: a triple is not unique, so (P + Q) − Q need not give back the input triple, and the input cannot be erased. That is 3n qubits of garbage per addition, affordable only once L5 limits how many inputs are alive.

POINT TO the peach box; the table, 4,769 against the bold 2,642 Toffolis, and 83 against 98 qubits.

93 The Jacobian addition, in code: unwind out of order

OPTIONAL 30 s

SAY Numbered steps, then one loop that undoes all but the three outputs. The subtle one is T_0: X_3 must survive, so T_0 is released by undoing step 12, which produced it, not step 13. Because the representation is not unique, the tests sweep every representative Z of each input.

POINT TO the undo list at the bottom, where b13, b17 and b18, the OUTPUT steps, are missing.

94 L5: window by window — look up, add, unlook up

CORE 1.5 min

SAY L5 is the outer loop, and it is Act VII's windowing applied to points. Cut each scalar into windows of w bits, and the oracle becomes a sum of table entries: table j holds all 2^w multiples of $2^{jw}P$, computed classically before the run. The window is now a coherent address, not a control bit: look up the point it selects, add it to R, then unlook it up. At n = 256 and w = 16 that is 16 windows per scalar, 32 updates in all.

POINT TO the lookup box on the blue wire feeding the mixed point addition on the red wire; the three-arrow line at the bottom, where register A goes from 0 to T_j[z] and back to 0.

95 L5: the circuit with its window controls expanded

OPTIONAL 45 s

SAY The whole oracle drawn out: each blue line is now a w-bit window, and each addresses its own table. Sixteen updates for P and sixteen for Q all land on one accumulator R, which is discarded at the end. This is the fully coherent picture; the semiclassical version interleaves the inverse QFT with the windows and recycles the control qubits.

POINT TO the two braces, "16 P-window updates" and "16 Q-window updates".

96 L5: windows, zig-zag, and one control qubit

CORE 1.5 min

SAY Three count-level ideas. Windowing turns 2n + 2 additions into 2⌈(n+1)/w⌉, roughly 2n over w; a lookup costs 2^w Toffolis, far less than a point addition, so w = 16 is affordable. Zig-zag, from Grassl et al., tames the Jacobian garbage: on ρ registers, add along a run, undo all but the last, then run again one shorter, for ρ(ρ+1)/2 additions in total. The semiclassical QFT then turns both scalar registers into one recycled qubit. At n = 256, 514 additions become 34, and 24,576 garbage qubits become 6,144.

POINT TO the zig-zag picture for ρ = 3: red adds, blue undos, dark squares kept, six additions on three registers; then the bold 34 and 6,144 in the table.

97 L5: one windowed addition is seven in-place updates

OPTIONAL 1 min

SAY Zoom into one windowed addition: just two field registers, X and Y, and the result lands back in them. Take the differences, then divide to get λ into Y. Add 3x_A and subtract λ^2, and X holds x_A minus the new x; multiply Y by it and subtract y_A, and Y is the new y. A final flip of X gives the new x, and the grey box checks the algebra.

POINT TO the "after it" column, top to bottom; the right-hand column maps each update to the step numbers in the code.

98 L5: the same addition as six stages on two registers

CORE 1 min

SAY Here is the same addition as a picture; keep it in mind, because Act XI builds on it. Three lookups come from the window register: A, then 3x_A, then A again to clean up. The live-state rows show what X and Y hold after each stage. The two expensive stages, 2 and 5, are the same machine, a dialog GCD walk that produces λ and then replays a fresh dialog to multiply by X; the three lookups are what Act XII makes cheap.

POINT TO stages 2 and 5; the live-state rows underneath.

99 The windowed point addition, in code

OPTIONAL 45 s

SAY The code follows the seven updates. Step 6 runs the multiplier backwards to divide and step 11 runs it forwards, so λ is consumed rather than cleared by a second division. A zero window is free, because O is stored as (0, 0) and steps 6 and 11 cancel; the addition fails only on x_R = x_P, x_3 = x_P, or adding O when x_R = 0.

POINT TO the two lines `inplace_div` (# 6) and `inplace_mul` (# 11).

100 Act VIII's optimisations in plain words (1/2)

CORE 1 min

SAY If you skipped the detail, this is the first half of the act in one table: what each idea does, and in red what it bought. Two rows deserve a pause. L1: the control touches only a copy, and a controlled add goes from 128 to 63 Toffolis. L2: the shape of the prime turns a doubling from 25 Toffolis into 4, at the price of failures that force 40 to 50 top bits at size. And note that L3 carry-save buys depth, not gates.

POINT TO walk the red column in one sweep, top to bottom, without reading the middle column aloud; stop on the L1 and L2 rows.

101 Act VIII's optimisations in plain words (2/2)

CORE 1 min

SAY The second half is where the large wins are. The dialog row: two bits per GCD round, replayed, give inversion and multiplication from one machine, 54 percent fewer Toffolis. Jacobian solves the same problem by never dividing, and pays in garbage. The last row is the biggest number in the act: 514 additions down to 34, and garbage from 24,576 qubits to 6,144. All of these are toy-size measurements, each re-checked for correctness.

POINT TO sweep the red column quickly again; stop on the "Euclidean dialog" row and the L5 row.

102 Everything at once: how the savings compose

CORE 1.5 min

SAY Do the savings multiply? Only partly, and the bullets sort them into three kinds. At gate level, Gidney's adder and the temporary AND are one idea counted twice, while pseudo-Mersenne, at 6.2 times, does compose. Structurally you pick one: the dialog and Jacobian both answer the inversion, and at p = 7 both cut a point addition by about 45 percent, but the windowed dialog does it on 49 qubits, not 75. At count level they multiply, windows dividing the additions by 15 and zig-zag the garbage by 4, which is the takeaway: halving a point addition is worth half as much as running fifteen times fewer of them.

POINT TO the three-row table (2,756, then 1,544, then 1,523 Toffolis; 64, 75, 49 qubits); the whole-circuit line: 69 qubits and 105,339 gates with real arithmetic, 12 qubits with a permutation oracle, or 7 with one recycled control qubit.

103 ECDLP optimisations, measured

CORE 1 min

SAY One chart for the whole act: each bar is what a single optimisation saved on its own fixed task. Read the text on the right as the price tags: pseudo-Mersenne doubling is approximate, Jacobian leaves 3n garbage qubits, carry-save spends four times the qubits on depth. So Difficulty 8 is resolved: the same circuit solves the elliptic-curve discrete logarithm. The division makes its box costlier, and it is all Toffolis.

POINT TO the bars around 52 to 54 percent (the two adders and the dialog); then the Resolved box.

THEN Each step costs more for the curve. Does that make the curve the harder target? That is Act IX.

Act IX — Why ECDSA falls before RSA

QUESTION At equal classical security, are RSA-3072 and ECDSA on P-256 equally hard for Shor?

TIME 7 min (core 6 min).

LAND THIS The curve key is 12 times shorter and both attacks are cubic, so despite a heavier group operation an elliptic-curve key costs about two orders of magnitude fewer Toffolis: ECDSA falls first.

104 Act IX — what an attacker must solve

CORE 1 min

SAY Difficulty 9 is short, and it is the practical punchline. RSA-3072 and ECDSA on P-256 are rated equally secure against classical computers. Shor breaks both: factoring N gives the RSA private exponent, the discrete logarithm gives the curve's private key. So the only question is the price, and the price depends on n, which is $\log_2 N$ for RSA and $\log_2 p$ for the curve.

POINT TO the two grey boxes side by side, the last line of each: $n = \log_2 N$ against $n = \log_2 p$.

105 Same classical security, very different key sizes

CORE 1 min

SAY Factoring has a sub-exponential classical attack, the number field sieve, so RSA keys must be long. On a good curve no sieve works; the best attack is generic, Pollard's rho, about 2^{n/2} steps. So at 128-bit security the table pairs a 3072-bit RSA modulus with a 256-bit curve: twelve times shorter. That efficiency is why elliptic curves are used, and, as we will see, it is exactly what hurts them against Shor.

POINT TO the bold 128 row; the RSA column running away down the table.

106 The price of one group operation

CORE 1 min

SAY Both attacks have the same skeleton: 2n controlled group operations, $\Theta(n)$ qubits, $\Theta(n^3)$ Toffolis. Only the constant differs. For RSA the group operation is a modular multiplication; for the curve it is a point addition, with its division. On our circuits, at the same n, one point addition costs 6.1 to 6.7 controlled multiplications, and that ratio is constant in n.

POINT TO the two parallel curves on the log plot, purple above red; the ×6.7 labels between them.

107 At 128-bit security

CORE 1.5 min

SAY Now combine the two effects. The key is twelve times shorter and the cost is cubic, so 12 cubed, 1,728; divide by the 6.7 penalty and you expect about 260 times fewer Toffolis for P-256. Our circuits give about 270, minimal-width formulas from the literature 148, optimised literature about 180: two orders of magnitude, however you count. Qubits depend on the construction: 2.6 times fewer in like-for-like circuits, but RSA-2048 now fits in about 1,400 (Gidney 2025).

POINT TO the three pairs of bars, purple against red, on a log axis, with the ratio above each pair.

THEN (Asked about other key sizes? Backup B13.)

108 The gap grows with security

OPTIONAL 1 min

SAY And the gap is not a constant. To keep ahead of the sieve, an RSA key grows roughly like the cube of the security level, so Shor's cost grows like its ninth power, up to logarithms. A curve key is just twice the security level, so Shor's cost stays cubic. In minimal-width formulas the ratio goes from 19 at 80 bits to 733 at 192; on our circuits it is about 1,300 at 192.

POINT TO the red and purple lines pulling apart from left to right.

109 What it means

CORE 1.5 min

SAY

So here is the irony: the short key that makes elliptic curves efficient classically makes them the easier quantum target. The 2026 optimised estimates are 4 to 9 times 10^7 Toffolis for a 256-bit curve (Google, IonQ), against 6.5 times 10^9 for RSA-2048 (Gidney 2025). For key exchange, traffic recorded today can be decrypted later: harvest now, decrypt later. For signatures, once a large machine exists anyone can forge, so migrate to ML-KEM and ML-DSA. Difficulty 9 resolved: ECDSA falls first, with the same cubic cost, a 12 times shorter key and an about 7 times heavier group operation: 20 minutes against 5 days on the same hardware.

POINT TO

the peach box; then the two columns, ECDH and ECDSA; then the Resolved box.

Script 4 — Acts X and XI (slides 110–163)

About 54 min in full, 37 min core. Act X is self-contained: drop it for a cryptography audience.

Act X — Where gates come from: Hamiltonians and hardware

QUESTION Acts I–IX treated gates as matrices, but a laboratory only switches on lasers, microwaves and voltages, so which interaction makes which gate, on which hardware?

TIME 22 min (core 13.75 min).

LAND THIS Every platform reaches the same entangling gate by a different physical route, but the arithmetic has no native physics, so Shor costs its logical Toffoli count everywhere; the hardware only sets the error rate, the clock, the connectivity and the parallelism.

110 Act X — a laboratory switches on interactions, not matrices

CORE 1 min

SAY Until now a gate has been a matrix on a page. Nobody in a lab applies a matrix. You switch on a laser, a microwave tone or a voltage for a while, and the question of this act is which interaction gives which gate, on which machine. The one idea to keep is on the left: a gate is a Hamiltonian run for a time, and the exponent is the area of the pulse. For Pauli Hamiltonians the exponential is just a cosine and a sine, because every Pauli string squares to the identity.

POINT TO The Difficulty 10 box, then U = e^{−iHt} on the left and the cos/sin formula on the right.

THEN Let us rewrite the gates of this deck in that language.

111 The gates of this deck, as Hamiltonians

CORE 1 min

SAY Each row is a gate we have already used, written as an exponential, next to the interaction a lab would need for it. CZ and the controlled phases of the QFT are an Ising ZZ coupling; iSWAP is exchange; SWAP is the full Heisenberg coupling. The surprise is the first row. The T gate, the expensive gate of Act II, is physically the easiest: on superconducting chips a z rotation is just a change of frame, zero time and zero error. What we paid for in Act II was protection, not physics.

POINT TO The T row, then the "interaction" column.

THEN If T is free in the lab, why does a Toffoli cost seven of them?

112 Why a Toffoli costs seven T gates: a three-body phase

CORE 1 min

SAY Write CCZ as a phase on the product of three number operators. Expanded in Paulis, that product has seven terms, one for each non-empty parity of the three bits. They all commute, so the exponential splits exactly into seven π/8 rotations. Each is a T or T-dagger applied to a parity that a few CNOTs gather onto one wire: that is the seven-T Toffoli of Act II, derived rather than memorised. A genuine three-body interaction would make CCZ in one step, and the Rydberg blockade later in this act does exactly that.

POINT TO The row of seven purple boxes and the brace underneath.

THEN Adders have a Hamiltonian too. Multipliers do not.

113 Addition is a translation; multiplication has no Hamiltonian

CORE 1 min

SAY Draper's adder from Act IV has a neat physical reading: adding a constant is the translation operator, momentum generating a shift in position. In the Fourier basis it is a product of single-qubit phases, with no carries at all. Modular multiplication is different. It is a permutation whose generator is a dense, non-local matrix that no laboratory provides, so it gets compiled into Toffolis, and each Toffoli into magic states. That is the whole cost model of this deck in one line: the arithmetic has no native physics.

POINT TO The translation formula in the grey box, then the peach box.

THEN So which gates can a given device make at all?

114 Which gates can a device make? The Lie algebra decides

CORE 1 min

SAY A device has an always-on drift plus a few controls it can switch on and off. Brockett's result says: close those generators under commutators, and the reachable gates are the exponentials of what you get. The device is universal exactly when that algebra contains all of $su(2^n)$. Commutators come for free, because alternating two pulses forwards and backwards produces their bracket. The diagram is the smallest case: X and Z controls give Y, and with it every single-qubit gate.

POINT TO The diagram on the right: the two blue generators feeding the purple Y.

THEN Not every interaction passes this test.

115 Not every interaction is universal

OPTIONAL 45 s

SAY These dimensions were computed by closing each set of generators under commutators. Without an interaction you stay in independent single-qubit rotations. XX plus YY couplings only give so(2n), free fermions, which a classical computer can simulate. Add an Ising ZZ and you reach 15, 63 and 255, the full $su(2^n)$. Exchange alone is universal only on encoded qubits.

POINT TO The bold Ising row, against the free-fermion row above it.

THEN For two qubits there is a neat geometric picture.

116 Two-qubit gates are points in a tetrahedron

CORE 1 min

SAY Cartan's KAK decomposition says any two-qubit gate is local gates wrapped around one core, fixed by three numbers a, b and c. Same three numbers, same gate up to single-qubit gates, so every two-qubit gate is a point in this tetrahedron. CNOT, CZ, the ions' Mølmer–Sørensen gate and IBM's echoed cross-resonance gate all sit on the same corner. That is why a circuit compiled for one platform retargets to another for free. And no two-qubit gate needs more than three CNOTs.

POINT TO The red corner labelled CNOT = CZ = MS = ECR, then iSWAP and SWAP.

THEN Now the hardware, starting with how a single qubit is driven.

117 Driving a qubit: rotating frame and rotating-wave approximation

OPTIONAL 45 s

SAY In the lab frame a driven qubit looks messy. In a frame rotating with the drive it becomes a fixed field in the x–y plane, plus terms at twice the drive frequency that average out: the rotating-wave approximation. The phase of the drive picks the rotation axis and its area picks the angle. So every single-qubit gate is one resonant pulse, plus virtual Z gates that cost nothing.

POINT TO The Bloch sphere: the red axis set by φ, the blue vector rotating about it.

THEN Two-qubit couplings are subtler; most come from virtual transitions.

118 Effective Hamiltonians: interactions from virtual transitions

OPTIONAL 1 min

SAY Most entangling interactions are not wired in directly. There is a weak coupling to levels far away in energy, and a change of basis removes it to leading order. What is left is a correction of size V squared over the gap. It is a Lie bracket again, and it describes virtual hops through levels that are never actually populated. The table shows where this appears: transmon ZZ, cross-resonance ZX, exchange in quantum dots, light shifts in ions and atoms, even readout.

POINT TO The table on the right, one row per platform.

THEN First platform: superconducting circuits.

119 Superconducting circuits: the transmon

CORE 1 min

SAY A transmon is a capacitor plus a Josephson junction, which behaves as a nonlinear inductor. The cosine well is almost a parabola, so the levels are almost evenly spaced: a weakly anharmonic oscillator. The lowest two levels are the qubit, and the anharmonicity keeps pulses from exciting the third. With the parameters at the bottom, the qubit sits at 4.74 GHz with an anharmonicity of −287 MHz.

POINT TO The cosine well against the dashed parabola, and the gap between the blue levels and the red $|2\rangle$.

THEN That third level, the one we avoid for single-qubit gates, is what makes the CZ.

120 Transmon CZ: one full cycle through |20⟩

CORE 1 min

SAY Couple two transmons and tune one with flux until |11⟩ meets |20⟩. The two states repel; the plot shows the avoided crossing. Wait one full Rabi cycle there and |11⟩ comes back with a minus sign, while no other computational state has a partner, so nothing else picks up a phase. That is a CZ: 35 ns in simulation, and Google's Willow reports a mean CZ error of 0.33% across 105 qubits.

POINT TO The red gap at zero detuning.

THEN Fixed-frequency transmons cannot be tuned, so IBM uses another trick.

121 Fixed-frequency transmons: the cross-resonance ZX gate

OPTIONAL 45 s

SAY Drive the control qubit at the target's frequency. Through the exchange coupling the target then rotates about x, in a direction set by the control: a ZX interaction, which is a CNOT up to single-qubit gates. The echo cancels the unwanted terms and keeps ZX, and the exact rate, −59.9 kHz, matches the formula's −60.0. IBM's Eagle reaches a median error of 7.4×10^{-3} with this gate; Heron moved to tunable couplers and CZ and reaches 2.8×10^{-3}.

POINT TO The echo sequence: CR(+), X, CR(−), X.

THEN Trapped ions use something else entirely as the coupler: motion.

122 Trapped ions: a shared vibration as the quantum bus

CORE 45 s

SAY All the ions in a trap share their vibrational modes. A laser on the qubit frequency drives the carrier, which gives single-qubit gates. Tuned one phonon below or above, it drives the red or blue sideband, which flips the spin and changes the motion together. That entangles each ion with the shared motion, and through it with every other ion. The qubits are hyperfine levels that stay coherent for seconds.

POINT TO The three arrows: red sideband, carrier, blue sideband.

THEN The standard gate built on this bus is Mølmer–Sørensen.

123 The Mølmer–Sørensen gate: a geometric phase

OPTIONAL 1 min

SAY Two tones near the sidebands push the motion in a direction that depends on the spins. The motion traces a loop in phase space and comes back to where it started. Because the expansion stops at second order, what remains is a pure spin phase set by the enclosed area. Pick the detuning so the loops close and you land exactly on the CNOT point of the tetrahedron. The phase is twice the enclosed area, whatever the initial motion; that was checked to 10^{-12}.

POINT TO The closed loops: one turn for K = 1, three smaller turns for K = 3.

THEN Ions hold the accuracy records, and pay for them in speed.

124 Trapped ions: records and the price of accuracy

CORE 45 s

SAY Ions have the most accurate two-qubit gates: 99.99% fidelity, driven electronically on a chip, from Oxford Ionics and IonQ in 2025. Quantinuum's Helios runs 98 qubits at an error of 7.9×10^{-4}. The price is speed: gates of 10 to 200 microseconds plus shuttling, so an error-correction cycle takes milliseconds. Remember this for Act XI: IonQ's 19,397-ion Shor blueprint assumes exactly these 10^{-4} errors.

POINT TO The 99.99% in the grey box, then the 6.7 ms in the peach box.

THEN Neutral atoms get their interaction from a very different effect.

125 Neutral atoms: the Rydberg blockade

CORE 1 min

SAY Excite an atom to a Rydberg state, principal number around 50 to 70, and it becomes enormous. Two such atoms a few microns apart shift each other by hundreds of megahertz, so if one is excited the other cannot be: that is the blockade. From $|11\rangle$ only the shared state W is reachable, and the drive couples to it $\sqrt{2}$ times faster. Jaksch's original protocol, pulses π, 2π, π on atoms 1, 2, 1, turns this into a CZ.

POINT TO The $|rr\rangle$ level, pushed up by V and out of reach.

THEN Today the same CZ comes from one pulse on both atoms.

126 A CZ from one global pulse on both atoms

OPTIONAL 45 s

SAY Levine and colleagues drive both atoms with the same two pulses, the second with its laser phase jumped. $|01\rangle$ and $|11\rangle$ oscillate at different rates, Ω against $\sqrt{2}\,\Omega$, so the phase jump can bring both home with the right phases: $|11\rangle$ gets twice the phase of $|01\rangle$, plus π. That is a CZ. Because the pulse is global, every blockaded pair in the array gets the gate at once, and three atoms in blockade give a native CCZ.

POINT TO Both curves returning to zero at 2τ, and the dashed phase jump at τ.

THEN Where neutral atoms stand today.

127 Neutral atoms: records, and atoms that move

CORE 45 s

SAY Fidelity has reached 99.5% on 60 atoms in parallel, and 99.85% in 2026. The architectural advantage is that optical tweezers move atoms mid-computation, giving the long-range links that dense codes need, and many errors show up as detectable erasures. The cost is the clock: moves and imaging make the cycle about a millisecond. That millisecond is the clock of the Oratomic blueprint in Act XI.

POINT TO The Architecture box, then the 1 ms in the takeaway.

THEN Three more platforms, briefly.

128 Spins in silicon: exchange from tunnelling

OPTIONAL 45 s

SAY Two electrons in neighbouring quantum dots interact by tunnelling through a barrier controlled by a gate voltage. Only the singlet can hop virtually, and Schrieffer–Wolff turns that into an exchange coupling J. With equal Zeeman energies you get a SWAP; with a strong field gradient only ZZ survives and you get a CZ. The appeal is CMOS manufacturing: CZ gates at 99.04 to 99.56% now come from a 300 mm industrial foundry. The challenge is variability from device to device.

POINT TO The barrier gate between the two dots.

129 Photons: linear optics is a small Lie algebra

OPTIONAL 45 s

SAY For photons the Lie-algebra argument bites directly. Beam splitters and phase shifters only generate u(m), with m^2 dimensions, while N photons in m modes span a far larger space. So single-qubit gates are exact, but entangling gates are not: optics alone cannot reach them. The way out is measurement, today a Bell measurement called fusion. PsiQuantum reports 99.22% for fusion; photon loss is the main error.

POINT TO The Hong–Ou–Mandel picture: two photons in, always leaving together.

130 Cat qubits: one kind of error switched off

OPTIONAL 45 s

SAY A cat qubit stores the bit in two coherent states of a resonator, $|\alpha\rangle$ and $|-\alpha\rangle$, far apart in phase space. A bit flip needs tunnelling between them, which is exponentially suppressed; over 10 seconds has been measured. Phase flips still happen. With only one kind of error left, a repetition code is enough, and AWS ran one at distance 5 in 2025. That bias is behind the 13,436-qubit RSA and 126,133-qubit ECDLP estimates you will see in Act XI.

POINT TO The two blobs and the dashed "bit flip: rare" arrow.

THEN Where the error rate itself comes from.

131 Where the error rate p comes from

OPTIONAL 1 min

SAY Decoherence sets a floor: roughly gate time over coherence time. For a 40 ns CZ with the T1 and T_φ shown, that gives 5.6×10^{-4} for two qubits, a sixth of Willow's measured 0.33%. So most of today's error is control, not coherence. Control errors add in amplitude rather than probability, which is why random Pauli frames are used to make them stochastic. Leakage and loss are not Pauli errors at all, but if you detect them as erasures the surface code tolerates 4.15% instead of 0.94%.

POINT TO The worked numbers under the formula on the left.

THEN All the platforms side by side.

132 Six technologies, one table

CORE 1 min

SAY This is the act on one slide; read the columns as trade-offs. Transmons are fast, tens of nanoseconds per gate and about a microsecond per error-correction cycle. Ions are the most accurate, with gates of 10 to 200 microseconds and cycles of milliseconds; atoms gate in under a microsecond but cycle in about a millisecond. The grey footnote matters: these are reported numbers with different benchmarking protocols, often on two qubits rather than a full processor. And modulo single-qubit gates, nearly every row makes the same gate: the CNOT point.

POINT TO The "2q gate time" and "QEC cycle" columns, then the footnote.

THEN What all this means for Shor.

133 What the physics means for Shor

CORE 1.5 min

SAY Three conclusions. First, the cost model does not depend on the platform: T and even CCZ are physically cheap, but none of that survives encoding, so logical Toffolis still need magic states and Shor costs its logical Toffoli count everywhere. Second, the platform enters through four numbers only: error rate, cycle time, connectivity and parallelism. Third, those numbers spread the platforms out: superconducting qubits fast and local, ions accurate and connected but 10^3 times slower, atoms mobile and parallel on a slow cycle. That spread is exactly what we will see in the blueprints of Act XI.

POINT TO The Resolved box.

THEN Act XI: where these circuits stand in 2026, and the machines that would run them.

Act XI — The 2026 frontier and the machines

QUESTION The textbook circuits are the floor, so where is the 2026 ceiling, and what does a machine that runs it look like?

TIME 32 min (core 23.25 min).

LAND THIS Arithmetic has brought ECDLP-256 down to 4 to 9 × 10^7 Toffolis; what separates 10^4 from 10^6 physical qubits is the code, the clock and parallelism, and what the cheapest designs assume (long-range connectivity, real-time decoding, parallel surgery) is the least tested.

134 Act XI — where the circuits of this deck sit today

CORE 1 min

SAY Everything so far was the textbook floor. This act asks where the ceiling is in 2026, and what a machine that reaches it looks like. Four things changed in 2025 and 2026. The arithmetic for a 256-bit elliptic-curve key fell from 1.3 × 10^{11} Toffolis to 4 to 9 × 10^7; magic states got ten times cheaper; new codes need about 25 physical qubits per logical one instead of about 1,000; and four groups published blueprints all the way down to hardware. Before the machines, three slides of vocabulary.

POINT TO The four numbered changes on the left.

THEN First word: what a logical qubit is made of.

135 Vocabulary I — logical qubits are made of physical ones

CORE 1 min

SAY A code stores a few logical qubits in many noisy physical ones and catches errors on fewer than d of them, d being the distance. Below a threshold of about 1% physical error, the logical error falls exponentially in d; the standard assumption is $p = 10^{-3}$. The surface code uses one square patch of $2d^2 - 1$ qubits per logical qubit, with nearest-neighbour gates only. For a 10^{10}-Toffoli run that means $d \approx 27$, about 1,500 physical qubits per logical one.

POINT TO The takeaway formula: logical qubits times the ratio, plus factories.

THEN Next, the clock.

136 Vocabulary II — clocks, decoders and factories

CORE 1 min

SAY Each cycle measures all the checks: about a microsecond on superconducting chips, about a millisecond for atoms and ions. A classical decoder works out the error, and the reaction time, about 10 microseconds of decoding plus control latency, is what every non-Clifford gate waits for. Magic states come from factories: distillation turns 15 noisy copies into one, while cultivation, since 2024, grows one copy in a single patch for about ten times less. Logical operations are lattice surgery: merge two patches, measure their joint parity for about d cycles, split.

POINT TO The takeaway: runtime ≈ Toffolis × time per Toffoli.

THEN And the last piece: denser codes.

137 Vocabulary III — denser codes

CORE 1 min

SAY The surface code's $2d^2$ per logical qubit is forced by keeping every check local on a flat grid. qLDPC codes keep each check small but let it reach across the chip. IBM's gross code stores 12 logical qubits in 288 physical ones, where surface codes would need about 3,000. The price is long-range connections, slower access to qubits sharing a block, and harder decoding. One last term: a Pareto frontier is the set of estimates that nothing beats on both axes at once, and every plot from here on draws one.

POINT TO The gross-code numbers, then the takeaway.

THEN The first frontier comes from a public competition.

138 ECDLP-256: a competition on one point addition

CORE 1.5 min

SAY In March 2026 Google claimed a secp256k1 key falls to at most 1,200 logical qubits and 90 million Toffolis, but published only a zero-knowledge proof that such a circuit exists, not the circuit. In June Schrottenloher published open circuits with the same counts. From May to September, ECDSA.fail ran a public leaderboard on a single point addition, scored by qubits times Toffolis, and 100-plus people with their AI agents halved Google's score. The purple staircase is the leaderboard's frontier, as reported by the challenge. Mind the axis: Toffolis per point addition, not per key.

POINT TO The red Google dots and blue Schrottenloher squares near the 3B dashed line, then the purple diamonds near 1.5B, ending on the "1,151 / 1.30M" label.

THEN Let us open that middle entry.

139 Inside the leaderboard's 1,151-qubit entry

CORE 1 min

SAY First, exactly what is scored. The challenge measures one in-place point addition, R plus A, where the addend A is classical: it arrives as bits and is written into qubits only while it is used. As evaluated by the challenge, submission 8e9c9a2 peaks at 1,151 logical qubits and executes 1,299,453 Toffolis on average; these are the challenge's numbers, as it reports them. The grey box is the caveat to keep in mind for the whole walkthrough: no scalar registers, no Fourier transforms, and no coherent lookup of A. A full Shor circuit needs that lookup, and in this repo's build the loads and repair alone cost 197,364 Toffolis per addition at w = 16.

POINT TO The peach box, then the grey "Not in the submission" box.

THEN The submission runs Act VIII's seven updates as six stages.

140 Six stages, six optimisation stacks

CORE 1 min

SAY This is the map for the walkthrough. Each row is one stage of the addition: what it does to the X and Y registers, and the tricks stacked on it. Don't read the right-hand column now; look for the three ideas that keep coming back. Liveness: nothing is held longer than it is needed. Specialisation: everything is tuned to the one secp256k1 prime and to a classical addend; and measurement replaces running gates backwards wherever it can.

POINT TO The "update" column top to bottom, then the sentence under the table.

THEN Stage 1: the two coordinate differences.

141 Stage 1: coordinate differences through one transient register

CORE 1 min

SAY Stage 1 subtracts the addend's coordinates, so X becomes d_x and Y becomes d_y. Because the addend is classical, loading it costs no Toffolis: you put an X gate wherever a bit is 1, and the same X gates unload it. The two subtractions are independent, so they take turns in one 256-qubit register C, and only one coordinate copy is ever live. That is liveness in its simplest form.

POINT TO The two rows: allocate, load, subtract, unload, with the same C reused in the second row.

THEN Two details of the subtraction itself. (Short talk: go straight to 144.)

142 Stage 1 detail: subtract, then fix one borrow

OPTIONAL 1 min

SAY Subtracting on 256 bits leaves a final borrow when y is smaller than x. For a general modulus you would now compare against p. Here p is 2^{256} minus a small constant F, so 2^{256} is congruent to F, and the fix is to subtract F when the borrow is set: a controlled constant, not a comparison. The correction is applied only to the low 53 bits, and the borrow is cleared by a phase test on the top 19. The slide says it plainly: those truncations are validated on the benchmark's inputs, not proven for all inputs.

POINT TO The bit strip: purple top 19 bits, red low 53 bits; then the last line of the slide.

THEN And the internal carries are thrown away by measurement.

143 Stage 1 detail: venting a carry instead of uncomputing it

OPTIONAL 1 min

SAY A carry is an AND of two qubits, computed by a Toffoli into a fresh ancilla, and normally uncomputed by a second Toffoli. Instead, measure the ancilla in the X basis. That leaves only a sign on the $q_1q_2 = 11$ branch, when the outcome is 1, and a classically controlled CZ removes it exactly. Some internal borrows of the subtraction are vented this way; the final borrow stays live until the phase test. It is Act VII's temporary AND applied to a carry chain, and the same idea Gidney's adder in Act XII uses on every carry.

POINT TO The circuit left to right: Toffoli, use, H, measurement, then the CZ fed by the classical wire.

THEN Stage 2 is where most of the work happens: the division.

144 Stage 2: the slope by dialog-GCD division

CORE 1.5 min

SAY Stage 2 computes the slope λ, d_y divided by d_x modulo p. It runs a binary Euclidean walk on the pair (p, d_x), driving it towards (1, 0). Every decision of that walk is applied at the same moment to a second pair, Y and Z, which starts holding d_y and ends holding λ: the Bézout coefficients do the division. The decisions are also written into a compressed transcript τ, so the walk can be run backwards to restore X and clear its scratch. This is Act VIII's dialog, specialised.

POINT TO Top row: the walk, the τ box, the reverse walk. Bottom: the red Y wire going from d_y to λ, and the "live step controls" arrow joining the two.

THEN One step of the walk, close up. (Short talk: go straight to 150.)

145 Stage 2 detail: one Jump-2 step

OPTIONAL 1 min

SAY One step halves v, perhaps halves it again, then orders the pair and subtracts. Its decisions fit in three bits: subtract or not, swap or not, second halving or not. Allowing two halvings per step brings the walk down to 261 steps, where Act VIII's dialog takes 401. And only five of the eight possible three-bit symbols can actually occur.

POINT TO The six boxes left to right, ending in the record σ; then the five symbols in the right panel.

THEN Why do these decisions produce a quotient?

146 Stage 2 detail: the same decisions build the quotient

OPTIONAL 1 min

SAY When the walk finishes it has proved that $\alpha p + \beta d_x = 1$, so β is the inverse of d_x and d_y times β is λ. Each halving, swap and subtraction is a linear map M on the walk's pair, and the same live controls apply M's inverse transpose to (Y, Z), which carries d_y to λ. The decisions depend on the quantum value d_x, so they cannot be measured or thrown away: the submission keeps all 261 symbols, uses them live, and replays them backwards to restore X. This repo's own Jump-2, built from general cells, also takes 261 rounds.

POINT TO The two arrows: M on the walk, M^{-T} on (Y, Z).

THEN Keeping 261 symbols costs qubits, so they are compressed.

147 Stage 2 detail: a base-5 transcript

OPTIONAL 1 min

SAY Stored raw, 261 three-bit symbols would take 783 qubits. Since only five symbols occur, relabel them reversibly as base-5 digits and pack three digits into one number of at most 124. That fits in 7 qubits instead of 9, and the whole tape takes 609. Act VIII did the same for the dialog of eprint 1128: three pairs of bits have 27 values, so they fit in 5 qubits.

POINT TO The three panels left to right: five values, then $z = d_0 + 5d_1 + 25d_2$, then 609.

THEN The tape is also written and read a piece at a time.

148 Stage 2 detail: streamed replay

OPTIONAL 45 s

SAY Each symbol is used at once to update the slope, and each full group of three is encoded straight away, releasing two wires. As the walk proceeds, u and v shrink, and the tape grows into the space they free. On the way back only the current group is decoded, and the tape returns to zero. Note that τ is quantum and entangled with d_x: it is not a classical log.

POINT TO The bar at the bottom: shrinking u, v giving way to the growing tape.

THEN The widths follow a schedule.

149 Stage 2 detail: width scheduling and qubit reuse

OPTIONAL 45 s

SAY At step i only the low w_i bits of u and v are kept, and lanes known to be zero are released. The comparison behind each swap reads only the top k_i bits, and those windows were calibrated on the benchmark, not proven in general. The peak width at each step is the sum at the bottom: 512 for the Bézout pair, the tape, twice w_i, and a few work qubits. The schedule is tuned so that the falling 2w_i absorbs the growing tape.

POINT TO The underbraced formula for Q_i.

THEN Stages 3 to 6 are shorter.

150 Stage 3: prepare the X workspace

CORE 45 s

SAY Stage 3 adds 3x_A to X. The multiplication by 3 is done classically, on a laptop; the result is loaded with X gates, added in place, and unloaded. Why 3x_A? Because d_x + 3x_A is x_R + 2x_A, so subtracting λ^2 in the next stage leaves exactly x_A minus the new x coordinate. In a full Shor circuit this would be one more table in the lookup, which Act XII puts on SELECT-SWAP.

POINT TO The dashed x_A wire at the top, which is classical; then the note X = x_R + 2x_A.

THEN Stage 4 subtracts the square.

151 Stage 4: subtract the square, one Karatsuba piece at a time

CORE 1 min

SAY Stage 4 subtracts λ^2 from X. λ is split into a low half L and a high half H, and Karatsuba writes the square from three half-size squares: of L, of H and of their sum. The top power, 2^{256}, folds into the small constant F thanks to the special prime. Each square is built, used to update X, and unbuilt before the next, so only one is ever live. The three signed updates along the bottom add up to minus λ^2.

POINT TO The three columns, W, U and V built above and the X updates below; then the formula under the diagram.

THEN What makes each square cheap. (Short talk: go straight to 153.)

152 Stage 4 detail: build, apply, unbuild — and four specialisations

OPTIONAL 45 s

SAY Four specialisations stack here. Symmetric squaring needs 24,512 cross products for the three squares, 24.9% fewer than one full 256-bit square. Streaming means no 512-bit square is ever held. The fold turns $2^{256}V$ into FV, and F is sparse, so FV is five shifted additions. For comparison, this repo uses IonQ's squarer and fold instead, at 79,160 Toffolis; a Karatsuba split from its general cells cost more.

POINT TO The four coloured panels on the right.

THEN Stage 5 needs a multiplication, and gets it from the division.

153 Stage 5: multiply by running a fresh division backwards

CORE 1 min

SAY Stage 5 needs λ times X. Rather than build a multiplier, the submission reuses the division of stage 2. Division maps Y to Y over X; run that circuit backwards and it maps Y to Y times X. X now holds a different value, so a fresh walk records a fresh transcript, and the coefficient steps are applied in reverse. Y ends up holding λ times $(x_A - x_R')$.

POINT TO The two formulas, U_div and its inverse; then the final label on the red Y wire.

THEN What that trick buys, and what it does not. (Short talk: go straight to 155.)

154 Stage 5 detail: what running division backwards buys

OPTIONAL 45 s

SAY Be clear about the trade. It saves width: X, Y, Z and the tape are reused, with no extra 256-qubit product register. It does not save Toffolis: building and replaying the transcript is still a dominant cost. And the product is correct only if the 261-step walk converges, which is validated on the benchmark's inputs. It is the same trade as Act VIII's in-place multiplication.

POINT TO The two panels: width saved, Toffolis not.

THEN The last stage returns the output point.

155 Stage 6: recover the affine output

CORE 1 min

SAY The last stage turns the workspace into the output point. On Y: load y_A, subtract, unload, and Y holds the new y coordinate. On X we need x_A minus X, a subtraction the wrong way round. The trick: the complement of X plus (x_A + 1) is x_A − X modulo 2^{256}, so one complement and one vented addition do it, with x_A + 1 computed classically, and the secp256k1 fold finishes. By the end every loaded coordinate, carry and flag is gone.

POINT TO The X row: complement, vented add, fold; and the classical box computing c = x_A + 1.

THEN Now let us set this entry against what this repo builds.

156 The 1,151-qubit entry, read against this repo

CORE 1.5 min

SAY Side by side the totals look close: the entry's 1,151 qubits and 1,299,453 Toffolis on average, as the challenge reports them, against 1,457 qubits and 1,310,693 Toffolis in this repo. But they are not the same object. The entry adds a classical point, while this repo's windowed addition includes the coherent lookups and their repair, 197,364 Toffolis that the entry never pays. Nor is it the same evidence: the entry's walk length, comparison windows and 19-bit tests are validated on the benchmark's inputs, while this repo's cells are checked on every input at toy primes. What does carry over intact is the bottom line: the Jump-2 walk, the base-5 tape, streamed replay, the Karatsuba square and carry venting.

POINT TO The "what is counted" column, then the "of which lookups and repair" row.

THEN Back out from one addition to the whole key.

157 The logical frontier, September 2026

CORE 1.5 min

SAY Now Toffolis for one full key. The grey squares are the estimates from 2017 to 2023. The purple staircase is the frontier today: Luo et al. with the fewest qubits, 835, at 2.0×10^9 Toffolis; then Google, Schrottenloher and Google again; and IonQ at 1,457 qubits and 3.9×10^7 Toffolis. The red dashed corner is RSA-2048 for comparison, Gidney 2025 at 1,399 qubits and 6.5×10^9 Toffolis. RSA's arithmetic did not move in 2026; its progress was in the machines.

POINT TO The purple staircase from Luo down to the IonQ star, then the red dashed RSA corner.

THEN So what do those machines look like?

158 Five blueprints, one table

CORE 1.5 min

SAY These are the published end-to-end designs; read the last two columns. On superconducting surface codes, RSA went from 2.0×10^7 physical qubits in 2019 to 897,864 in 2025, and Google's elliptic-curve design needs under 500,000 and runs in 18 to 23 minutes. Denser codes on slower hardware trade the other way: Oratomic at about 26,000 atoms in a few days, IonQ at 19,397 ions in 25.7 days per attempt, assuming 10^{-4} errors. Read the footnotes: Oratomic's days assume many Toffolis in parallel, and each IonQ attempt succeeds 41 to 63% of the time.

POINT TO The "phys. qubits" and "runtime" columns, then the footnotes.

THEN Why do the runtimes differ so much?

159 What sets the time per Toffoli

CORE 1.5 min

SAY Runtime is Toffolis times the time per Toffoli, divided by how many run in parallel. The time per Toffoli is set by the slowest of three things: the reaction time, the factory output, and one lattice-surgery operation. Google's fast clock makes it reaction-bound at 15 microseconds; Gidney 2025 runs few factories to save qubits, so it is factory-bound at 62 microseconds. IonQ is bound by logical measurements and transport at 29.5 ms, and serial Oratomic by about 250 surgery cycles at 1 ms each. On a slow clock, only parallelism helps.

POINT TO The "bound by" column.

THEN Step back and look at a decade of these estimates.

160 A decade of estimates: physical qubits

CORE 1 min

SAY Physical qubits over time, red for RSA and blue for elliptic curves. The filled red points are RSA-2048 on the surface code, from Fowler's 10^9 down to Gidney–Ekerå and then Gidney's 2025 design; Pinnacle continues the line at around 10^5. That is 10^9 to 10^5 in fourteen years. Hollow markers are other codes and hardware, IonQ's ions among them, and that is where the lowest counts sit.

POINT TO The dashed red line from Fowler to Pinnacle, then the hollow blue IonQ marker.

THEN The logical side improved just as fast. (Short talk: go straight to 162.)

161 A decade of estimates: the logical cost of ECDLP-256

OPTIONAL 45 s

SAY For elliptic curves the Toffoli count fell from 1.3×10^{11}, Roetteler et al., to 3.9×10^7 in nine years. That is three and a half orders of magnitude from arithmetic alone, on roughly the same 1,000 to 2,500 logical qubits. The tools are the ones in this deck: windowing, measurement-based uncomputation, cheaper inversion and register sharing.

POINT TO The blue line, from Roetteler down to Google and IonQ.

THEN Where do all those physical qubits go?

162 What the physical count is made of

CORE 1 min

SAY Divide each design's total footprint by its logical qubits, factories and check qubits included. The ratio runs from 642 for Gidney's surface-code RSA down to 9 for Oratomic. Take IonQ's 19,397 ions apart: 73% is memory, 7% is four Toffoli factories, the rest is cleaning and measurement ancillas, and distance 9 is enough only because p = 10^{-4} is assumed. The lesson is that almost every logical qubit in Shor sits idle at any moment, so the density of the memory code is the main lever and the factories are a small slice.

POINT TO The bars from Gidney RSA at the top to Oratomic at the bottom, then the grey box.

THEN Last plot: qubits against time.

163 Physical qubits against time

CORE 1.5 min

SAY Physical qubits against runtime in days, red for RSA and blue for elliptic curves, with each staircase a frontier: fewer qubits cost more days. Read it with care, because the points mix clocks 10^3 apart and error rates 10 times apart. Counted in qubits times cycles, Pinnacle and serial Oratomic are within 2 to 3 times of the surface-code baselines; the points about 100 times cheaper assume many logical operations at once. So the Resolved box names what is least tested: long-range connectivity, real-time decoding and parallel surgery.

POINT TO The two staircases; Google's blue square at the lower right; the IonQ star.

THEN Act XII: the 2026 optimisations, built and measured in this repo.

Script 5 — Act XII and Closing (slides 164–211)

About 47.5 min in full, 26 min core. Act XII has the most optional slides in the deck: in a short talk keep the core slides and the three plain-words recaps (207–209), which carry the act on their own.

Act XII — The 2026 optimisations, built and measured

QUESTION Act VIII built the eprints as printed at 2.6M Toffolis per point addition, while IonQ reports 1.39M on 1,457 qubits, so where is the factor 2?

TIME 45 min (core 23.5 min).

LAND THIS The factor 2 is three ideas, all reproduced here, and with IonQ's adder, signed windows and SELECT-SWAP the built circuit reaches 1.31M executed Toffolis on 1,457 qubits, below IonQ's own count.

164 Act XII — from the papers as printed to below IonQ's count

CORE 1.5 min

SAY This act answers one question with circuits I actually built and counted. Act VIII built the two eprints as printed: 2.6 million Toffolis per point addition. IonQ reports 1.39 million on 1,457 qubits, so where is the factor of two? The table is the short answer: one windowed point addition falls from 6.28 million to 1.72 million, and the whole ECDLP-256 from 213 million to 48 million. Three changes do it: measured uncomputation, a cheaper GCD, and a squarer.

POINT TO The Difficulty 12 box, then the two multiplications row (3.9M to 1.44M), the biggest saving. The five modular additions do not move (5,115). Last row: 34 to 28 additions.

THEN Before the ideas, how I know these numbers are right.

165 How every optimisation is checked

CORE 45 s

SAY Every optimisation is a switch beside the construction it refines. Switch it off and you get the old circuit, gate for gate. At toy primes I check every basis input and every ancilla back to zero, and a regression test pins every count I quote. Measurements are the hard part, because the exhaustive simulator cannot see phases, so a sparse state-vector simulator runs the real measure-and-repair and checks the phase on every basis input. An approximate repair fails exactly on the inputs the tests predict, no more.

POINT TO The grey box on the right.

THEN First switch: the lookups.

166 1. Lookups: ten recomputed → three loads and one repair

CORE 1.5 min

SAY Before, every table lookup was undone by running it again: ten lookups, each of size $L = 2^{16}$. But X-measuring a loaded register costs nothing. It only leaves a phase on the address, and a phase lookup repairs that in about $2\sqrt{L}$ ANDs instead of L. IonQ add one more step: the three unloads share one address, so their tables are XORed and a single repair serves all three.

POINT TO Top row: ten vermillion L boxes. Bottom row: three loads, each followed by an X measurement, then one purple phase-fix box. Peach box: 1,310,700 → 197,364 Toffolis per addition.

THEN That was the cheap part. Two thirds of the cost is the multiplier.

167 2. The GCD: conditionally inverted steps

CORE 1.5 min

SAY Two thirds of a point addition is the in-place multiplier, and most of that is the walk and replay of the binary GCD from Act VIII. IonQ's change is a re-encoding: store $\tilde{v} = v + (1-a)u$, so both registers stay odd. Then the next parity is two CNOTs, and each step is one uncontrolled adder between two rows of CNOTs: n−1 ANDs, where [1128] spends 2n on a controlled subtraction. The recorded bits are exactly the dialog's, so the replay and the packing carry over unchanged.

POINT TO The code: a row of CNOTs, one add, the same row of CNOTs. Peach box: 1,955,276 → 720,565 per in-place multiplication.

THEN How it compares with the other GCDs.

168 Five GCDs behind one interface

OPTIONAL 45 s

SAY One module holds five GCDs behind one interface, so they can be compared on the same multiplier. The dialog as printed costs 1,955,276; fused 77-bit compares and the width schedule bring it to 865,588; IonQ's walk gets to 720,565. ECDSA.Fail's ping-pong is cheapest per round but needs 704 rounds, and Jump-2 needs only 261 rounds but costs 1.16 million in all. One warning: the ping-pong budget of 2.75n rounds is not a worst case, because exhaustively n = 16 needs 3.9n.

POINT TO The rounds column against the Toffolis column; bold 720,565.

169 The GCD, as a circuit: walk, record, replay

OPTIONAL 1 min

SAY This is the multiplier as a circuit. The walk runs 401 rounds of conditionally inverted adds on u and $\tilde{v}$ and writes its decisions into the purple record; the replay then applies 401 signed steps to y. Afterwards the walk runs backwards to clear u and $\tilde{v}$, and a second walk unrecords, leaving $x \cdot y \bmod q$ in place. Two details come back later: the record is packed at 5 Toffolis a round and grows into the qubits the narrowing walk frees, and IonQ's replay cell costs 352 Toffolis a step against 712. One sum, $x + y = p$, defeats that cell, so the first 37 steps keep an exact "careful" cell.

POINT TO Left: the vermillion "ci add" boxes on u and $\tilde{v}$, the purple record cells below. Right: the blue "±, /2" boxes on r, s and y, fed by the record.

THEN The third change: squaring.

170 3. Squaring: a dedicated squarer and a pseudo-Mersenne fold

CORE 1 min

SAY Step 10 of the point addition subtracts λ^2, and with the general multiplier that is two multiplications. IonQ's squarer rewrites x^2 as a sum of signed terms, one per bit, each an unconditional subtraction on a window of n+1−i bits. That costs ½n(n+3) ANDs, with no controlled adder anywhere. The reduction uses the shape of the prime: $2^{256} \equiv 2^{32} + 977$, so the overflow folds back into the accumulator. Measured: square-subtract goes from 1,050,623 to 79,160.

POINT TO The vermillion staircase: each row is one signed window, shifted by 2i+1 and subtracted from the grey register z. Then the peach box.

171 The squarer, in code

OPTIONAL 30 s

SAY Do not read the code; look at its shape. For each row i, CNOTs from x_i flip h so the operand becomes ±(h_i + x_i), one subtraction hits the window of z at offset 2i+1, and the same CNOTs flip h back. The sign costs only CNOTs, so the subtraction itself is never controlled.

POINT TO The two CNOT loops around `A.sub`.

172 Building 256 bits on a laptop: one gate per shape

OPTIONAL 45 s

SAY A 256-bit point addition is tens of millions of gates but few distinct pieces: the replay calls one modular adder 800 times. So a tracing wrapper builds each call once per shape and appends it as one opaque gate. Shape means register widths, shared qubits and classical parameters, and running a gate backwards inverts it. It equals the flat count at toy size in 8 configurations, and Qualtran agrees on the exported bloqs.

POINT TO The cache lookup on `key`, and the last line, which appends one `NodeGate`.

THEN With that, the first full-size builds.

173 The first full-size builds

CORE 1.5 min

SAY These are the first full 256-bit builds, read top to bottom. Qualtran's ECAdd from Litinski 2023 is the reference at 8.35 million; [1128]'s dialog with exact arithmetic is 5.16 million, and each indented refinement below it cuts further. IonQ's walk and replay with the squarer reach 1.71 million, and the whole algorithm builds too: 28 additions, 48 million Toffolis on 2,251 qubits. Two gaps remain: the best row is 23% above IonQ's count, and every row needs more qubits than any paper reports.

POINT TO Bold 1,711,194; bold 1,873 in the qubits column, the narrowest row; the takeaway naming the two gaps.

THEN First, the Toffoli gap.

174 4. Matching IonQ: the gap is in the replay

CORE 1.5 min

SAY To find the 23%, I split the count by component. The lookups, the walks and the squarer are already IonQ's; the Bézout replay is not. Each of its 800 steps adds ±r into s modulo p, and the cell costs 712 Toffolis, of which only 256 are the addition; the rest are top-bit tests: reduce or not, exactly p, was the input zero? Replacing that cell takes one addition to 1,426,965 executed, within 2.5% of IonQ's 1,392,608, and the 28-addition algorithm to 40.3 million against IonQ's 39.0 million.

POINT TO The Bézout replays row, 833,312 → 564,086, and the signed-additions row under it. Lookups, walks and square-subtract do not change.

THEN The cell itself.

175 IonQ's modular adder: the carry is the flag

CORE 1.5 min

SAY This is the idea I like best in the act. Add x into y with a carry-out: if the sum passed 2^{256}, subtracting p is just "drop the carry, add $f = 2^{32} + 977$", which moves only the low 65 bits. No comparison decides the reduction; the carry is the flag. Afterwards the carry equals $[y' < x]$, so it can be X-measured, and half the time a 32-bit comparator on the top bits repairs the phase. That is 352 Toffolis worst case, 336 counting what executes, against 712.

POINT TO The ADD box; the carry wire controlling +f on the low κ bits; the X meter; the classical wire into the phase box, applied only if m = 1.

THEN It is approximate. Next, how approximate.

176 IonQ's modular adder, in code

OPTIONAL 30 s

SAY Look for three things: the add whose carry lands in `ay`, the controlled constant add on the low κ bits, and the `mbu_flag` call at the bottom, which measures the carry and runs the comparator. The cell is approximate only to order f/p or 2^−δ. At toy primes it is wrong on exactly the sums in $[p, 2^n)$ and phase-correct everywhere else, with every measurement performed.

POINT TO The first `A.add` line, the `cadd_const` line, and the last line.

177 The one case that is not rare: 37 careful steps

OPTIONAL 1 min

SAY One of those rare cases is not rare. The replay must pass through zero: dividing, the first swap moves a zero into s; multiplying, the same step adds x + y = p exactly, the one sum the carry cannot see. When it happens is fixed by the 2-adic valuation of x, so it falls within the first k steps with probability 1 − 2^−k. So the first 37 steps keep the exact cell, the same 37 IonQ report. Multiplying also gets its own replay, because the inverse of "measure, sometimes repair" is a recompute that always runs.

POINT TO The purple line halving per step; the red dashed line at 37, labelled $2^{-37} = 7{\times}10^{-12}$. Bottom line: signed additions 569,600 → 307,574.

THEN Now a step past IonQ.

178 Signed windows: half the table, and never O

CORE 1.5 min

SAY Read the w address bits as an odd signed digit: with w = 3, addresses 0 to 7 become the odd digits −7 to 7. The lookup only needs |d|, so the table halves, 2^{15} entries instead of 2^{16}, and the sign becomes a negation of y before and after. Consecutive windows merge their negations into one, controlled by the XOR of the signs. And an odd digit is never zero, so the addend is never O, the point at infinity. Measured: 1,328,915 executed per addition, 4.6% below IonQ, and 2,745,400 fewer Toffolis over the 28 additions.

POINT TO Addresses on top, digits below: the blue half is handled by negating y, the vermillion half is the table of 4. Then the peach box.

179 Signed windows, in code — and the condition they need

OPTIONAL 45 s

SAY In the code, look at the `neg` calls: the first window negates once, and each later window closes the previous negation and opens its own with one negation, controlled by the XOR. The condition is y ≠ 0, which holds when every visited point has odd order. secp256k1 has prime order, so it qualifies. On an order-43 toy curve, over every input, one run finds k 43% of the time, against an ideal 77%.

POINT TO The two `neg` calls and the `cx(ps, s)` pair around the second one.

180 IonQ's count, then IonQ's width, in one table

CORE 1 min

SAY This is the running total, one switch per row. The phase-approximate adder and signed windows bring the Toffolis down; then packing, sharing, lean cells and CNOT ends bring the width from 2,233 to exactly IonQ's 1,457 without adding Toffolis. SELECT-SWAP on the 3x lookup gives the last step: 1,310,693 executed, against IonQ's published 1,392,608 on the same 1,457 qubits. The whole ECDLP-256 is 37.1 million Toffolis on 1,457 qubits, 5.9% below IonQ per addition.

POINT TO The qubits column dropping to 1,457 on the fourth row; the bold last row against the "published: IonQ" row.

181 Where the 1 457 qubits go

OPTIONAL 45 s

SAY The peak sits in one replay cell: the record of 669, y and the accumulator at 512, the 16-qubit window, and 260 of scratch. While I chased it, the peak moved four times, and none of the fixes adds a Toffoli. The careful cell frees its scratch after its adder, 259 qubits instead of 647. The replay's copies became n CNOTs, the fold's 8 subtractions run on the phase adder, and the walk's u is shared with the record.

POINT TO The bar, left to right: purple record, vermillion y and accumulator, the thin blue window, grey scratch.

182 5. SELECT-SWAP lookups: trade qubits that are free anyway

CORE 1 min

SAY SELECT-SWAP trades qubits for Toffolis inside a lookup. Look up on the high address bits to write 2^k words at once, then k layers of Fredkins on the low bits move the addressed word into place. The other words are a known function of the address, so they are X-measured into the same merged repair. The extra qubits live only during the load, and at the 3x load only 798 are live, so k = 1 there is free under the 1,457 peak. Further out it stops being free: k = 3 reaches 1.25 million executed but needs 4,635 qubits.

POINT TO The diagram: vermillion lookup, grey words, blue Fredkin layers. In the table, the bold row (0, 1), then the last row.

183 SELECT-SWAP, in code

OPTIONAL 30 s

SAY The loop is the whole idea: layer i swaps block j with block j + 2^i when address bit i is set, so after k layers block 0 holds the addressed word. Then `unlookup` measures every block out and repairs all the phases in one pass.

POINT TO The `cswap` inside the triple loop, and the `unlookup` line.

184 6. Fewer qubits: 1 246 on lean cells, 851 on three field registers

CORE 1.5 min

SAY Now the other axis: width. Each row adds one option, so read the table as a price list. Packing and sharing reach 1,863 qubits almost for free; CDKM adders and lean cells reach 1,246, but at 2.82 million Toffolis instead of 1.71. Luo's register-shared Euclid reaches 1,107 for 86 million, and three field registers reach 851. For scale, 1,246 is 4.87n, against 1,192 from [1128]'s formula, 1,457 for IonQ and 835 for Luo et al.

POINT TO The bold 1,246 row; the jump in the Toffolis column at Luo's row; the footnote's breakdown of 1,246.

THEN How the peak came down.

185 Chasing the peak: take the scratch out of whatever sets it

OPTIONAL 1 min

SAY The method is simple: the qubit count is a statement about one moment, so find the step that sets the peak and take the scratch out of it. The left column shows where the peak sat as it moved: replay, square-subtract, dialog walk, replay again, and last the squarer. The cells that came out are on the right: CDKM adders, a comparator with one ancilla, an all-ones test in $\sqrt{k}$ chunks, and a constant adder whose carry goes into an increment on borrowed qubits. Each new cell gives the same output as the one it replaces on every input tried, failures included.

POINT TO The purple chain on the left, with the reason beside each box.

186 An increment on borrowed qubits

OPTIONAL 45 s

SAY g plus its complement is all ones. So subtract a dirty register g, flip it, subtract again and flip back: w gains one, whatever g holds, and g ends as it began. That is Gidney's trick from 2015. It needs no clean scratch, because the qubits are borrowed from registers that are idle at that moment.

POINT TO The two −g boxes and the two X gates on g; the identity under the circuit.

187 A controlled CDKM adder at 3n Toffolis, not 4n

OPTIONAL 1 min

SAY This one corrected a package. The package documented the textbook 3n Toffolis but put the control on every MAJ and every UMA, which is 4n. The fix: compute the carries unconditionally, one Toffoli a bit, and unwind with a UMA that writes the sum bit only under the control, two more. With the control off, b returns to y.

POINT TO The vermillion gate, the only one that touches ctrl.

188 The controlled CDKM adder, in code

OPTIONAL 30 s

SAY Two loops: the carries ripple up through plain `_maj`, which never sees the control, and come down through `_cuma`, where only one Toffoli uses `ctrl`. The comments give each register's contents on entry.

POINT TO The `ccx(ctrl, c, b)` line in `_cuma`.

189 Below 1 000 qubits: share the lanes, vent a register

CORE 1 min

SAY Getting below a thousand qubits takes two ideas. Luo's extended Euclid lets remainders and cofactors share lanes, so the record disappears, but each quotient bit costs a shifted comparison: 30 times the Toffolis for 139 qubits. Venting: once z = y/x exists, y is a function of the others, so I X-measure it and reuse its qubits for the backward Euclid. The phase that leaves is cancelled by recomputing y = xz around a Z^b. The tests check that the phase cancels, and that without the Z^b it does not.

POINT TO The takeaway: `vent`, `zfix`, and the negative test.

190 The register-shared Euclidean algorithm (Luo et al.)

OPTIONAL 1 min

SAY Here is the shared bank: n+3 qubits, the purple remainder shrinking while the vermillion cofactor grows into the freed lanes. ECDSA.Fail's narrowest circuit uses this instead of the dialog, so there is no record. The table is the trade at n = 256: division in 1,091 qubits against the dialog's 2,088, but 42.8 million Toffolis against 1.75 million. The inversion count is Luo's formula, exactly.

POINT TO The three bars, start, middle and end; then the bold n = 256 row.

191 Three field registers, measured

OPTIONAL 1 min

SAY Across the top is the whole division: Euclid forward, z = y/x, vent y, Euclid back on y's qubits, then recompute y with the Z fix. The division alone is 835 qubits, exactly Luo et al.'s $3n + 6\lfloor \log_2 n \rfloor + 19$, and the signed windowed addition adds 16 window qubits for 851. The price is 85.2 million Toffolis per addition and 2.39 billion for the whole algorithm. On Gidney's adder the same circuit is exact for every odd p, at 835 qubits and 44.2 million Toffolis.

POINT TO The two purple boxes, vent and fix; bold 835 and 851 against the two published rows.

192 Three field registers, in code

OPTIONAL 30 s

SAY Read the code as the flowchart: `eea_forward`, `mul_acc` for y/x, the vent, the inverse Euclid, then `mul_acc`, `zfix`, `mul_unacc`. Without the Z^b the same circuit fails the phase check, so the check really sees what the vent does.

POINT TO The three-line `mul_acc` / `zfix` / `mul_unacc` block.

193 7. Gidney's classical-quantum adder: x += d on two clean qubits

OPTIONAL 1 min

SAY The three-register circuits need to add a classical constant d with almost no clean qubits. Gidney's adder vents each carry by X-measurement once its bit is summed, because the carries can be reproduced later by adding d into the complement of the result. The leftover phases are undone on a dirty register by Z, carry-XOR, Z, carry-XOR. A vent cannot run backwards, so subtraction and halving are built forwards. Measured: 762 Toffolis on n−1 dirty and 2 clean qubits, and Luo's modular addition on it costs 2,813.

POINT TO The peach box.

194 The vented adder, in code

OPTIONAL 30 s

SAY In the loop, each next carry is a majority computed with one AND, and once x_k is summed the current carry is vented and freed. So only two carry qubits are ever live.

POINT TO The `and_` line, and the "vent c_k" comment near the bottom.

195 The carry-XOR, derived rather than copied

OPTIONAL 1 min

SAY The repair needs every carry XORed into a dirty register g, and I derived that pass rather than copied it. The carries obey a recurrence with one product per bit, so one pass P over g gives w XOR a linear, invertible Q of g. Run Q inverse, then P, and g picks up exactly w; XOR in x to turn w into the carries. That is about 2n Toffolis with no ancilla, and since d enters only through X and CNOT gates, a control on it is free. The table matches Luo et al.: 11.0n for modular addition and 6.0n for doubling, against their 11n and 6n.

POINT TO The two displayed equations, the grey box, then the table's last two rows.

196 The carry-XOR, in code

OPTIONAL 30 s

SAY The code follows the derivation: the descending loop is Q inverse, the next loop is P, the last turns w into c. In the adder each carry goes into g as it is made, so one carry-XOR finishes the repair, 3n in all; with no dirty qubits, each half of the target serves the other, at 4n.

POINT TO The comments on the three loops.

197 8. Depth: carry-lookahead adders

OPTIONAL 1 min

SAY Until now I counted Toffolis, but on an error-corrected machine time is Toffoli depth. Ripple adders and the GCD's sequential rounds make the depth 0.8 to 0.95 times the count. [106] uses the Draper–Kutin–Rains–Svore adder: carries as a prefix over generate and propagate pairs, all known after $\log_2 n$ rounds. At n = 256 that is depth 31 instead of 255, for 1,723 Toffolis instead of 255.

POINT TO The purple prefix tree; bold 31 in the table.

198 Depth of one point addition: spend lookahead where the depth is

CORE 1.5 min

SAY Carry-lookahead adders find all the carries in $\log_2 n$ rounds at a higher Toffoli count, so spend them where the depth is. Start from ripple adders: 2.0 million Toffolis, depth 1.5 million. Switch the replay, then the walk, then the squarer, and the depth falls to 394,281; register sharing then brings the width back to 2,223 at about the same depth. Two things make it work: fan shared controls out with CNOTs first, because a qubit is in one gate at a time, and measure the depth exactly from the cached hierarchy instead of estimating it. Net: depth divided by 3.8 for 4.6 times the Toffolis.

POINT TO The Toffoli depth column falling while the Toffolis column rises; bold 394,917; the takeaway.

199 The depth frontier, and the floor the lookups set

OPTIONAL 1 min

SAY Every build on one plot: grey for ripple, purple for carry-lookahead. The dashed line is a hard floor: unary iteration over 2^{16} entries is 2^{16} steps deep, and three lookups make 196,608, so no arithmetic gets below it. The second lesson: reusing a qubit orders the steps that use it. The 1,457-qubit circuit is 33.6 million deep in all; the 2,233-qubit one, 29.5 million.

POINT TO The dashed vermillion floor, and the purple squares, the closest any build gets to it.

200 9. Catalogue: a looked-up addend in Jacobian coordinates

OPTIONAL 1 min

SAY Section 9 is a catalogue: built and measured, but not in the headline circuit. First, [106]'s Algorithm 4 in Jacobian coordinates, with the addend looked up rather than classical. Per addition it looks cheaper: 175,250 controlled, against 262,797 for affine. But a cheaper addition is not a cheaper oracle: a projective triple depends on the path that made it, so the whole chain must be undone before anything is read. That doubles the additions, and at toy size the projective oracle costs about twice the affine one.

POINT TO The strip at the bottom: the affine chain, then the projective chain with "read" in the middle and the blue undo steps after it.

201 Catalogue: twisted Edwards curves

OPTIONAL 45 s

SAY Twisted Edwards coordinates have a complete addition law, with no exceptional cases at all, and need 7 multiplications where Jacobian needs 11. With a classical addend that is 3.2 times fewer Toffolis and 26% fewer qubits, at both sizes. But secp256k1 is not an Edwards curve, so this is a catalogue entry, not a step of the ECDLP-256 circuit.

POINT TO The bold Edwards Toffolis column against the Weierstrass column.

202 Catalogue: batch inversion

OPTIONAL 45 s

SAY Litinski shares one inversion across k independent affine additions, through the running products in the diagram. Built clean, with every intermediate uncomputed, it costs 2 inversions plus 5(k−1) multiplications instead of 2k inversions. The table says it plainly: always fewer Toffolis, always more qubits. And it only pays on the Kaliski path, because in the dialog a multiplication costs as much as an inversion.

POINT TO The chain of products; bold 1,503,226 against 2,814,928 at n = 64, k = 8.

203 Catalogue: ECDSA.Fail's fire census

OPTIONAL 45 s

SAY ECDSA.Fail prunes by census: run a sample, record which Toffolis ever fired, delete the rest. Over every input that is exact dead-code removal; over a sample it is a guess. On the q = 61 dialog multiplier, 10 samples leave 98% of fresh inputs wrong and 300 still leave 2.8%; only every input gives zero. At n = 256 the dead code is the $2.4\sqrt{n}$ margin rounds, and a census finds only what they never did on the inputs it saw.

POINT TO The "wrong" column.

204 10. RSA at size: Gidney 2025's residue arithmetic, loop for loop

OPTIONAL 1 min

SAY To compare with RSA on equal terms, I built Gidney 2025's residue arithmetic, loop for loop. Per prime of the residue system: look up and add the discrete logs, reduce, exponentiate by windowed multiplications, and subtract a truncated CRT term into an accumulator. At 2048 bits the build gives 6.95 billion Toffolis against the paper's 6.5 billion, and 1,467 qubits against 1,399. At toy N the circuit equals a bit-exact model on every exponent; at size, one prime's iteration is built and multiplied by the number of primes.

POINT TO The bold 2048 row, built and paper columns side by side.

205 ECDLP against RSA, on one surface-code model

CORE 1.5 min

SAY Now everything on one surface-code model: physical error rate 10^{-3}, 1 µs cycle, 10 µs reaction, six CCZ factories. It reproduces Gidney's 897,864 qubits and 4.96 days, which is my sanity check. ECDLP-256 with IonQ cells needs 2.14 million physical qubits for 15 minutes; the 851-qubit version, 1.32 million for 0.73 days; RSA-2048, 0.97 million for 2.1 days. At comparable width, secp256k1 costs 189 times fewer Toffolis than RSA-2048, and 537 times fewer than RSA-3072, its classical-security equal.

POINT TO The physical-qubits and time columns; then the takeaway.

206 The built frontier

CORE 1.5 min

SAY This plot is the act in one picture. Every grey dot is a circuit built here at n = 256, and Qualtran counts the same Toffolis. The purple staircase is the built frontier, the fewest Toffolis at each width; the crosses are published points from [1128], Google, IonQ and ECDSA.Fail. The frontier ends at IonQ's cells plus SELECT-SWAP: 1,457 qubits and 1.31 million executed, below IonQ's own point. So the factor two between [1128] and IonQ is three ideas, all reproduced.

POINT TO The step at 1,457, just under the IonQ cross; the labels for 851 and 1,246 further left; the Resolved box.

THEN Three tables recap the act in plain words.

207 Act XII in plain words (1/3)

CORE 45 s

SAY *(Walk the rows in about ten seconds without reading them, then stop on two.)* Each optimisation gets one line: what it does, and in vermillion, what it bought. If you keep one row from this page, keep the first: measuring instead of recomputing took the lookups from 1.31 million to 197,364. And the carry-is-flag row: the replay step from 712 to 352 Toffolis, because the adder's own carry decides the reduction.

POINT TO The vermillion column; rows 1 and 4.

208 Act XII in plain words (2/3)

CORE 45 s

SAY *(Same drill: scan, then stop on two or three rows.)* Signed windows are the row that took us past IonQ: half-size tables, and 4.6% below their count. Peak chasing is the other: 2,233 down to 1,457 qubits without a single extra Toffoli. Lean cells show the price of going narrower still: 1,246 qubits, but 2.82 million Toffolis.

POINT TO Rows 1, 2 and 4 in the vermillion column.

209 Act XII in plain words (3/3)

CORE 45 s

SAY *(Scan, then stop on two rows and the footnote.)* Venting is the idea that gets below a thousand qubits: three registers, 851 qubits, at 85.2 million Toffolis per addition. Carry-lookahead goes the other way: depth from 1.50 to 0.39 million, for 4.6 times the Toffolis. And note the footnote: the catalogue entries are built and measured, but they are not in the headline circuit.

POINT TO Rows 2 and 4; the grey footnote.

THEN What building all this turned up in the sources.

Closing

TIME 2.5 min.

LAND THIS Shor's cost is reversible arithmetic, and in 2026 ECDLP-256 came down to about 10^3 logical qubits and $4–9\times10^7$ Toffolis.

210 What reproducing the sources turned up

CORE 1 min

SAY Building everything, rather than quoting it, turned up four things in the sources. The GCD's top-bit comparisons need $k + 2.3\sqrt{n}$ bits, not a flat k: a flat 24 at $n = 64$ fails 15 of 40 multiplications, and our own first 256-bit builds used a flat 48 until the $n = 64$ simulation caught it. IonQ's replay needs its swap before the complemented adder, plus a $p \to 0$ repair the text does not name; and ECDSA.Fail's ping-pong budget is not a worst case. In Gidney 2025, the hot-qubit count of 131 evaluates to 152, and 9.1 shots is 9.2. That is why every number in this act is the gate count of a circuit that exists, verified exhaustively at toy primes and pinned by a regression test.

POINT TO The first bullet, then the takeaway.

THEN Seven things to keep.

211 Seven things to keep

CORE 1.5 min

SAY Shor is phase estimation of a group operation: the quantum part is short, the cost is reversible arithmetic, and uncomputing the garbage is mandatory. Price it in T gates, 7 per Toffoli, and optimise in order: rotations first, then the counting register, then the additions. Elliptic curves fall first: the same $\Theta(n^3)$, but a key 12 times shorter makes them 150 to 270 times cheaper. Gates are Hamiltonians, and every platform reaches the CNOT point. And the frontier moved in 2026: ECDLP-256 on about 10^3 logical qubits, which is half a million superconducting qubits for minutes, or twenty thousand ions or atoms for weeks.

POINT TO Item 7 last; leave the slide up for questions.

Backup slides (B1–B18)

Not part of the timed talk. Each backup answers one likely question: when it comes up, jump to the slide in the appendix of the PDF, give the short answer below, then return to where you were.

B1 Why the QFT ends with swaps

BACKUP

USE WHEN someone asks why the QFT circuit ends with swaps, or whether they cost anything.

SAY Output qubit q needs a phase that depends only on the low n−q bits of j, and the in-place loop builds it on qubit n−1−q, so the order comes out reversed, as the n = 3 table shows. The swaps are bookkeeping, 3 CNOT and 0 T, and are absorbed by relabelling before a measurement, inside an adder, or in the semiclassical QFT.

B2 Measured counts against the formulas

BACKUP

USE WHEN someone asks whether the gate-count formulas are fitted or exact, or how the factoring circuit scales with n.

SAY For N from 15 to 129 the measured qubits equal $4n+2$, the Toffolis equal $8n^3+22n^2$, and the controlled phases equal their formula, digit for digit. The plot shows the same counts against n; the legend gives slopes of 2.7 for Toffolis, 3.8 for controlled phases and 4.1 for small rotations.

B3 Counting in code

BACKUP

USE WHEN someone asks how gates are priced, or how a count is produced.

SAY The counter prices each gate by the table on the right: a CCX or CSWAP is one Toffoli and 7 T, a temporary AND is 4 T with a free uncompute. For the order-finding circuit with a = 7, N = 15 it prints 18 qubits, 864 Toffolis, 7,068 controlled phases and 12,855 rotations, then 14,309 T, or 978,434 with rotations included at 10^{-3}.

B4 Draper's adder in Qiskit

BACKUP

USE WHEN someone wants to see the Fourier-space constant adder as code.

SAY Three steps: a QFT on y, one phase per qubit proportional to X·2^(i−n), and the inverse QFT. The controlled version only swaps `p` for `mcp`: the controls go on the phases, not on the QFTs.

B5 The modular adder in Qiskit

BACKUP

USE WHEN someone asks how the Beauregard modular adder works step by step.

SAY The seven commented lines are the seven steps: add X under the controls, subtract N, copy the sign into a flag, add N back if the flag is set, then subtract and re-add X so the flag can be cleared from the answer. Only step 1 adds under the outside controls; step 4 adds under the flag.

B6 The ripple-carry adder in Qiskit

BACKUP

USE WHEN someone asks what a CDKM ripple-carry adder looks like in code, or where its controls go.

SAY MAJ up the chain, UMA back down. The controls reach only the two CNOTs into b of each MAJ and UMA, as in Act VII.

B7 The ripple-carry modular adder: control on two blocks

BACKUP

USE WHEN someone asks how the modular adder changes with ripple-carry instead of Fourier adders.

SAY The same seven steps as the Fourier version, but each constant is first written into an ancilla register with uncontrolled X gates and then added by the ripple adder. The title is the point: the control lands on two blocks only, the first +X under the controls and the +N under the flag.

B8 Factoring end to end, retrying

BACKUP

USE WHEN someone asks what happens when the measured order is useless, for example odd r or a trivial factor.

SAY Pick a random a; if gcd(a, N) is already a factor, stop. Otherwise run order finding with $t = 2\lceil \log_2 N \rceil$ counting qubits, and if r is even try gcd(a^(r/2) − 1, N); if that fails, pick a new a and go round again.

B9 The garbage demo is one extra line

BACKUP

USE WHEN someone asks how the deck shows that uncomputation is mandatory.

SAY The demo is the normal circuit plus one loop: a CNOT from each counting qubit into a garbage register, leaving a copy of x behind. That one line is enough to erase the peaks, which is item two of the seven things to keep.

B10 ECDLP: votes for k

BACKUP

USE WHEN someone asks how k is recovered from noisy two-register measurements.

SAY Each measured pair (j1, j2) is rounded to a candidate pair modulo the order, with a small search around it, and every invertible a1 gives a candidate $k = a2 \cdot a1^{-1}$. Candidates collect votes weighted by their counts and down-weighted by distance from the rounded point; the top vote wins.

B11 ECDLP end to end

BACKUP

USE WHEN someone wants to see the whole ECDLP pipeline on a toy curve.

SAY The curve is $y^2 = x^3 + 5x + 4 \bmod 7$ with a generator of order 5, and the public key is Q = [4]G. One call builds the circuit, runs it on Aer and post-processes the counts, then prints the recovered k and the qubit count.

B12 How the circuits are verified

BACKUP

USE WHEN someone asks how we know the circuits are right.

SAY Three levels: arithmetic exactly, on every basis input with every ancilla back to zero; distributions exactly, closed-form P(y) against Aer to 10^{-14}; and samples statistically. Test the distribution, not the factors: a broken circuit gives a flat histogram, and the classical checks can still print the right factors by luck.

B13 Elliptic curves against RSA, across key sizes

BACKUP

USE WHEN someone asks whether the ECC advantage holds at other key sizes or security levels.

SAY Each row pairs comparable classical security, using the minimal-width logical counts of Roetteler–Naehrig–Svore–Lauter and Häner–Roetteler–Svore, both 2017. At every level ECC needs fewer qubits and far fewer Toffolis, for example 1.3×10^{11} for 256-bit ECC against 1.9×10^{13} for RSA-3072, and error correction multiplies both sides by comparable factors.

B14 Reproduce every number

BACKUP

USE WHEN someone wants to rerun the results themselves.

SAY Make a virtual environment, install the requirements, then run the tests in three modes: fast, full with end-to-end factoring on Aer, and the ECDLP suite. The last two lines regenerate this deck's figures and its code listings, checked.

B15 References, Acts I–IX

BACKUP

USE WHEN someone asks for a source from the first nine acts.

SAY The classics are here: Shor, Beauregard, Draper, Cuccaro et al., Gidney's temporary AND and windowing, Gidney–Ekerå, Litinski, Roetteler et al., and the two 2026 eprints.

B16 References, the optimisations (Acts VII, VIII, XII)

BACKUP

USE WHEN someone asks where a specific optimisation comes from.

SAY Grouped as factoring, ECDLP and Act XII. The Act XII lines cite IonQ, ECDSA.Fail, Luo et al., Litinski, Low–Kliuchnikov–Schaeffer, Draper–Kutin–Rains–Svore for carry-lookahead, and Gidney's 2015 note on increment gates.

B17 References, Act X (Hamiltonians and hardware)

BACKUP

USE WHEN someone asks for sources on control theory, gate decompositions or a hardware platform.

SAY Control theory first, then two-qubit gate decompositions, then roughly one line per platform: superconducting, ions, neutral atoms, then spins and photons together, and finally cat and erasure qubits.

B18 References, Acts XI–XII

BACKUP

USE WHEN someone asks for the 2026 resource-estimate papers or the architecture proposals.

SAY Act XI's papers are Babbush et al., Schrottenloher, Luo et al., ECDSA.Fail and IonQ, then architecture work such as Pinnacle and Oratomic. Act XII adds Gidney and Low–Kliuchnikov–Schaeffer, plus Parts VII–VIII of the tutorial and the bench directory.